

Table of Contents

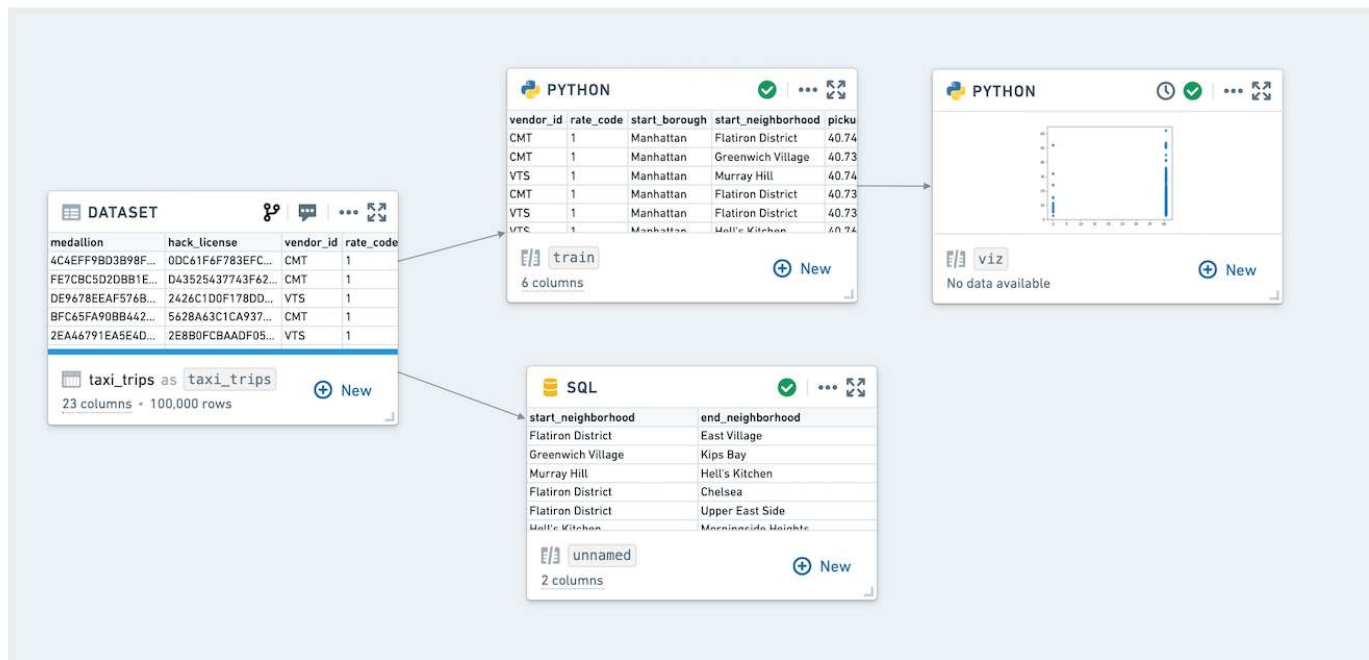
1. [Code Workbook \[Legacy\] • Overview • Palantir](#)
2. [Code Workbook \[Legacy\] • Overview • Palantir](#)
3. [Code Workbook \[Legacy\] • Getting started • Palantir](#)
4. [Code Workbook \[Legacy\] • Core concepts • Palantir](#)
5. [Code Workbook \[Legacy\] • Keyboard shortcuts • Palantir](#)
6. [Code Workbook \[Legacy\] • FAQ • Palantir](#)
7. [Code Workbook \[Legacy\] • Workbooks • Overview • Palantir](#)
8. [Code Workbook \[Legacy\] • Workbooks • Supported languages • Palantir](#)
9. [Code Workbook \[Legacy\] • Workbooks • Inputs and outputs • Palantir](#)
10. [Code Workbook \[Legacy\] • Workbooks • Console • Palantir](#)
11. [Code Workbook \[Legacy\] • Workbooks • Global code • Palantir](#)
12. [Code Workbook \[Legacy\] • Workbooks • Duplicate nodes • Palantir](#)
13. [Code Workbook \[Legacy\] • Workbooks • Moving to production • Palantir](#)
14. [Code Workbook \[Legacy\] • Transforms • Overview • Palantir](#)
15. [Code Workbook \[Legacy\] • Transforms • Visualize data • Palantir](#)
16. [Code Workbook \[Legacy\] • Transforms • Present visualizations • Palantir](#)
17. [Code Workbook \[Legacy\] • Transforms • Access unstructured files • Palantir](#)
18. [Code Workbook \[Legacy\] • Transforms • Spark • Palantir](#)
19. [Code Workbook \[Legacy\] • Transforms • Transforms FAQ • Palantir](#)
20. [Code Workbook \[Legacy\] • Templates • Overview • Palantir](#)
21. [Code Workbook \[Legacy\] • Templates • Getting started • Palantir](#)
22. [Code Workbook \[Legacy\] • Templates • Multi-node templates • Palantir](#)
23. [Code Workbook \[Legacy\] • Templates • Suggested templates • Palantir](#)
24. [Code Workbook \[Legacy\] • Environment • Overview • Palantir](#)
25. [Code Workbook \[Legacy\] • Environment • Code Workbook profiles • Palantir](#)
26. [Code Workbook \[Legacy\] • Environment • View resolved environment • Palantir](#)
27. [Code Workbook \[Legacy\] • Environment • Batch builds and interactive builds • Palantir](#)
28. [Code Workbook \[Legacy\] • Environment • Environment creation overview • Palantir](#)
29. [Code Workbook \[Legacy\] • Environment • Session history and session pinning • Palantir](#)
30. [Code Workbook \[Legacy\] • Environment • Troubleshooting guide • Palantir](#)
31. [Code Workbook \[Legacy\] • Branching • Overview • Palantir](#)
32. [Code Workbook \[Legacy\] • Branching • Getting started • Palantir](#)
33. [Code Workbook \[Legacy\] • Branching • Merging branches • Palantir](#)
34. [Code Workbook \[Legacy\] • Branching • Choose imported dataset branch • Palantir](#)
35. [Code Workbook \[Legacy\] • Datasets • Optional data persistence • Palantir](#)
36. [Code Workbook \[Legacy\] • Datasets • Project references • Palantir](#)
37. [Code Workbook \[Legacy\] • Code repositories • Comparison: Code Repositories vs. Code Workboo](#)
38. [Code Workbook \[Legacy\] • Code repositories • Exporting to a Code Repository • Palantir](#)
39. [Code Workbook \[Legacy\] • Code repositories • Hidden code repositories • Palantir](#)

Code Workbook [Legacy]

⚠ Legacy

Code Workbook is in the [legacy](#) phase of development, and no additional development is expected. Full support remains available. We recommend exploring other applications and tools to serve your use case purposes.

Code Workbook is an application that allows users to analyze and transform data in code using an intuitive graphical interface.



Code

[API Reference ↗](#)

Code Workbook was designed with these principles in mind:

[Send feedback](#)

- **Iteration speed:** Users can quickly test and refine logic for transformation and visualization in order to produce useful results.
- **Low barrier to entry:** Code Workbook's graphical interface and the ability to reuse pre-authored logic were designed to enable easy onboarding of users of varying technical skill.
- **Collaboration:** Disparate groups of users can share logic and work together on a single analysis in Code Workbook.
- **Platform interoperability:** Users can productionize their findings across the platform by adding visualizations to Notepad, promoting production-ready pipelines to Code Repositories, and iterating on machine learning models.

Features

Key features of Code Workbook include:

- An **interactive console** for rapid iteration on transformation logic and ad-hoc data exploration.
- **Visualization support** to create detailed, interactive images with commonly used packages (Matplotlib, Plotly, Seaborn).
- **Templates** that enable reuse of complex and domain-specific logic through a simple interface.
- **Multi-language support** for code and template languages to allow users to select the best language for an analysis.
- **Branching** to facilitate collaboration by isolating users' changes from one another.
- A **point-and-click user interface** that makes it easy to customize the Spark environment, set input types for transforms, and view relationships between nodes.

PREVIOUS

← [Quiver / Detailed card index / Year to date](#)

NEXT

[Getting started](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

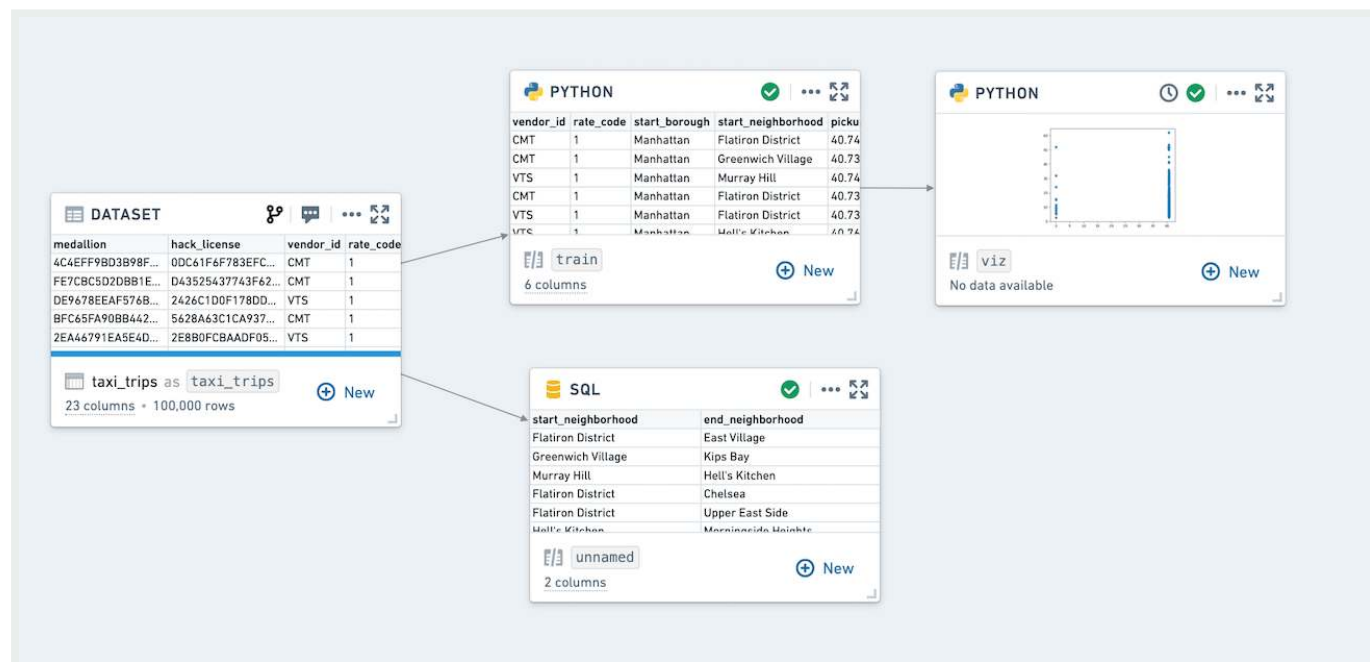
[Send feedback](#)

Code Workbook [Legacy]

⚠ Legacy

Code Workbook is in the [legacy](#) phase of development, and no additional development is expected. Full support remains available. We recommend exploring other applications and tools to serve your use case purposes.

Code Workbook is an application that allows users to analyze and transform data in code using an intuitive graphical interface.



Code
API Reference ↗

Code Workbook was designed with these principles in mind:

Send feedback

- **Iteration speed:** Users can quickly test and refine logic for transformation and visualization in order to produce useful results.
- **Low barrier to entry:** Code Workbook's graphical interface and the ability to reuse pre-authored logic were designed to enable easy onboarding of users of varying technical skill.
- **Collaboration:** Disparate groups of users can share logic and work together on a single analysis in Code Workbook.
- **Platform interoperability:** Users can productionize their findings across the platform by adding visualizations to Notepad, promoting production-ready pipelines to Code Repositories, and iterating on machine learning models.

Features

Key features of Code Workbook include:

- An **interactive console** for rapid iteration on transformation logic and ad-hoc data exploration.
- **Visualization support** to create detailed, interactive images with commonly used packages (Matplotlib, Plotly, Seaborn).
- **Templates** that enable reuse of complex and domain-specific logic through a simple interface.
- **Multi-language support** for code and template languages to allow users to select the best language for an analysis.
- **Branching** to facilitate collaboration by isolating users' changes from one another.
- A **point-and-click user interface** that makes it easy to customize the Spark environment, set input types for transforms, and view relationships between nodes.

PREVIOUS

← [Quiver / Detailed card index / Year to date](#)

NEXT

[Getting started](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Getting started

Getting started

Set up the dataset

This tutorial uses a publicly-available dataset ([download titanic_dataset.csv](#)) containing information about passengers on the Titanic.

The dataset includes name, age, sex, and other identifying information about passengers on the Titanic. Navigate to Foundry and open your personal Project. Create a folder named `Code Workbook Tutorial` and upload the Titanic dataset there, naming it `titanic_dataset`.

Set up a Code Workbook

Click the **New** button in the Project and select **Code Workbook** to create a new Workbook.

Import a dataset

Click **Import Dataset** to begin. In the dialog that appears, search for `titanic_dataset`. Choose the file you created during the setup process, which should be in `/user/Code Workbook Tutorial/titanic_dataset`.

Once you have identified the desired dataset, click on the file and then click Select to add the dataset to the graph.

[API Reference ↗](#)

Transforming data using Python

Send feedback

Now that `titanic_dataset` is imported into the workbook, we can transform it using code and pieces of reusable logic. Add a downstream transform by hovering over `titanic_dataset` and clicking the **+** sign. This brings up a dropdown showing various transformation options - choose Python.

The screenshot shows the Foundry interface. On the left, the 'Contents' panel lists 'titanic_dataset' as a dataset. The main workspace shows a graph with a node for 'titanic_dataset' (12 columns, Row count not computed). A dropdown menu is open, showing options: Python code, R code, SQL code, Templates, and Visualize. The 'Python code' option is selected. Below the dropdown, a preview table is visible, showing columns: PassengerId (Integer), Survived (Integer), Pclass (Integer), Name (String), and Sex (String). The table contains 7 rows of data.

PassengerId	Survived	Pclass	Name	Sex
1	0	3	Braund, Mr. Owen Harris	male
2	1	1	Cumings, Mrs. John Bradley	female
3	1	3	Heikkinen, Miss. Laina	female
4	1	1	Futrelle, Mrs. Jacques Heath	female
5	0	3	Allen, Mr. William Henry	male
6	0	3	Moran, Mr. James	male
7	0	3	McCarthy, Mr. Timothy J	male

A Python code node now appears on the graph, with a connecting line showing it is a child of `titanic_dataset`. Give this transform an alias, `titanic_filtered`, by clicking into the text box at the top of the logic tab.

By default, newly created transforms are not saved as datasets in Foundry. You can choose to save the results of a transform as a dataset by clicking on the **Save as dataset** toggle. [Learn more about saving transforms as datasets.](#) Transforms that are saved as datasets have two names: the alias, and the name of the Foundry dataset.

Using a Pandas dataframe

If you are comfortable using Pandas syntax, you are able to use Pandas in Python. You can use `titanic_filtered` to use Pandas.

[API Reference](#)

[Send feedback](#)

First, we need to change the input type of `titanic_dataset`. Click into the Inputs tab in the logic panel and expand the sidebar. You'll see that the Input Type is set as Spark dataframe. Click the dropdown and select Pandas dataframe to change the input type of `titanic_dataset` to a Pandas dataframe.

Next, let's update our code to work with a Pandas dataframe. We'll perform the same filter.

```
1 def titanic_filtered(titanic_dataset):  
2     output_df = titanic_dataset[(titanic_dataset['Survived'] == 1) & (tita  
3     return output_df
```

This code will output a Pandas dataframe with the female Titanic survivors.

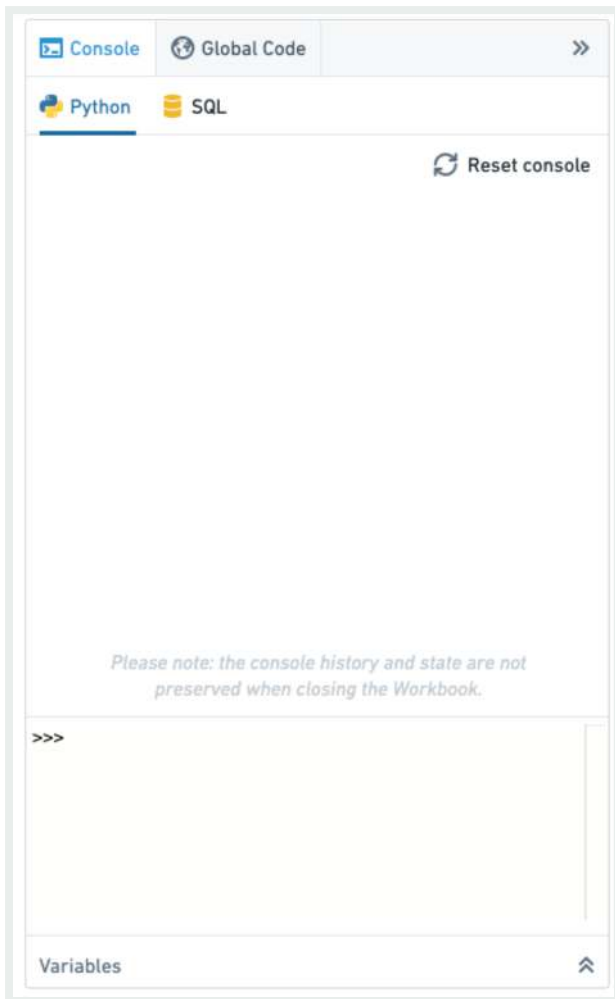
Using Console

The console provides a REPL (read-evaluate-print loop) for Code Workbook, enabling rapid, ad-hoc analysis of any transform or input dataset on the graph. To allow for quick iteration in your preferred language, there is a console for each enabled language in your workbook.

Open the console, located on the right hand side of the page. Choose the Python console:

[API Reference ↗](#)

[Send feedback](#)



You can quickly experiment with the data by executing commands in Python. You can also send code from a transform to run in the console by highlighting the code and using the [keyboard shortcut](#) `Cmd+Shift+Enter` (macOS) or `Ctrl+Shift+Enter` (Windows).

First, you must import the following PySpark SQL functions in the Python console:

```
1 import pyspark.sql.functions as F
```

Then, determine the maximum age of female Titanic survivors:

```
red.select(F.max('Age')).show()
```

Send feedback

You can also use the SQL console to calculate the same statistic:

```
1 SELECT max(Age) AS max_age FROM titanic_filtered
```



← PREVIOUS
Overview

NEXT →
Core concepts

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Core concepts

Workbooks

The main resource you interact with in Code Workbook is a **Workbook**. Workbooks are used to import datasets from Foundry and transform these input datasets for purposes such as:

- Cleaning and joining raw data imported from some external source to produce curated datasets for other users.
- Analyzing processed datasources to derive useful insights.
- Training and applying models to do predictive analysis.
- Creating parameterized visualizations to display in a report.

[Learn more about workbooks.](#)

Transforms

Transforms are pieces of logic that take one or more inputs and return a single output. Inputs and outputs can be Foundry datasets or models.

[Learn more about transforms.](#)

Templates

API Reference ↗

able users with a range of technical experience to collaborate by
code away behind a simple form-based interface. Values

Send feedback

substituted into a code template, which can then be run like any other transform in the Workbook.

[Learn more about templates.](#)

Environment

Each Code Workbook is associated with an **environment**. An environment includes a set of Conda packages and Spark settings installed on the Spark module backing computation in the Workbook.

[Learn more about environments.](#)

Branching

Branching in Code Workbook provides a version control experience tailored to data transformation, enabling teams to operate on logic and data simultaneously in a Workbook.

[Learn more about branching.](#)

← PREVIOUS
Getting started

NEXT →
Keyboard shortcuts

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

API Reference ↗

Send feedback

Keyboard shortcuts

This page lists the current macOS and Windows keyboard shortcuts available in Code Workbooks.

Select the **Help** menu in the upper left corner, then select **⌘ Keyboard shortcuts** to view a pop-up with all keyboard shortcuts. Alternatively, you can view all keyboard shortcuts by using **Shift+?**.

macOS shortcuts

Editor

- Run selected code in console: **Cmd+Shift+Enter**
- Open completion suggestions: **Tab**
- Close full screen viewer or editor / clear new node: **Esc**
- Open full screen editor: **Cmd+Alt+E**

Execute

- Run selected dataset: **Cmd+Enter**
- Run all saved datasets: **Cmd+Alt+Enter**

API Reference ↗



Selected node: **Del**

Send feedback

- Delete selected nodes: `Backspace`
- Zoom to selected nodes: `Space`
- Layout all graph nodes: `Cmd+Shift+A`
- Layout all graph nodes by color group: `Ctrl+Shift+G`
- Layout selected graph nodes: `Cmd+L`
- Select next node: `Right arrow`
- Select previous node: `Left arrow`
- Select all graph nodes: `Cmd+A`

Workbook

- Import dataset: `Cmd+Alt+R`
- Open branch menu: `Cmd+B`
- Search in workbook: `Cmd+Shift+F`
- Close all helper panels: `Ctrl+Cmd+C`

Windows shortcuts

Editor

- Run selected code in console: `Ctrl+Shift+Enter`
- Open completion suggestions: `Tab`
- Close full screen viewer or editor / clear new node: `Esc`
- Open full screen editor: `Ctrl+Alt+E`

Execute

- Run selected dataset: `Ctrl+Enter`
- Run all saved datasets: `Ctrl+Alt+Enter`

[API Reference ↗](#)

Graph

[Send feedback](#)

- Delete selected node: `Del`
- Delete selected nodes: `Backspace`
- Zoom to selected nodes: `Space`
- Layout all graph nodes: `Ctrl+Shift+A`
- Layout all graph nodes by color group: `Ctrl+Shift+G`
- Layout selected graph nodes: `Ctrl+L`
- Select next node: `Right arrow`
- Select previous node: `Left arrow`
- Select all graph nodes: `Ctrl+A`

Workbook

- Import dataset: `Ctrl+Alt+R`
- Open branch menu: `Ctrl+B`
- Search in workbook: `Ctrl+Shift+F`
- Close all helper panels: `Ctrl+Ctrl+C`

← PREVIOUS
[Core concepts](#)

NEXT →
[FAQ](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Code Workbook FAQ

The following are some frequently asked questions about Code Workbook.

For general information, view our [Code Workbook documentation](#).

- [Can I easily move all of my work from one workbook to another?](#)
- [Can I set a transformation node to be a variable?](#)
- [Why does each transformation node need to return a table or a dataframe?](#)
- [Can Code Workbook handle merging and merge conflicts?](#)
- [How long does it take to initialize my environment?](#)
- [How do I restore code I previously deleted from my code workbook?](#)
- [Spark environment and custom libraries](#)
 - [Cannot find required Python library](#)
 - [Trouble importing Python libraries](#)
 - [Failed to create environment](#)
- [Python code is failing](#)
 - [Retryer error](#)
- [Failed to save as dataset](#)
- [Workbook not automatically recomputing after updating input dataset preview](#)
- [Unable to merge my branch into master](#)
- [Make cell computation faster](#)
 - [1. Refactoring](#)
 - [2. Caching](#)
 - [3. Function calls](#)
 - [4. Downsampling](#)

[API Reference ↗](#)



Send feedback

Can I easily move all of my work from one workbook to another?

Yes. You can copy-paste nodes (elements on the graph of a workbook) between workbooks.

1. Hold `Ctrl` while selecting the nodes you would like to copy from one workbook. Copy-pasting Contour board nodes is not supported.
2. Copy the selected nodes using the right-click context menu.
3. Select the graph of your target workbook and paste the nodes using the combination `Cmd+V` (macOS) or `Ctrl+V` (Windows). The new nodes will be imported into your new workbook.

[Return to top](#)

Can I set a transformation node to be a variable?

No. Each transformation node on a code workbook graph must return a table or a dataframe (such as a two-dimensional data structure with columns).

[Return to top](#)

Why does each transformation node need to return a table or a dataframe?

The atomic unit of artifacts within Foundry is the dataset. Each transformation node needs to return a table or a dataframe (such as a two-dimensional data structure with columns) so that each node can be registered as a Foundry dataset and therefore available throughout the rest of Foundry. Moreover, the tables or dataframes in Code Workbook must be returned

with the following constraints: column names are not duplicated, column names do not contain invalid letters, and so on.).

[API Reference ↗](#)

[Send feedback](#)

Can Code Workbook handle merging and merge conflicts?

Yes. For more information, view the documentation on [branching and merging](#).

How long does it take to initialize my environment?

Code Workbook's default configuration has an average initialization time of 3-5 minutes. However, if you have added additional packages to your Code Workbook profile, initialization times may range from 20-30 minutes, depending on the complexity and interdependencies of these packages. Initialization time tends to increase significantly as the number of packages in the environment increases.

Slow initialization generally indicates that the environment definition is too large or too complex. Initialization time tends to increase superlinearly with the number of packages in the environment, so you may want to simplify any custom environments. In some cases, Code Workbook may pre-initialize commonly-used environments to speed up initialization. If you have created a custom environment based on a default profile, the slower initialization time may be due to the lack of pre-initialization. Learn more about [optimizing the initialization time of a custom environment](#)

If the browser tab is inactive for more than 30 minutes, the environment may be lost due to inactivity.

How do I restore code I previously deleted from my Code Workbook?

If these transforms were built into a dataset, you can use the **Compare** feature of the resulting Dataset Preview to view the code at that time. From there you can copy-paste the relevant transforms. Unfortunately, if this code was in an intermediate transform, it cannot be recovered.

Code Workbook is a more iterative platform than Code Repositories, the latter of which has a full git commit and publish functionality. If you have any other branches available, we recommend checking them for the deleted transform.

[Return to top](#)

Spark environment and custom libraries

Cannot find required Python library

By default, any Python package available in [Conda Forge](#) is available to add to a custom environment for your workbook. If the Python library is included in Conda Forge, then you can customize your environment to include it directly.

To troubleshoot, perform the following steps:

1. Select **Environment > Customize Environment** in the top-right.
2. Search for the desired library and add either **automatic** (which will take upgrades, but may leave you vulnerable to unintentional breaks if the module upgrades) or a specific version of this library.
3. Select **Update Spark environment**.

Note that it will take a while for the environment to reload, working with custom libraries. This process will be slower than with the stock environment, since a pool of stock environments is kept "warmed" whereas each custom environment is created from scratch.

[API Reference](#)

[Send feedback](#)

Trouble importing Python libraries

Sometimes it may be required to use a library that is not already available within Code Workbook. It is possible to have these added to your available list, but this requires some hands-on work.

To troubleshoot, perform the following steps:

1. Confirm that the Python library is not available in the workbook custom Spark environment list.
2. If it does not exist, then vet whether another available library might provide the same functionality.
3. If necessary to include this library, contact Palantir Support.
4. Once the library is available, you can refer to the previous troubleshooting steps to add a library.

Failed to create environment

I am trying to update my workbook environment, and it states "Waiting for Spark / Initializing environment" for a while and then errors out with a "Failed to create environment" message.

To troubleshoot, perform the following steps:

1. Confirm whether there were any updates to customize the workbook's Spark environment. If there have been, then your investigation should center on any custom libraries added.
2. ~~Ensure none of the~~ modules were pinned to a specific version. It is generally good [API Reference ↗](#) the latest version of a module.
3. Remove any custom libraries and see if the Spark environment loads.

4. If a particular library is preventing your Spark environment to load, escalate to Palantir Support.

[Return to top](#)

Python code is failing

This section discusses failures that are generally specific to Code Workbook.

For additional information, you may also refer to our guidance on [Builds and checks errors](#).

[Return to top](#)

Retryer error

Running an import package or any basic command returns the following error:
"com.github.rholder.retry.RetryException: Retrying failed to complete successfully after 3 attempts. at com.github.rholder.retry.Retryer.call(Retryer.java:174)". When using a workbook in Pandas, the Code Workbook application will still convert from a Spark dataframe to a Pandas dataframe before your transformation, which consumes significantly more memory on the driver and is likely making it OOM.

To troubleshoot, perform the following steps:

1. Verify that your transformation cell is using Pandas and that this problem is exhibited for code that specifically runs Pandas.
2. If this computation is possible in Spark, we strongly recommend re-configuring your code to use Spark since it is parallel and more scalable.
3. As a last resort when this computation must be done on the driver, we recommend using a profile that increases driver memory.

[Return to top](#)

API Reference ↗

Send feedback

Failed to save as dataset

This issue could occur for a variety of reasons, but the most common circumstance is with returning a table or dataframe that contains a valid schema. If none of the below steps help identify the particular error you are seeing, refer to our guidance on [Builds and checks errors](#).

To troubleshoot, perform the following steps:

1. Ensure the transformation node is returning a table or dataframe.
2. Confirm the table or dataframe conforms to the basic requirements of being written to Foundry as a dataset: For example:
 1. There is at least one column.
 2. All column names are valid (such as they do not contain spaces or invalid characters).
 3. There are no duplicate column names.
3. If you are confident this transformation node should be capable of being written out as a dataset, then try running another transformation node or building a new, very simple transformation node.
 1. This can help identify if it is a local issue (something unexpected happening with that particular transformation node, for example) or a more systemic issue (such as transformation nodes in general are failing to save to Foundry).
 2. If it is a local issue, continue to debug the code being used to create this table / dataframe.
 3. If it is systemic, escalate the issue to Palantir Support providing all the relevant information from your investigation.

[Return to top](#)

Workbook not automatically recomputing after updating input dataset preview

API Reference ↗

When opting to **Update table preview** for input tables, only the view for this table is updated and the underlying dataset in Foundry is not automatically r

Send feedback

To troubleshoot, perform the following steps:

1. If the input datasets do not refresh after selecting **Update table preview** for input tables, then:
 - Open these datasets and see if they have been failing to build.
 - If the input datasets have been failing to build, then this would explain why you are not seeing any updated information in your workbook. Continue debugging from here.
 - If the input datasets have been building as expected, and there is a mismatch with what you are seeing as the preview in the code workbook, then:
 - Try deleting the input table from the code workbook and adding it back in.
 - If this does not work, and there is still a discrepancy between the preview and the actual table, then contact Palantir Support.
2. If the transformation nodes are not updating when selecting **Update table preview**, then this is expected behavior as these are not impacted by this operation. To update all the transformation nodes within the code workbook, select `run -> run all`. This will run all the transformation nodes in your code workbook while respecting build dependencies.

[Return to top](#)

Unable to merge my branch into master

The most common issue when you find yourself unable to merge your branch back into main is that the `master` branch is protected. There may be issues around merge conflicts, but that is covered in another section.

To troubleshoot, perform the following steps:

1. Determine whether the branch is protected.
2. If the branch is protected then identify the owner and request that they merge your branch.
 - For a given workbook, turn on branch protection on `master`, and ensure that the users you want to restrict from merging branches only have `compass:edit`

[API Reference ↗](#) in the Workbook.

The Workbook has four permission levels related to branches: `view`, `edit`, `maintain`, and `manage`. By default, `compass:read` ex

[Send feedback](#)

`compass:edit` expands to `edit`, and `compass:manage` expands to `maintain` and `manage`.

- Creating a branch and preparing a merge into a parent branch always requires only `edit` permissions. Merging into a protected branch requires `maintain` permissions. Changing branch protection settings requires `manage` permissions.
- More information is available in our [Branching overview](#).

3. If the branch is not protected, then you run into merge conflicts, where this is non-additive code that conflicts with the code in the `master` branch.

- In these instances, resolve these merge conflicts before merging your branch back into `master`.
- More information is available in our [Merging branches](#) documentation.

4. If the `master` branch is not protected, and there are no merge conflicts, and you are still unable to merge your branch into `master`, contact Palantir Support.

[Return to top](#)

Make cell computation faster

Assuming I have dataset inputs of (1000 rows * 30 columns) + (1 million rows * 30 columns), and a transformation with a lot of windowing / column derivation steps, how can I make the computation run faster?

To troubleshoot, perform the following steps:

1. Refactoring

For experimentation or fast iteration, it is often a good idea to refactor your code into several smaller steps instead of a single large step.

This way, you compute the upstream cells first, write the data back to Foundry, and use this pre-computed data in later steps. If you were to keep re-computing without changing the logic, this creates excessive work.

[API Reference ↗](#)

Concretely:

[Send feedback](#)

```

workbook_1:
  cell_A:
    work_1 : input -> df_1
      (takes 4 iterations to get right): 4 * df_1
    work_2: df_1 -> df_2
      (takes 4 iterations to get right): 4 * df_2 + 4 * df_1
      = 4 df_2 + 4 df_1
    work_3: df_2 -> df_3
      (takes 4 iterations to get right): 4 * df_3 + 4 * df_2 + 4 * df_1
  total work:
    cell_A
      = work_1 + work_2 + work_3
      = 4 * df_1 + (4 * df_2 + 4 * df_1) + (4 * df_3 + 4 * df_2 + 4 * df_1)
      = 12 * df_1 + 8 * df_2 + 4 * df_3

```

Instead, if you wrote work_1 and work_2 into their own cells, the work you perform would instead look like:

```

workbook_2:
  cell_A:
    work_1: input -> df_1
      (takes 4 iterations to get right): 4 * df_1
  cell_B:
    work_2: df_1 -> df_2
      (takes 4 iterations to get right): 4 * df_2
  cell_C:
    work:3: df_2 -> df_3
      (takes 4 iterations to get right): 4 * df_3
  total_work:
    cell_A + cell_B + cell_C
      = work_1 + work_2 + work_3
      = 4 * df_1 + 4 * df_2 + 4 * df_3

```

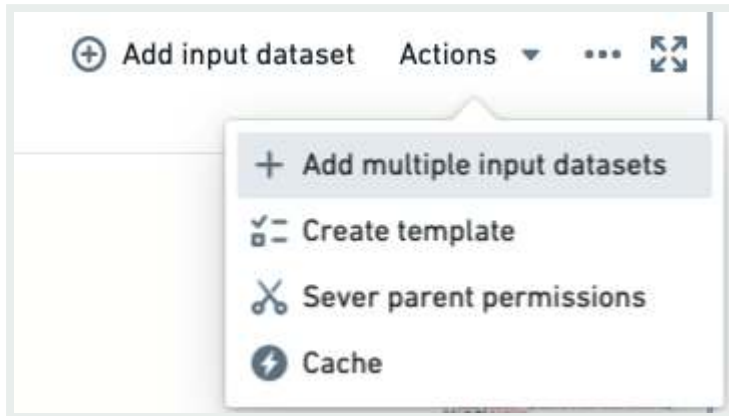
If [API Reference](#) ↗ df_2, and df_3 all cost the same amount to compute,

workbook_1.total_work = 24 * df_1, whereas workbook_2.total_work = 12 * df_1, so you can expect closer to the order of a 2x speed improvement on iteration.

Send feedback

2. Caching

For any "small" datasets, you should cache them by selecting the workbook, then choosing **Actions > Cache**.



This will keep the rows in-memory for your workbook and not require fetching from the written-back dataset. "Small" is an arbitrary judgment given several factors that must be considered, but Code Workbook does a good job of trying to cache it and will warn you if it is too big.

3. Function calls

You should stick to native PySpark methods as much as possible and never use Python methods directly on data (such as looping over individual rows, executing a UDF). PySpark methods call the underlying Spark methods that are written in Scala and run directly against the data instead of the Python runtime; if you simply use Python as the layer to interact with this system instead of being the system that interacts with the data, you will get all the performance benefits of Spark itself.

4. Downsampling

If you can derive your own accurate sample of a large input dataset, this can be used as the mock input for your transformations, until such time you perfect your logic and want to test it against the full set.

[API Reference ↗](#)

Consider downsampling and caching datasets above one million rows in a single line of PySpark code; you may experience faster turnaround times with

[Send feedback](#)

bugs slowly due to large dataset sizes.

5. Summary

A good code workbook looks like the following:

- Discrete chunks of code that do particular materializations you expect to re-use later but do not need to be recomputed over and over again.
- Down-sampled to "small" sizes.
- Cached "small" datasets for very fast fetching.
- Only native PySpark code that exploits the fast, underlying Spark libraries.

[Return to top](#)

PREVIOUS
← [Keyboard shortcuts](#)

NEXT
[Workbooks / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Workbooks

The main resource you interact with in Code Workbook is a **Workbook**. Workbooks are used to import datasets from Foundry and transform these input datasets for purposes such as:

- Cleaning and joining raw data imported from some external source to produce curated datasets for other users.
- Analyzing processed datasources to derive useful insights.
- Training and applying models to do predictive analysis.
- Creating parameterized visualizations to display in a report.

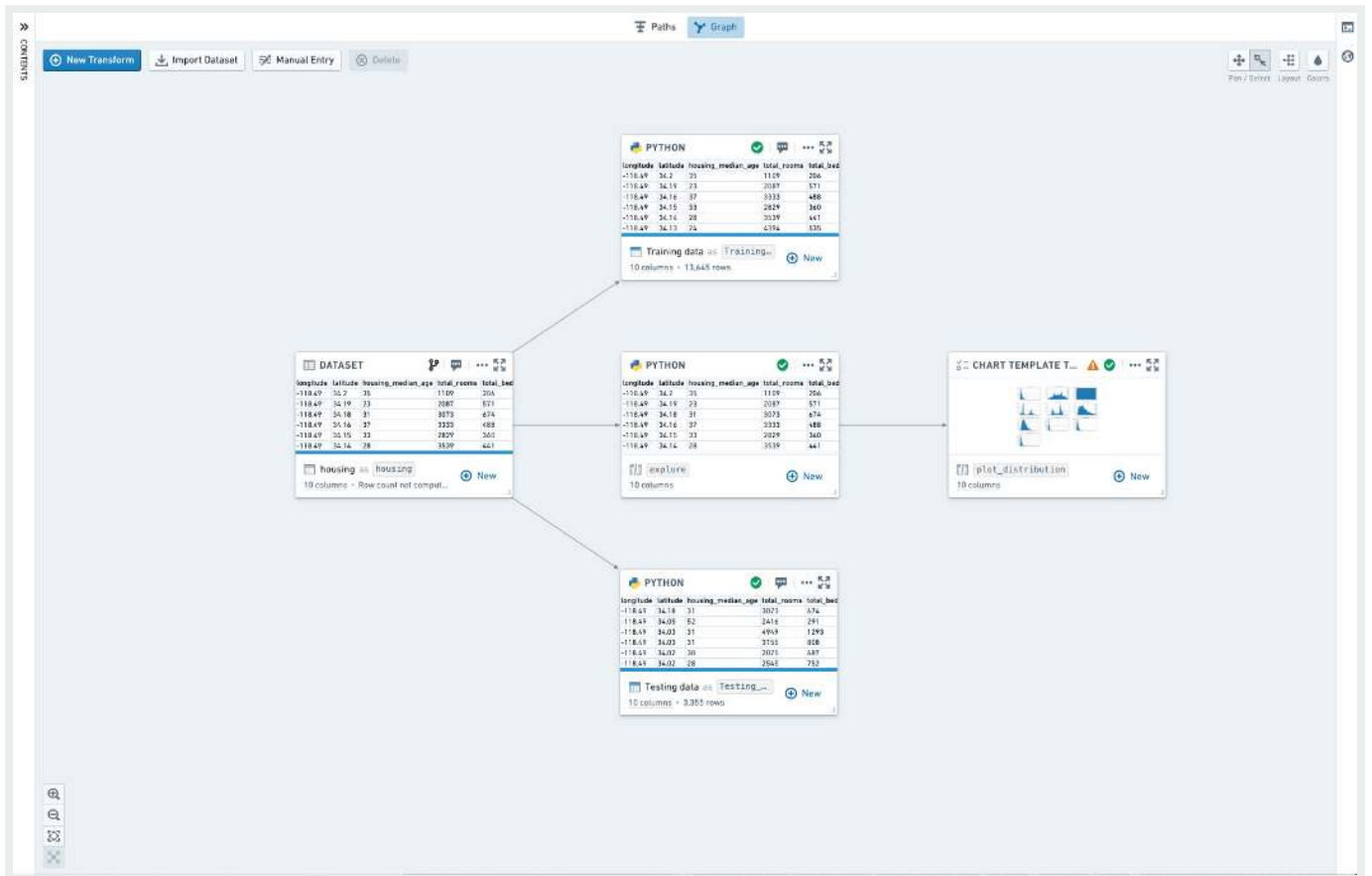
Graph

The core component of the Workbook interface is the Graph. The Graph represents how data flows through logical steps in your Workbook to produce outputs.

API Reference ↗



Send feedback



Input datasets are imported from elsewhere in Foundry to be used as source data in the workbook. Other than input datasets, each node in the graph represents a **transform**. A transform is a piece of logic that can return an output dataframe or model, and possibly render a visualization. A transform may be saved as a derived **dataset**. When a transform is saved as a derived dataset, running the transform will automatically write the results to Foundry, allowing them to be used outside of this Code Workbook and easily shared with others.

There are three types of transforms available in Code Workbook:

- **Code** transforms allow you to write code to process inputs and return results.
- **Template** transforms provide a form-based interface to transformation code, making it simple to reuse and share code.
- **Manual entry** transforms allow users to input data into a node.

Navigating the graph

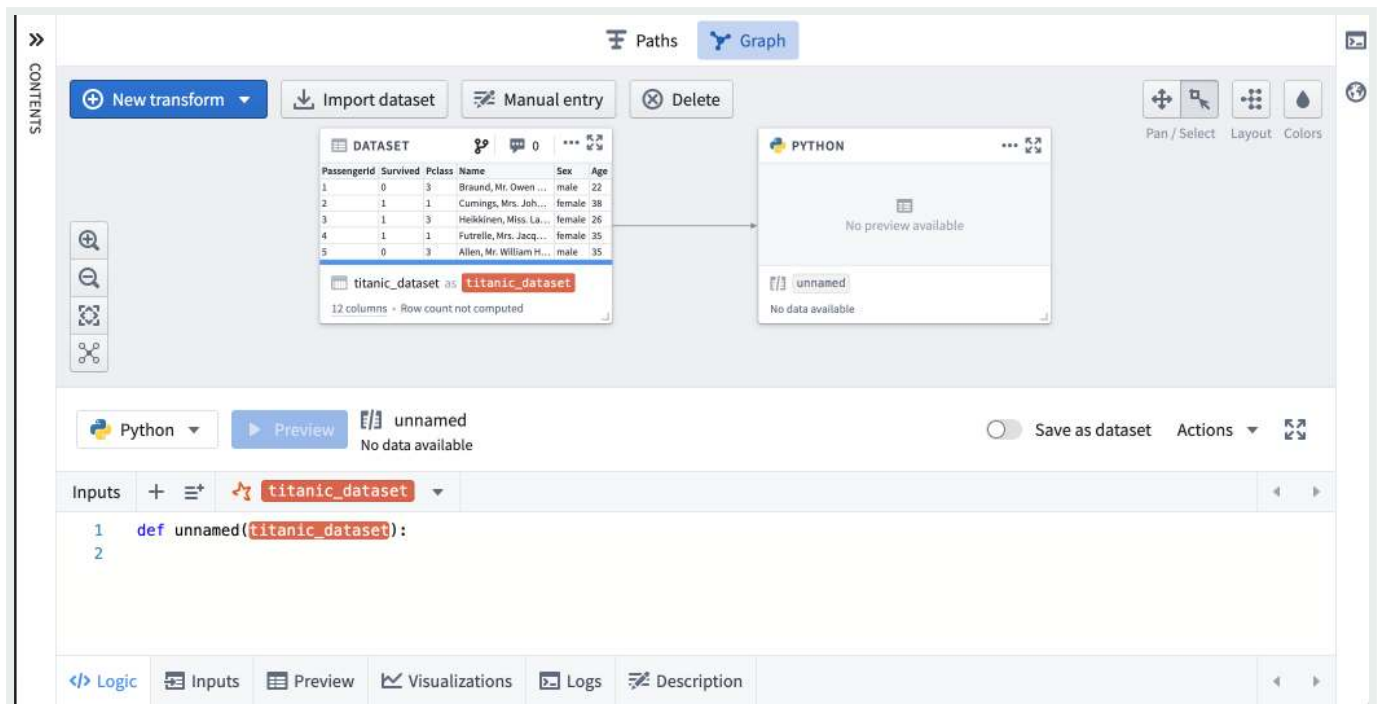
[API Reference ↗](#)

Click on any transform in the graph to open the **Logic** panel at the bottom of the interface.

This allows you to view and edit transform logic and view the transform's output.

[Send feedback](#)

select multiple transforms in the graph with Ctrl+Click on Windows or Cmd+Click on macOS.



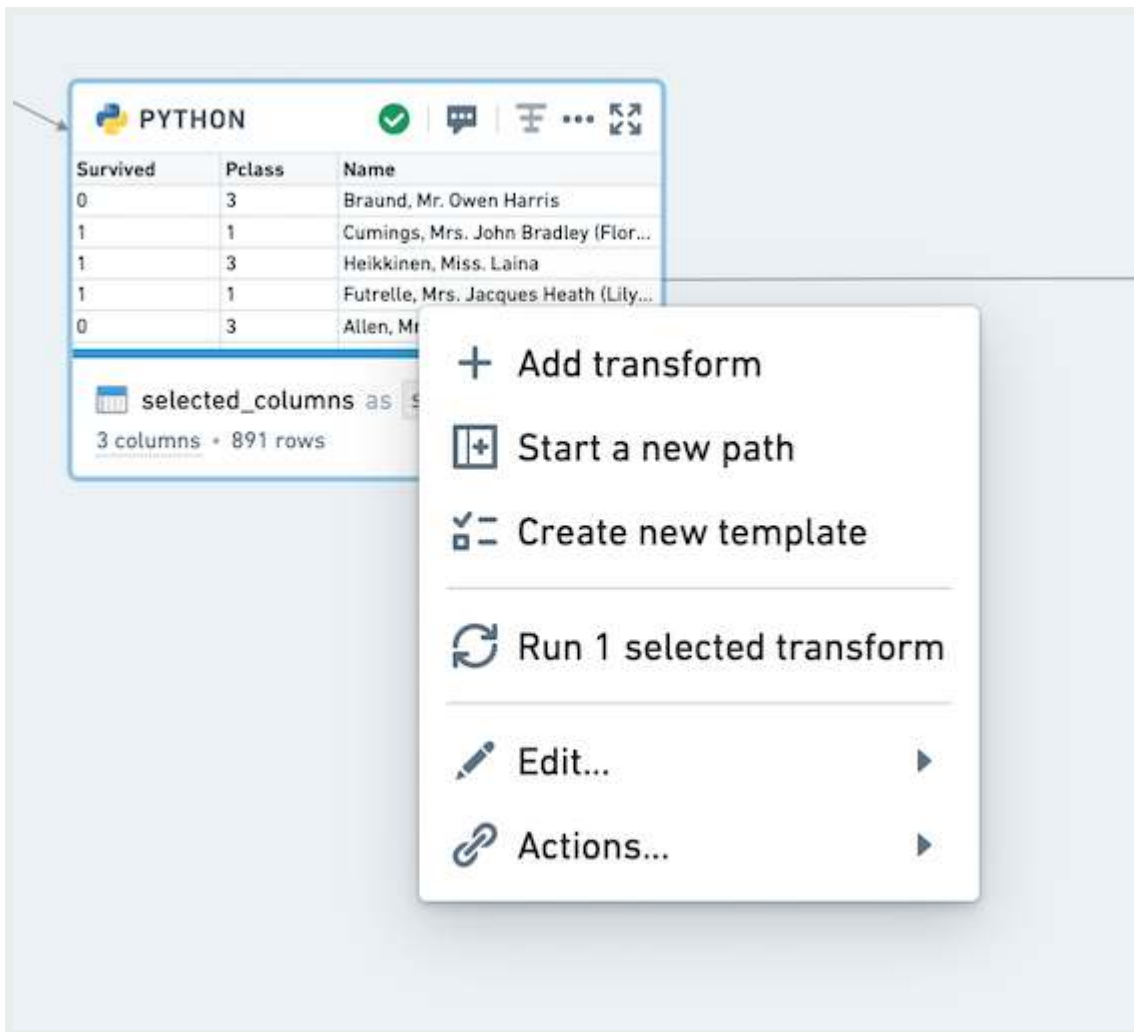
To help navigate large Workbooks, you can use the zoom options in the lower left of the graph, which include a **Zoom to fit** button that zooms the graph to bring all transforms into view.

Interacting with transforms

A key way to interact with transforms is through the *context menu*, which you can open by clicking on the ellipsis (...) on the top right of each transform in the graph, or by right-clicking on the transform in either the graph or the sidebar.

[API Reference ↗](#)

[Send feedback](#)



This menu provides a range of useful actions, including adding a new downstream transform, running the transform associated with this dataset, or deleting this transform from the Workbook. For transforms that are saved as datasets, the **Actions** option allows you to access the usual set of Foundry actions associated with the dataset.

If you select multiple transforms (by using Ctrl+Click on Windows or Cmd+Click on macOS), you can right-click anywhere on the graph to open the context menu for all the selected transforms. This can be helpful for adding a new transform with multiple inputs, or for running multiple transforms at once.

Hovering over a dataset name in the workbook will open a tooltip with the full path of the dataset in Foundry.

F API Reference ↗

You can switch to the Paths view at the top of your workbook. The Paths view is well suited for linear development within a workbook. The Paths view is well suited for

Send feedback

workflows that drill down on one dataset and perform sequential transformation steps.

The screenshot shows the 'Filtering Addresses by City' workflow in the Paths view. The left sidebar contains a 'Contents' panel with a tree view of the workflow, a 'TRANSFORMATION PATHS' section with a 'New' button, and a list of paths including 'Path 1' (2 transforms), 'Filtering Addresses by City' (2 transforms), and 'a bunch of boards' (3 transforms). The main panel displays the workflow steps. The first step is a dataset named 'addr_benefit' with 9 columns and 303,680 rows. It is followed by a 'CODE TRANSFORM' step named 'path_step_1' which contains the following Python code:

```
1 def path_step_1(addr_benefit):
2     df = addr_benefit.withColumn('CITY', F.rtrim(F.col('untitled_column_7')))\
3         .select('PERIOD', 'PRACTICE', 'NAME', 'CITY')
4     return df
```

The 'Run' button is visible, and a green checkmark indicates the step is successful. Below this is another 'CODE TRANSFORM' step named 'path_step_2' which contains the following Python code:

```
1 def path_step_2(path_step_1):
2     import matplotlib.pyplot as plt
3     import pandas as pd
4     df = path_step_1.filter(F.col('CITY') == 'CLEVELAND').toPandas()
```

The 'GLOBAL CODE' section is visible at the bottom of the main panel.

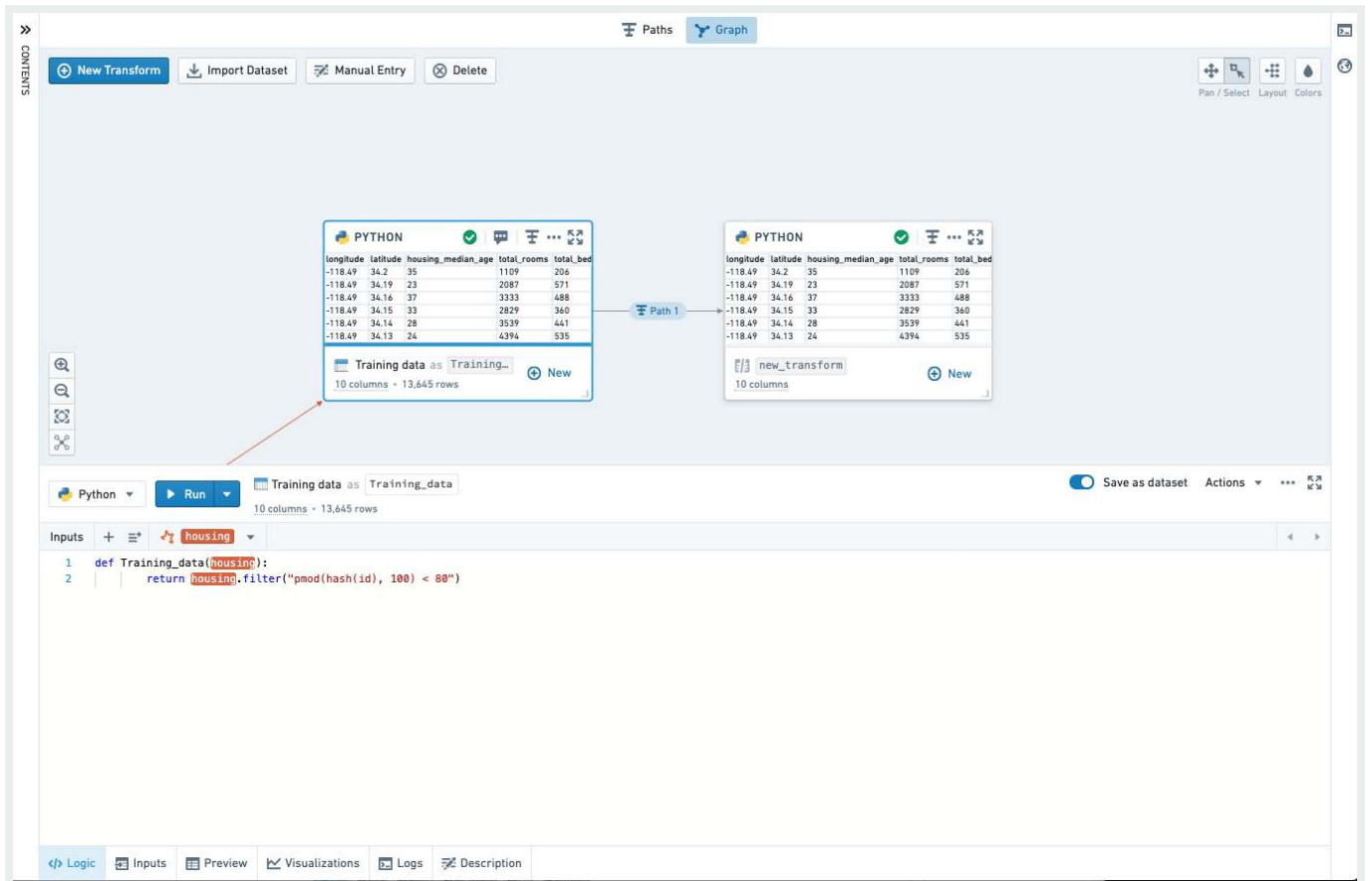
Paths can be started from datasets imported into the workbook, or transforms created in the workbook. In the Paths view, use the left sidebar to navigate between paths.

All transforms created in the Paths view are also persisted to the Graph. Navigate from a path node to the graph by clicking the **Open transform in Graph** icon () on the path node.

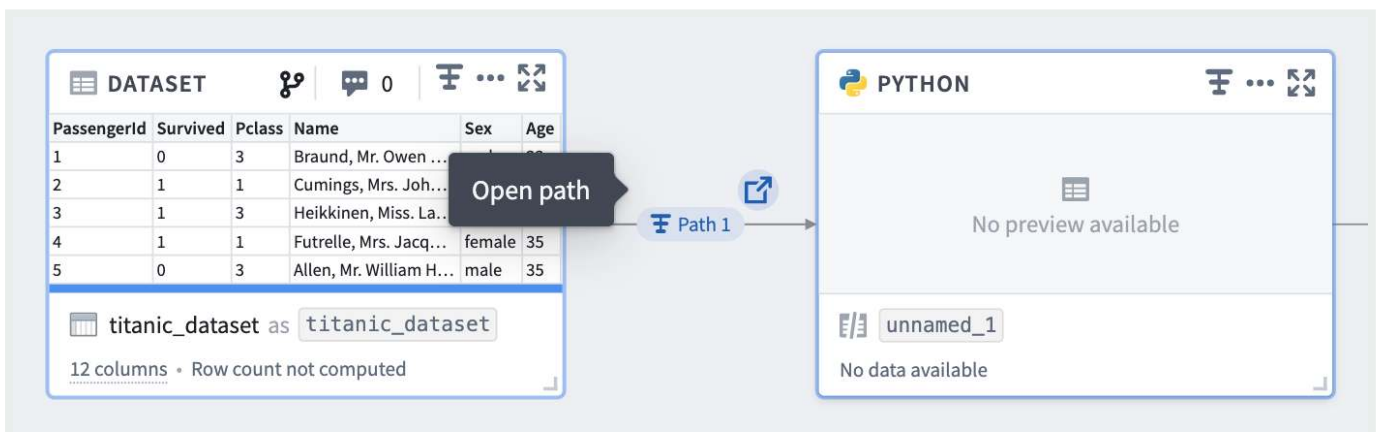
Within the Graph, you can see which nodes are part of a path by looking at the annotations on the edges between nodes. Hover over the annotation to open the source path.

[API Reference ↗](#)

[Send feedback](#)



You can also click on the Path icon on a node to open it in its source path.



Full Screen Editor

You may want to focus on one transform and interact with that transform in full screen

From the Graph or the Paths view, click on the expand button () on a transform node to open the Full Screen Editor.

[API Reference](#) ↗

[Send feedback](#)

» CONTENTS

Paths Graph

New transform In Open full screen editor ⌘ E Delete

DATASET 0

PassengerId	Survived	Pclass	Name	Sex	Age
1	0	3	Braund, Mr. Owen ...	male	22
2	1	1	Cumings, Mrs. Joh...	female	38
3	1	3	Heikkinen, Miss. La...	female	26
4	1	1	Futrelle, Mrs. Jacq...	female	35
5	0	3	Allen, Mr. William H...	male	35

titanic_dataset as titanic_dataset

12 columns • Row count not computed

Path 1

PYTHON

No pre

unnamed_1

No data available

Run titanic_dataset as titanic_dataset

12 columns • Row count not computed

When in fullscreen mode, you can rearrange your tabs to view two tabs at once by using the dock action or drag and drop.

Preview bar_chart_of_row_counts

2 columns A partial preview is available - transform is not saved as dataset

Transformation Logic

Inputs titanic_dataset

```

1 def bar_chart_of_row_counts(titanic_dataset):
2     import matplotlib.pyplot as plt
3     from pyspark.sql import functions as F
4     import numpy as np
5
6     input_df = titanic_dataset
7     categorical_column = "Sex"
8
9     # calculate the counts
10    total = input_df \
11        .groupBy(categorical_column) \
12        .agg(F.count("*").alias("count")) \
13        .orderBy("count")

```

Inputs Preview Logs De Visualizations

Sex	count
female	314
male	577

Dock to right

OUTPUT VISUALIZATION Download image Add to report

Please note: the console history and state are not preserved when closing the Workbook.

Variables

API Reference ↗

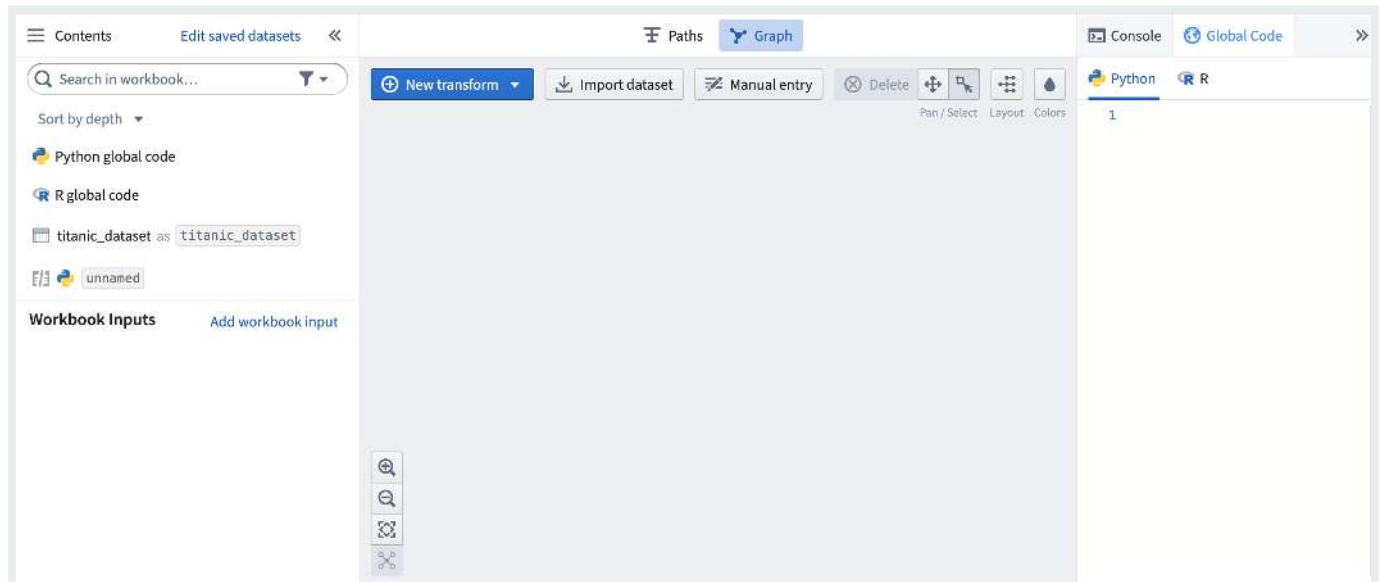
To exit the Full Screen Editor and return to the Graph or Paths, use the

collapse button (↖).

Send feedback

Panes

Code Workbook has three interface panes - [Contents](#), [Global Code](#), and [Console](#) - that are always available from the Graph, Paths view, and Full Screen Editor.

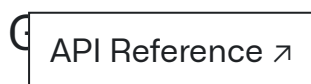


Contents

On the left-hand side of the interface, click on the **Contents** bar to open the Workbook contents pane.

When in Graph mode, the Contents pane shows a list of all transforms in the Graph in order to summarize the entire Workbook. By default, this list of transforms is sorted in topological order; input datasets are at the top and the furthest downstream transforms are at the bottom. Next to each transform in the list, there is a zoom button that will center the graph on this transform. You can also zoom in on a selected transform using the space bar. Click on **Edit Saved Datasets** to change which transforms, if any, are saved as datasets.

When in the Paths view, the Contents pane shows a mini-graph of all of the transforms in the workbook, allowing quick navigation between nodes.



Use the [Global Code pane](#) on the right-hand side of the Workbook in (like variables or functions) that will be available in all code transforms of that language

Send feedback

across the Workbook. For instance, you can use global code to define constants that will be used in multiple transforms or define helper functions you want to use repeatedly. [Learn more about how to use global code.](#)

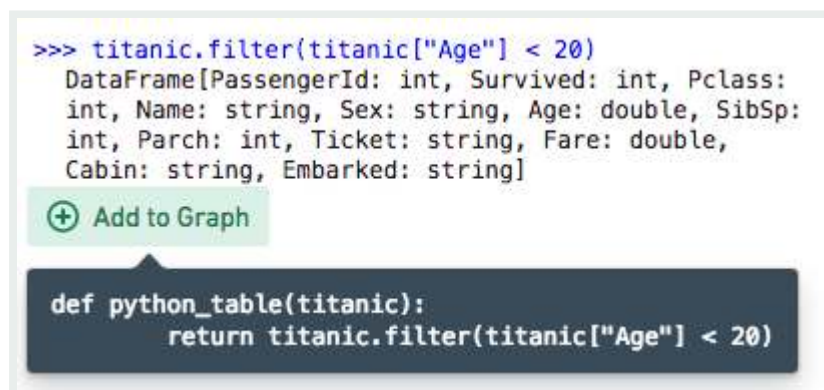
Console

On the right side of the workbook, the [console](#) provides a REPL (read-evaluate-print loop), enabling rapid, ad-hoc analysis of any dataset on the graph. [Learn more about how to use the console in each Workbook language.](#)

Beneath the console, use the Variables pane to set the input type of a given transform in the console.

If a console command returns a dataframe, you can use the **+ Add to graph** button to convert the console command into a transform. This allows you to experiment with logic in the console before promoting it to be a repeatable transform.

You can use the [keyboard shortcut](#) Ctrl+Shift+Enter on Windows or Cmd+Shift+Enter on macOS to send code from a transform directly to the console.



```
>>> titanic.filter(titanic["Age"] < 20)
DataFrame[PassengerId: int, Survived: int, Pclass:
int, Name: string, Sex: string, Age: double, SibSp:
int, Parch: int, Ticket: string, Fare: double,
Cabin: string, Embarked: string]
```

+ Add to Graph

```
def python_table(titanic):
    return titanic.filter(titanic["Age"] < 20)
```

PREVIOUS
← FAQ

NEXT
Supported languages →

API Reference ↗

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

Technologies Inc. All rights reserved.

Send feedback

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Languages

Supported languages and versions

Code Workbook currently supports three languages: Python, R, and SQL.

The currently supported versions of Python in Code Workbook include Python 3.10 and Python 3.12. Lower versions of Python are not supported, and environments using them will fail to resolve. We strongly recommend using the latest available version of Python, as Palantir Foundry discontinues support for Python versions that are [considered end-of-life](#) by Python Developer documentation.

The currently supported versions of R include R 3.6, R 4.0, R 4.1 and R 4.2. The versions R 3.3, R 3.4, and R 3.5 are not supported, and their respective environments will fail to initialize.

The SQL variant supported in Code Workbook is [Spark SQL](#).

To enable a specific language on a Code Workbook profile, see the Conda Environment section of the [Configuring Code Workbook profiles](#) documentation. A series of examples for each of the supported languages are provided below, in the respective introductions to [Python](#), [R](#), and [SQL](#).

Enable languages in a workbook

See the [API Reference](#) for more information on how to use the API. It is necessary for supported languages to function, as discussed in the [API Reference](#) sections below.



Enable R

Send feedback

 R is not yet available for self-service.

Two things must be true in order to have the ability to create an R transform in Code Workbook:

- The R language must be specifically enabled on your enrollment. Contact your Palantir representative for assistance with enablement.
- The package `vector-spark-module-r` must be present in the environment currently used in the workbook. This can be achieved in either of the following ways:
 - Toggle the **R** checkbox in the profile's configuration in Control Panel. This will automatically add the `vector-spark-module-r` package to the profile's environment.
 - Manually add `vector-spark-module-r` to the environment using the **Add package** dropdown menu.

See [Configuring Code Workbook profiles](#) for more information.

Enable Python

The package `vector-spark-module-py` must be present in the environment currently in use in the workbook. This can be achieved in either of the following ways:

- Toggle the Python checkbox in the profile's configuration in Control Panel. This will automatically add the `vector-spark-module-py` package to the environment.
- Manually add `vector-spark-module-py` to the environment using the **Add package** dropdown menu.

See [Configuring Code Workbook profiles](#) for more information.

Enable SQL

SQL transforms do not require any additional packages to function. As a result, SQL transforms will always be available by default for any given profile.

API Reference ↗

ⓘ If you do not plan on using either Python or R on a given profile, consider removing the associated `vector-spark-module` package to reduce your e

Send feedback

can always add them back when you need them.

Introduction to Python

Python transforms

A Python transform is defined as a Python function with any number of inputs, at most one output, and optionally one or more visualizations. By referencing a transform's alias as a function argument, Code Workbook will automatically pass as input of the transform the output of the mentioned alias. For more information about transforms in Code Workbook, consult the [Transforms overview](#) documentation.

A simple example of a Python transform could include a single PySpark DataFrame as input, transform the data using PySpark syntax, and have a transformed Spark DataFrame as output.

```
1 def child(input_spark_dataframe):  
2     from pyspark.sql import functions as F  
3     return input_spark_dataframe.filter(F.col('A') == 'value').withColumn()
```

Conversion between Spark and Pandas in Python

Within a Python transform, converting between Spark and Pandas dataframes is straightforward.

```
1 # Convert to PySpark  
2 spark_df = spark.createDataFrame(pandas_df)  
3 # Convert to pandas  
4 pandas_df = spark_df.toPandas()
```

[API Reference ↗](#)

Converting to pandas means collecting data to the driver. As a result, constrained by the available driver memory on the Spark module. If you are working with a

[Send feedback](#)

large dataset, you may want to first filter and aggregate your data using Spark, then collect it into a pandas DataFrame.

```
1 from pyspark.sql import functions as F
2 def filtering_before_pandas(input_spark_dataframe):
3     # Use PySpark to filter the data
4     filtered_spark_df = input_spark_dataframe.select("name", "age").filter(F.col("age") > 18)
5
6     # Convert to a pandas df, which collects the data to the driver
7     pandas_df = filtered_spark_df.toPandas()
8
9     # Perform pandas operations
10    mean_age = pandas_df["age"].mean()
11    pandas_df["age_difference_to_mean"] = pandas_df["age"] - mean_age
12
13    # Output the resulting DataFrame
14    return pandas_df
```

i To keep the order of a sorted pandas DataFrame after saving, save it as a Spark DataFrame with a single partition:

```
1 import pyspark.pandas as p
2
3 return p.from_pandas(df).to_spark().coalesce(1)
```

Introduction to R

R Transforms

An R transform is defined as an R function, with any number of inputs, at most one output, and optionally one or more visualizations. By referencing a transform's alias as a function argument, Code Workbook will automatically pass as input of the transform the output of the previous transform. For more information about transforms in Code Workbook, consult the [API Reference](#) documentation.

Send feedback

A simple example of an R transform could include one parent R data.frame, transform the data using R, and have one R data.frame as output.

```
1 child <- def(r_dataframe) {  
2   library(tidyverse)  
3   new_df <- r_dataframe %>% dplyr::select(col_A, col_B) %>% dplyr::filter(  
4   return(new_df)  
5 }
```

Conversion between Spark and R dataframes

Within an R transform, converting between Spark DataFrames and R data.frame is straightforward:

```
1 # Convert from Spark DataFrame to R data.frame  
2 new_r_df <- SparkR::collect(spark_df)  
3  
4 # Convert from R data.frame to Spark DataFrame  
5 spark_df <- SparkR::as.DataFrame(r_df)
```

Note that converting to an R data.frame means collecting data to the driver. As a result, the size of the data is constrained by the available driver memory on the Spark module. If you are working with a large dataset, you may want to first filter and aggregate your data using Spark, then collect it into an R data.frame.

```
1 output_dataset <- function(spark_df) {  
2   library(tidyverse)  
3  
4   # Use SparkR to filter the data  
5   input_dataset_filtered <- SparkR::select(spark_df, 'column_A', 'column_B')  
6  
7   # Convert to R data.frame  
8   local_df <- SparkR::collect(input_dataset_filtered)
```

API Reference ↗

```
10   # Use tidyverse functions to transform your data  
11   local_df <- local_df %>% dplyr::filter(column_A == 1)  
12 }
```

Send feedback

```
13     # Output an R data.frame
14     return(local_df)
15 }
```

R troubleshooting

When an imported dataset in Code Workbook is read in as an R `data.frame`, the dataset is converted from a Spark `DataFrame` to an R `data.frame` by collecting to the driver.

- You will not be able to read in arbitrarily large data as an R `data.frame`. The size of data is constrained by driver memory on the Spark module. To work with large data, consider first reading in your dataset as a Spark `DataFrame`, using SparkR to transform the data into something smaller, and then calling `SparkR::collect()` to convert it to an R `data.frame`. Alternately, use Python or SQL to transform your data into something smaller prior to using R.
- There are some known issues when collecting certain data types to R `data.frame`. In the majority of cases, when collecting we use a library called [r-arrow](#), which speeds up serialization and deserialization. In particular, when using r-arrow the `Long`, `Array`, `Map`, `Struct`, and `Datetime` types are not convertible. Consider dropping these columns or casting them to other data types (such as `String`). You will receive a warning in the interface when attempting to read an input with these types as an R `data.frame`.

R in Code Workbook is single-threaded, meaning only one R job can be run at a time on the same Spark module. If you initiate multiple R jobs at the same time, they will run serially; jobs that are queued will appear as "Queueing in Code Workbook."

- If you have a long-running job or jobs where the transforms are saved as datasets, we recommend that you run a batch build. The batch build will run on its own Spark module, allowing you to continue iterating in the same workbook or other workbooks that share the Spark module.

Introduction to SQL

The [API Reference](#) supported in Code Workbook is [Spark SQL](#). The only supported input and output types are Spark `DataFrames`.

Send feedback

A simple example of a SQL transform could join two input DataFrames on a join key.

```
1 SELECT table_b.col_A, table_b.col_B, table_a.*
2 FROM table_a
3 JOIN table_b ON table_a.col_C == table_B.col_C
```



To add a parent to a SQL node, referencing the alias within the code is not sufficient. You must use the UI by selecting the input bar, or create the child node using the **+** button.

PREVIOUS

← Overview

NEXT

Inputs and outputs →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

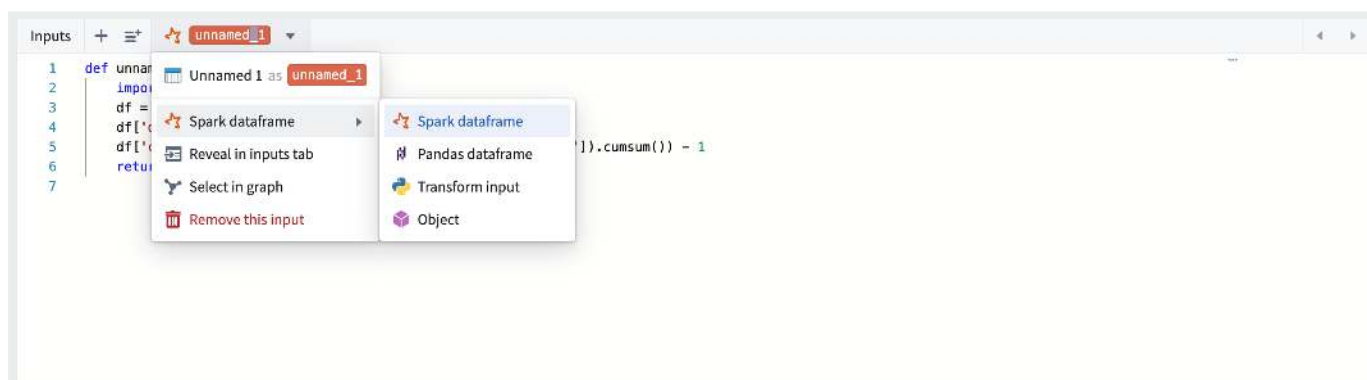
[API Reference ↗](#)

[Send feedback](#)

Inputs and outputs

Input types and conversions

Code Workbook allows you to specify the type of each input that a transform will receive. You can set the input type by selecting the dropdown menu under the transform's name:



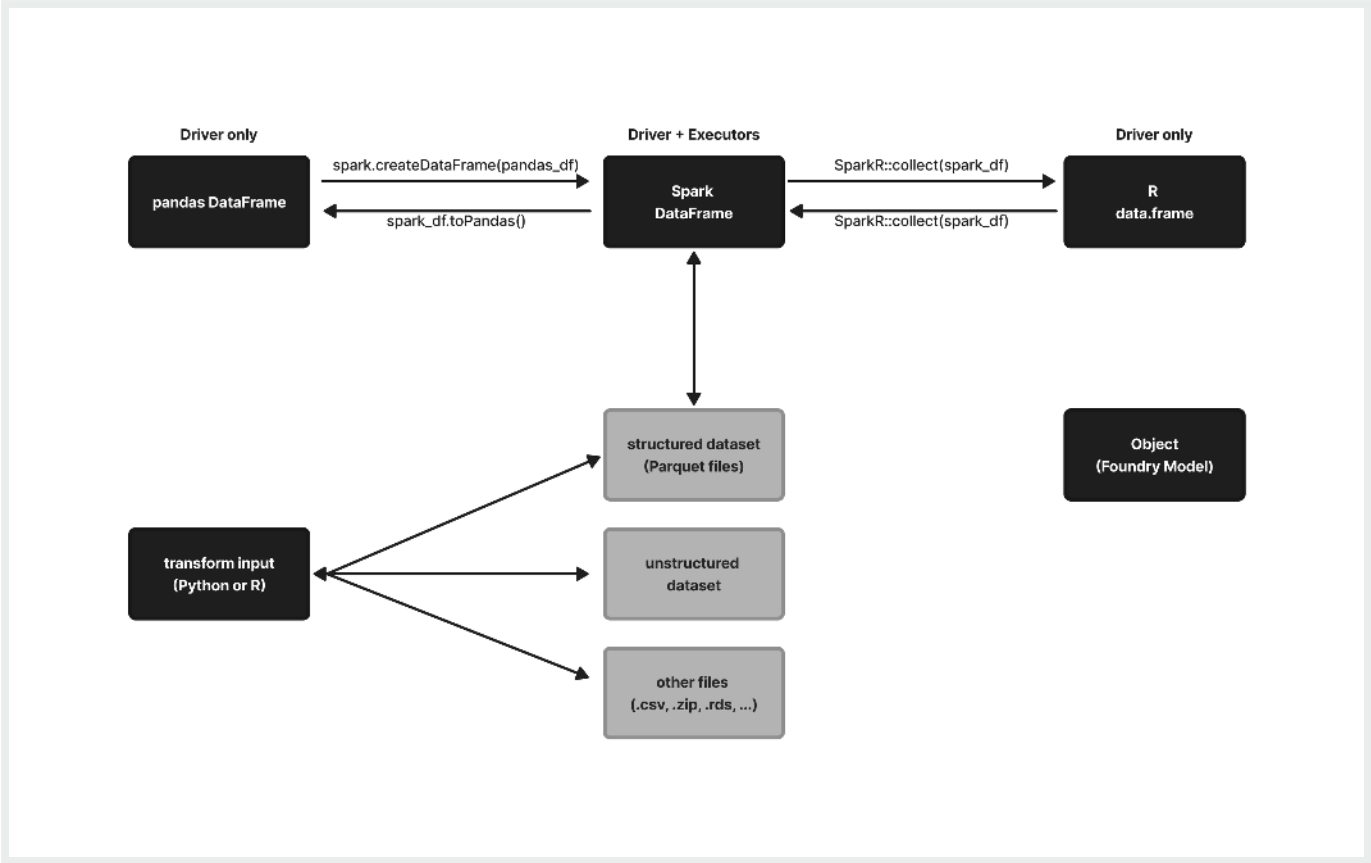
In case the requested input type does not match the type of the actual input coming in, Code Workbook performs the necessary conversions.

For example: If `transform_A`'s output is a pandas dataframe, and its derived transform `transform_B` manually sets the input as a Spark dataframe, Code Workbook will convert the pandas dataframe into the requested Spark dataframe. If `transform_C`'s output is a pandas dataframe, and its derived `transform_D` requests its input to be an R data.frame, Code Workbook will first convert the pandas dataframe back into a Spark dataframe, then into an R data.frame.

The following flowchart provides an overview of how Code Workbook handles conversions

[API Reference ↗](#)

[Send feedback](#)



Input and output restrictions

Output types

A Code Workbook transform can have any amount of inputs, each of which can have a different input type configured. However, a transform can only have a single output. Multiple outputs are not currently supported.

Code Workbook only allows certain types of outputs in its transforms, depending on language:

Language	Permitted Output Types
Python	Spark dataframe, Pandas dataframe, FoundryObject, None
R	R data.frame, FoundryObject, NULL
API Reference ↗	Spark dataframe

If a transform returns any other type than the ones listed above, Code Workbook will throw an unsupported type error.

Send feedback

i Note on None and NULL output types

Code Workbook will let you run a transform that has a None/NULL return value, but downstream transforms will not accept None/NULL as an input.

Python input types

By default, a python transform will accept its inputs as Spark dataframes. You can set the input type by selecting the dropdown menu under the transform's name, and Code Workbook will take care of converting upstream outputs into the desired input type:



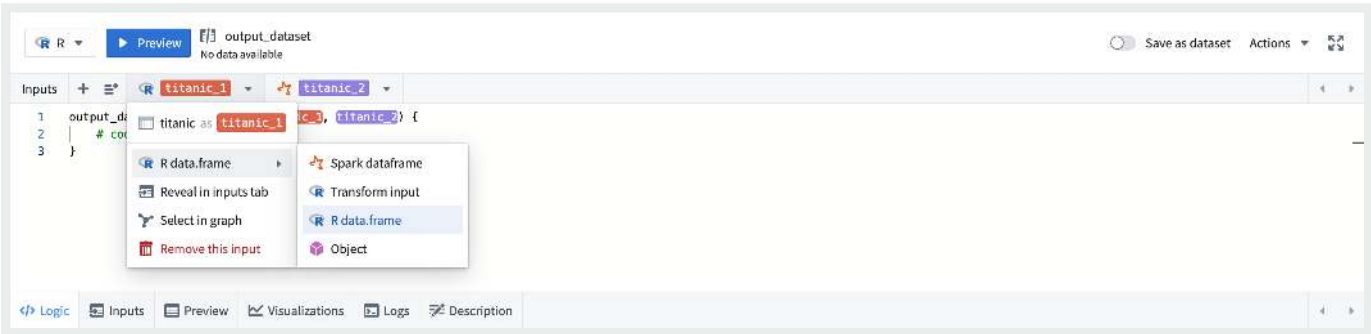
The following input types are supported in Python transforms:

Input types	Description
Spark dataframe (default)	pySpark dataframe of class <code>pyspark.sql.dataframe.DataFrame</code>
Pandas dataframe	Pandas dataframe of class <code>pandas.core.frame.DataFrame</code>
Python transform input (only available on inputs 'saved as dataset')	A <code>FileSystem</code> object that allows direct file access on the input. See Access unstructured files documentation.
API Reference ↗	A Foundry Model Object allowing you to perform modeling workflows. See Models documentation.

Send feedback

R input types

By default, an R transform will accept its input(s) as R data.frames. You can set the input type by selecting the dropdown menu under the transform's name, and Code Workbook will convert upstream outputs into the desired input type:



The following input types are supported in R transforms:

Input types	Description
Spark dataframe	SparkR dataframe of class SparkDataFrame
R data.frame (default)	R dataframe of class data.frame
R transform input (only available on inputs 'saved as dataset')	A FileSystem object that allows direct file access on the input. See Access unstructured files documentation.
Object	A Foundry Model Object allowing you to perform modeling workflows. See Models documentation.

SQL input and output types

The only supported input and output types in SQL transforms are Spark dataframes. The dataframe can be read within SQL as a table.

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Console

On the right side of the workbook, the console provides a REPL (read-evaluate-print loop), enabling rapid, ad-hoc analysis of any transform on the graph. There is one console for each language enabled on your branch.

The screenshot shows the Palantir Console interface. At the top, there are tabs for 'Console' and 'Global Code'. Below these are language-specific tabs for 'Python', 'R', and 'SQL'. The 'Python' tab is active. A 'Reset console' button is located in the top right corner of the console area. The main area displays a Python code snippet and its output:

```
>>> df = titanic_dataset.select('Name', 'Sex', 'Age').filter(F.col('Age') > 22)
>>> df.show()
```

Name	Sex	Age
Cumings, Mrs. Joh...	female	38.0
Heikkinen, Miss. ...	female	26.0
Futrelle, Mrs. Ja...	female	35.0
Allen, Mr. Willia...	male	35.0
McCarthy, Mr. Tim...	male	54.0
Johnson, Mrs. Osc...	female	27.0
Bonnell, Miss. El...	female	58.0
Andersson, Mr. An...	male	39.0
Hewlett, Mrs. (Ma...	female	55.0
Vander Planke, Mr...	female	31.0
Fynney, Mr. Joseph J	male	35.0
Beesley, Mr. Lawr...	male	34.0
Sloper, Mr. Willi...	male	28.0
Asplund, Mrs. Car...	female	38.0
Uruchurtu, Don. M...	male	40.0
Wheadon, Mr. Edwa...	male	66.0
Meyer, Mr. Edgar ...	male	28.0
Holverson, Mr. Al...	male	42.0
Ahlin, Mrs. Johan...	female	40.0
Turpin, Mrs. Will...	female	27.0

only showing top 20 rows

Below the output, there is a prompt for the next command: >>>

At the bottom left, there is a button labeled 'API Reference ↗'. At the bottom right, there is a button labeled 'Send feedback'.

When using the console:

- Use the Enter key to execute the current code input.
- Use Shift+Enter to input a newline and create multi-line statements.
- Use the Up and Down keys to navigate through previously executed commands.

Beneath the console, use the **Variables** pane to set the input type of a given transform to the console.

Variables	
Name	Type
titanic_dataset	 Pandas ▾
adult_titanic_passengers	 Spark ▾

If a console command returns a dataframe, you can use the **+ Add to graph** button to convert the console command into a transform. This allows you to experiment with logic in the console before promoting it to be a repeatable transform.

You can use the [keyboard shortcut](#) Ctrl+Shift+Enter on Windows or Cmd+Shift+Enter on macOS to send code from a transform directly to the console.

Global code

The Global Code pane, accessible on the right-hand side of the Workbook interface, allows you to define variables and functions that will be available in all code transforms of that language across the Workbook. For instance, you can use global code to define constants that will be used in multiple transforms or define helper functions you want to use repeatedly.

Example

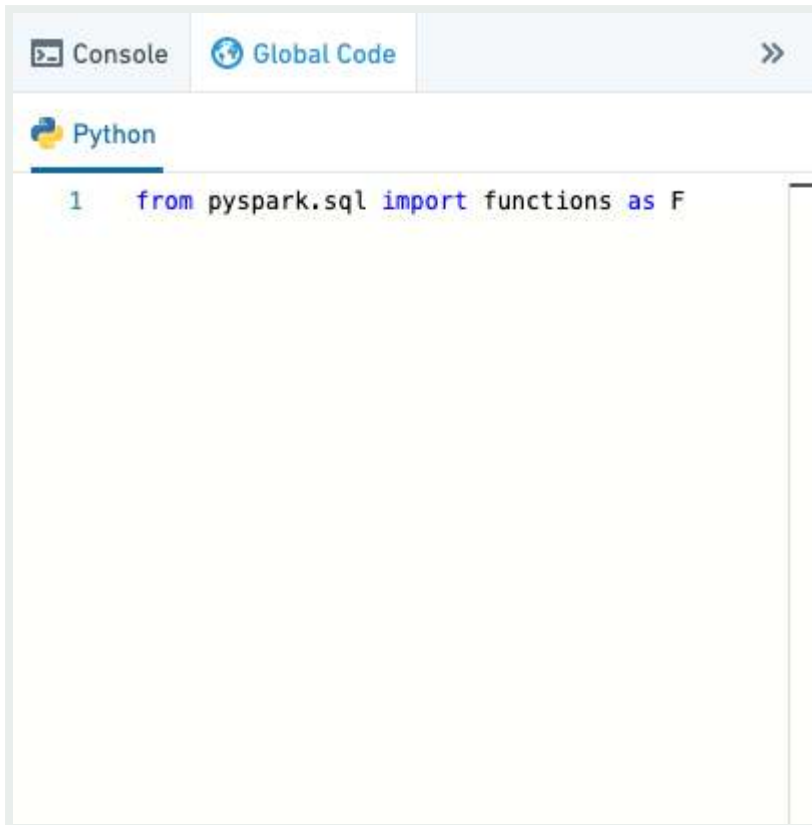
In this example, we'll write a simple function based on the `titanic_dataset` that takes the age of a passenger and returns their age bracket.

Let's begin by opening the Python Global Code panel at the right of the page, which looks like this:

[API Reference ↗](#)



[Send feedback](#)



Once the Global Code panel is open, copy-paste the following code into the panel:

```
1 def return_age_bracket(age):
2     if age is None:
3         return 'Not specified'
4     elif (age <= 12):
5         return '12 and under'
6     elif (age >= 13 and age < 19):
7         return 'Between 13 and 19'
8     elif (age >= 19 and age < 65):
9         return 'Between 19 and 65'
10    elif (age >= 65):
11        return '65 and over'
12    else: return 'N/A'
```

To use this global function, create a new Python transform derived from `titanic_dataset` and paste the following code into the transform:

```
1 def return_age_bracket_udf(titanic_dataset):
2     from pyspark.sql.functions import udf
3
4     input_df = titanic_dataset
```

API Reference ↗

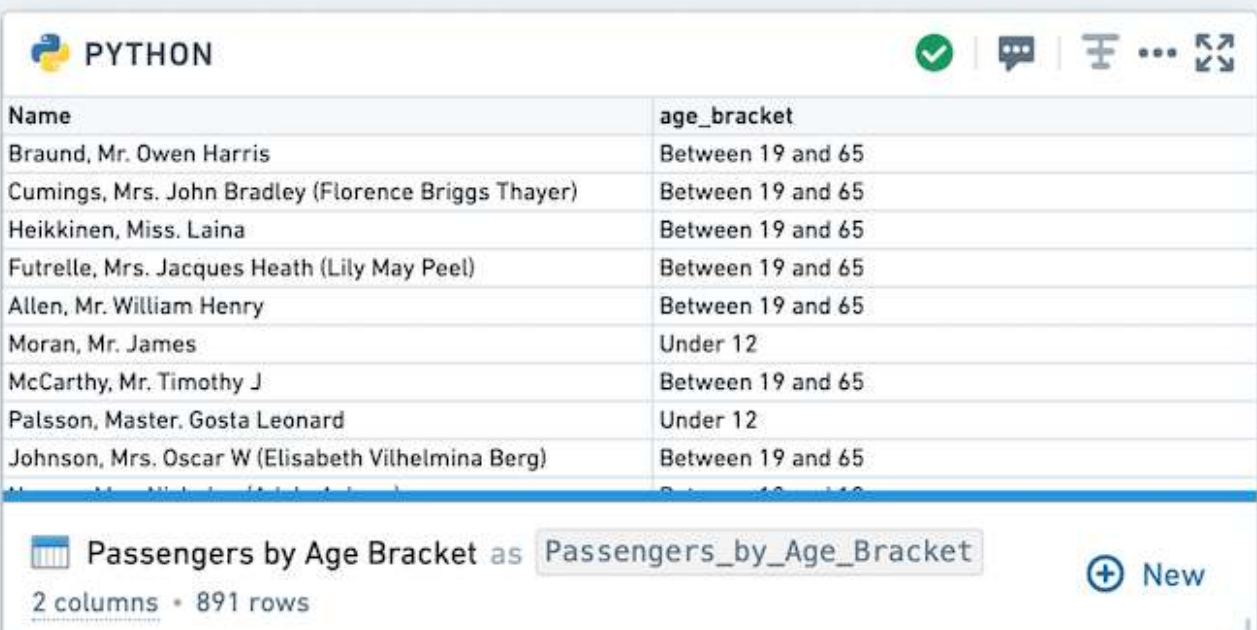
Send feedback

```

5
6 age_bracket_udf = udf(return_age_bracket)
7
8 output_df = input_df.withColumn("age_bracket", age_bracket_udf(input_df
9 output_df = output_df.select(output_df.Name, output_df.age_bracket)
10
11 return output_df

```

Now run the code. You will see the following output:



Name	age_bracket
Braund, Mr. Owen Harris	Between 19 and 65
Cumings, Mrs. John Bradley (Florence Briggs Thayer)	Between 19 and 65
Heikkinen, Miss. Laina	Between 19 and 65
Futrelle, Mrs. Jacques Heath (Lily May Peel)	Between 19 and 65
Allen, Mr. William Henry	Between 19 and 65
Moran, Mr. James	Under 12
McCarthy, Mr. Timothy J	Between 19 and 65
Palsson, Master. Gosta Leonard	Under 12
Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	Between 19 and 65
...	...

Passengers by Age Bracket as Passengers_by_Age_Bracket
2 columns • 891 rows

This code may take some time to run since user defined functions (UDFs), especially with loops, can often be inefficient. Using globally defined functions is not always a best practice. For this example, `pyspark.functions` offers a simpler method:

```
when((condition), result).otherwise(result).
```

Let's try to get the same result as above without using a UDF:

```

1 def passengers_by_age_bracket(titanic_dataset):
2     from pyspark.sql import functions as F

```

API Reference ↗

```

4     input_df = titanic_dataset

```

```

5     output_df = input_df.withColumn("age_bracket", F.when

```

```

6     .when(input_df.Age <= 12, F.lit("Under 12"))

```

Send feedback

```

7                                     .when(( (input_df.Age >
8                                     .when(( (input_df.Age >
9                                     .when(input_df.Age >= 6
10    output_df = output_df.select('Name', 'age_bracket')
11
12    return output_df

```

Under most conditions, the above transformation should run in a few seconds, compared to minutes with a UDF.

Note on reproducibility

Note that in order to ensure that results are reproducible, mutating variables and functions in global code will not propagate to other transforms. For example, in Python, if you define a list in global code like this:

```
1 my_list = [1,2,3,4]
```

And then update the list in your transform:

```

1 def my_transform(input_df):
2     my_list.append(5)
3     print(my_list)

```

Running `my_transform` will print `[1,2,3,4,5]`, but other transforms will still receive a value of `[1,2,3,4]`.

PREVIOUS

← Console

NEXT

Duplicate nodes →

API Reference ↗

© 2026 Palantir Technologies Inc. All rights reserved.

Send feedback

[Cookies Statement](#) ↗

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

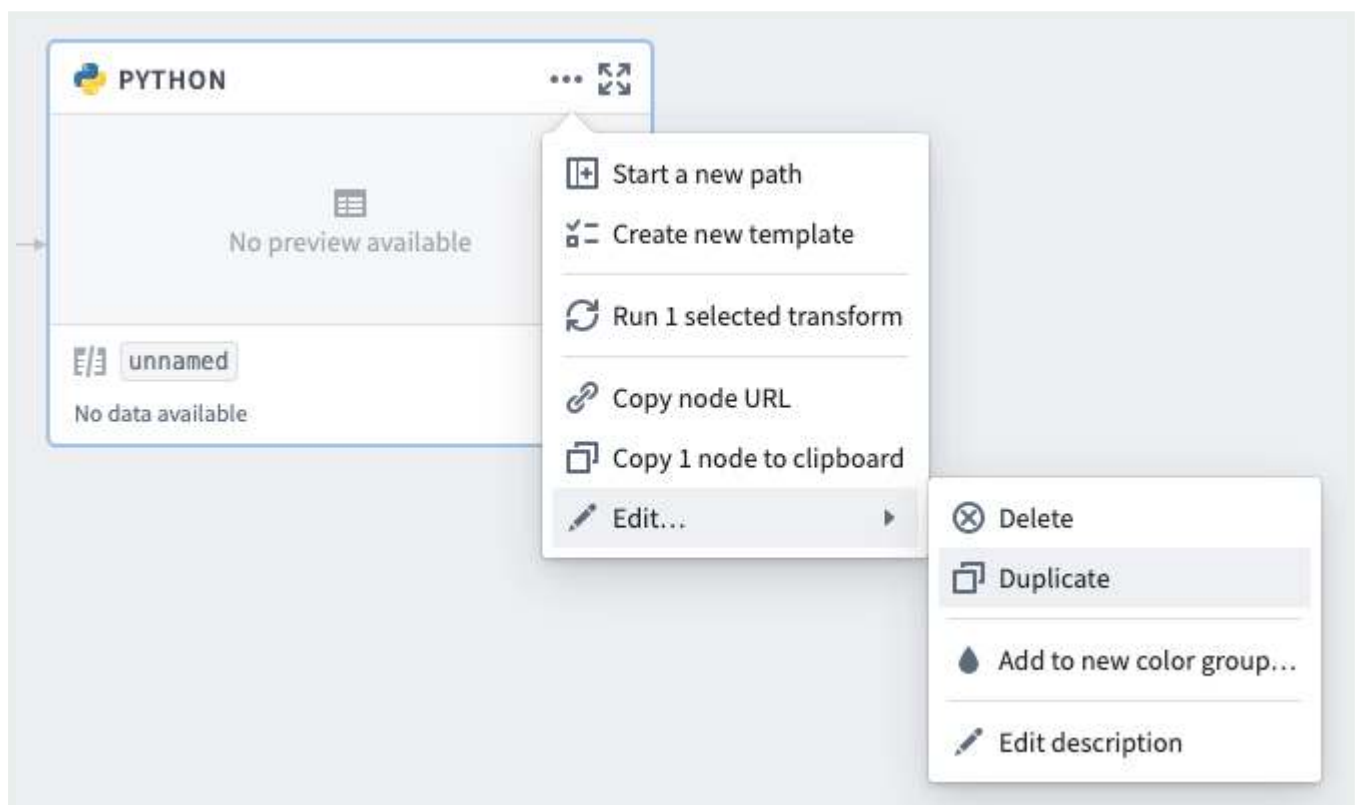
— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Duplicate nodes

Nodes can be duplicated one at a time within a Code Workbook. To duplicate a node, first open the node menu by either selecting the ... button in the top right of the node or right-clicking anywhere on the node. Then, hover on **Edit...** to reveal a submenu and select Duplicate.



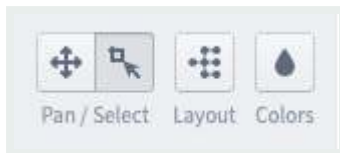
Copy/paste nodes

Nodes can be duplicated by using copy/paste. The following method allows you to copy a node or set of nodes within the same Code Workbook or in different Code Workbooks. Doing so allows you to transfer the contents of one Code Workbook to another.

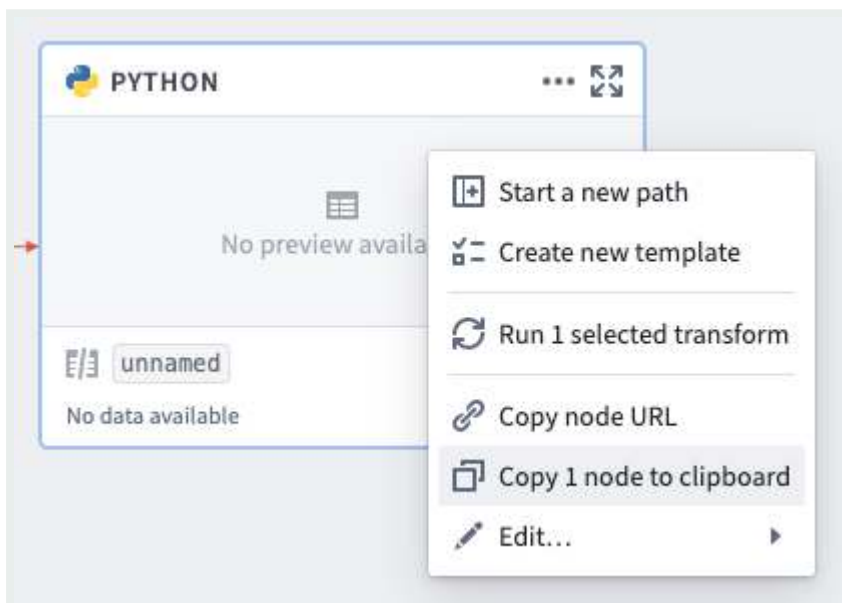
[API Reference ↗](#)

[Send feedback](#)

1. Switch to select mode in the top right corner of your screen.



2. Select the nodes to copy by clicking and dragging to draw a box around the desired nodes. You can also hold down `Cmd` on a Mac or `Ctrl` on a Windows machine and click on each node in turn.
3. Right-click on the selected nodes and choose **Copy X node to clipboard**. Alternatively, you can copy the nodes using the [keyboard shortcut](#) `Cmd+C` (macOS) or `Ctrl+C` (Windows).



4. While in select mode, paste the nodes into the desired destination using the [keyboard shortcut](#) `Cmd+V` (macOS) or `Ctrl+V` (Windows).

PREVIOUS
← Global code

NEXT
Moving to production →

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Moving to production

i We recommend using Code Workbook or Code Workspaces for code-based analyses and pipeline prototypes. We recommend Pipeline Builder and Code Repositories for robust production pipelines and support for workflows that require an additional layer of governance and scrutiny, high data, or optimized performance.

For more information about where to write your pipeline, review our [comparison of Code Workbook, Code Workspaces, and Code Repositories](#).

Once you're satisfied with the outputs you have derived in a Workbook, you should harden your logic and output datasets to make them reliable. Doing this in Code Workbook is straightforward. We'll run through a few simple steps that can help make your work robust and production-ready:

- [Protecting branches](#)
- [Using batch builds](#)
- [Data health checks](#)
- [Organizing outputs](#)
- [Organizing the graph](#)
- [Exporting to Code Repository](#)

Protecting branches

API Reference ↗



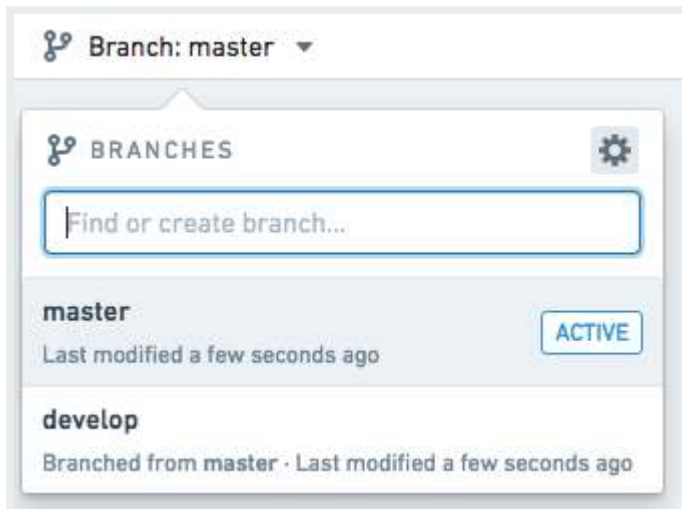
When you create a branch in a Workbook, you must have Owner privileges on that Workbook.

Send feedback

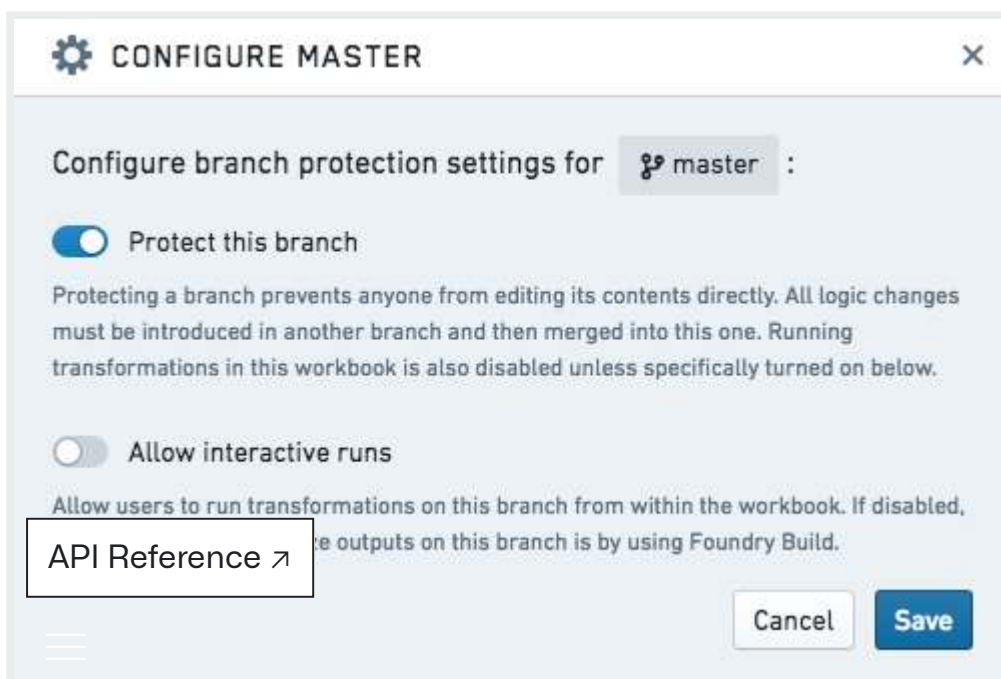
Branch protection allows branches within a Workbook to be locked down, preventing anyone from editing logic in that branch directly. Instead, logic changes must be created in another branch and then merged into the protected branch. Typically, users protect the **master** branch in a Workbook, but you can protect any other branches as well.



To protect a branch, click the settings icon () in the top right of the branch menu, as shown below.



Toggle on **Protect this branch** to enable branch protection, as shown below. By default, a protected branch does not allow any user to use the **Run** button on that branch to compute output datasets. This prevents Workbook runs from colliding with scheduled builds, which are described in the next section.



Send feedback

After saving, your branch will be protected and read-only.



Using batch builds

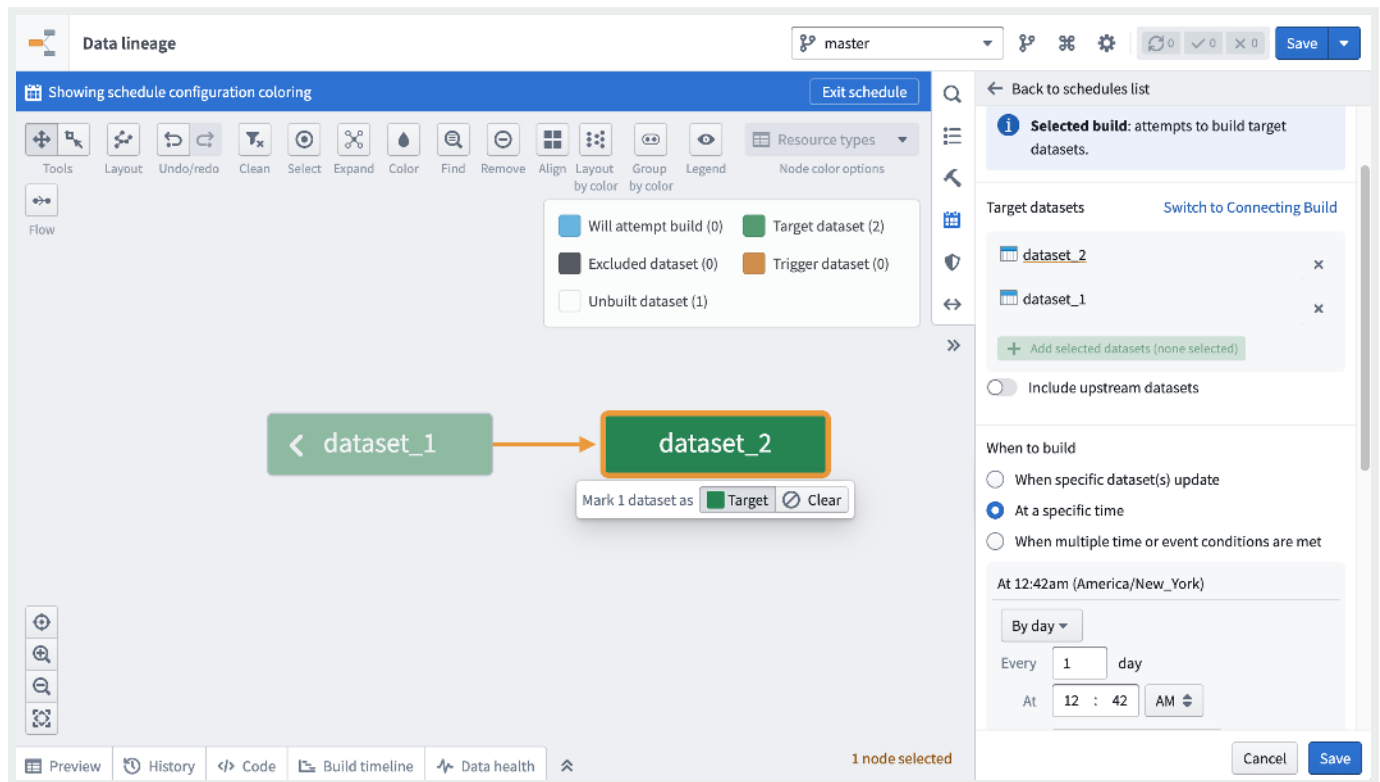
You may want to regularly refresh output datasets created in Code Workbook, either based on the input dataset(s) updating, or on a time-based cadence. You can do this by scheduling a recurring build on these output datasets.

- To schedule a recurring build on one output dataset, open dataset actions and click on **Manage Schedules**.
- To schedule a recurring build on multiple output datasets, click the cog icon at the top of the workbook and select **Explore Data Lineage**. You should now see all input datasets and saved output datasets from your workbook.

Both actions will bring you to the Data Lineage app. Click on the calendar icon in the right hand pane to open the **Manage Schedules** interface. Follow the prompts to set up a recurring schedule. In the image below, a schedule that builds `dataset_1` and `dataset_2` daily is shown.

[API Reference ↗](#)

[Send feedback](#)



[Learn more about creating schedules in Data Lineage.](#)

Note that batch builds will not update transforms that are not saved as datasets (e.g. unpersisted transforms). Concretely, consider the case where unpersisted transform A is the parent of persisted transform B. If I use batch build to build transform B, transform B will use the latest logic from transform A, and the latest data from the upstream input dataset. However, the preview shown in the workbook for transform A, as well as any visualization created in transform A, will not be updated by this batch build.

Data health checks

Another best practice for output datasets is adding **Data Health checks**. Open your output dataset and click the Health tab to access the Data Health page. Setting health checks allows you to receive notifications if a dataset's build fails, it becomes stale, or fails to conform to some other requirement you specify.

[Learn more about Data Health in Foundry.](#)

API Reference ↗

Send feedback



Organizing outputs

When using Workbooks as part of your project, we recommend creating the following folder structure in the project:

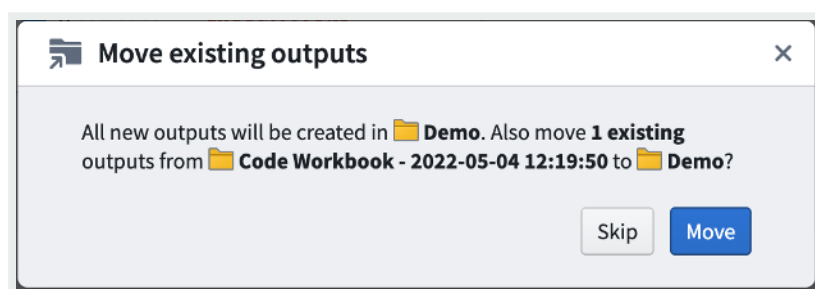
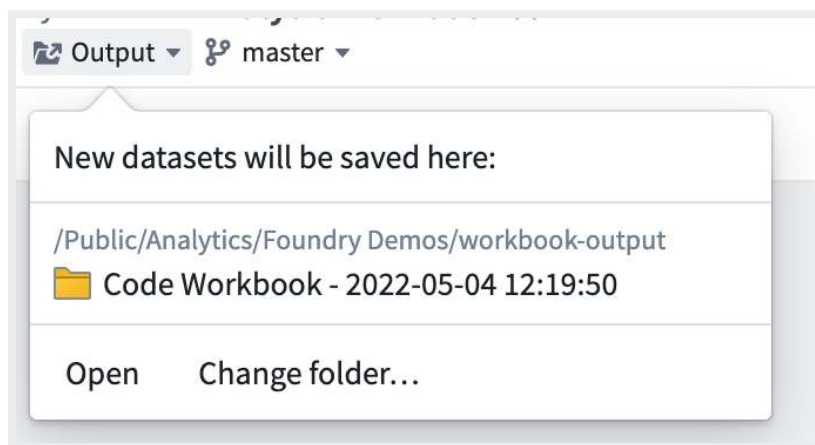
- **/data**
- **/workbooks**
- **/templates**

Once this folder structure is set up, you can share a new Workbook from your home folder easily:

1. Move your Workbook to the `/workbooks` directory.
2. If you created any templates in your Workbook, move them to the `/templates` directory.
3. To move datasets, click the Output dropdown in the top-left of the Workbook and then click “Change folder...” to choose a folder where new output datasets will be added. Choose the `/data` folder in your project. By default, all the datasets you derived in your Workbook will be moved to the new folder you choose.

[API Reference ↗](#)

[Send feedback](#)



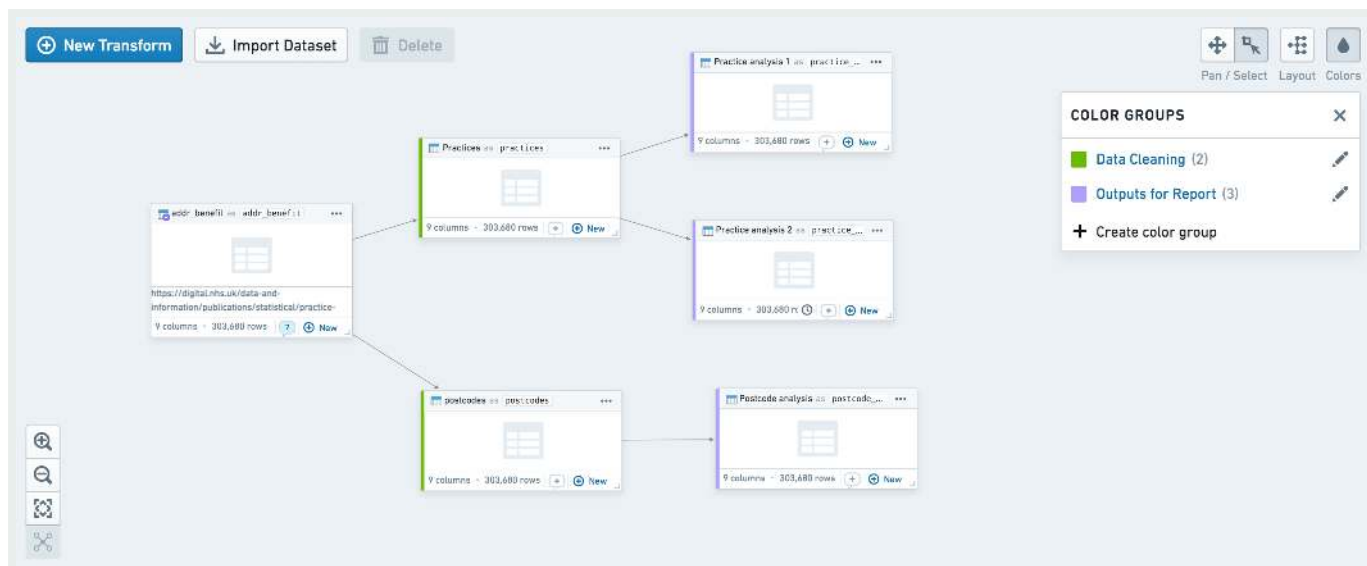
Organizing the graph

You may want to curate and organize your graph to allow other users to easily understand the flow of transformation. Two organizational features available in Code Workbook are node coloring and auto-layout.

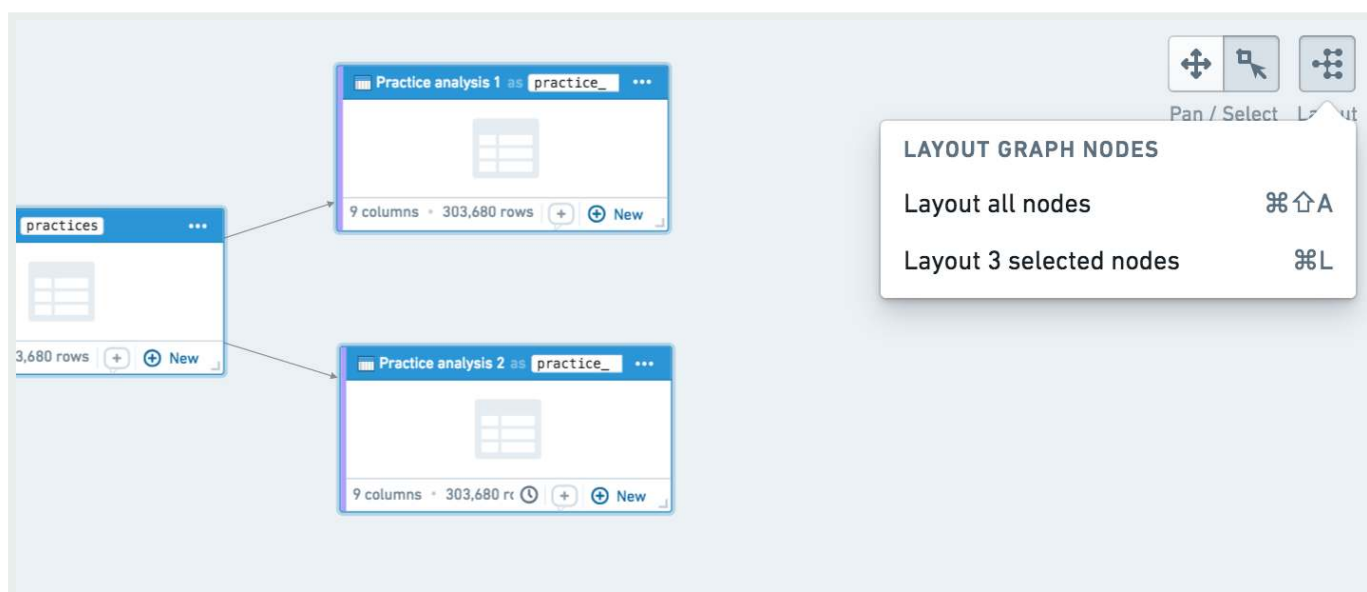
You can use node coloring to visually group nodes on the graph. Create a new color group by clicking on the **Colors** button at the top right, and add nodes to the color group by selecting them and using the **+** button on the color group. In the workbook contents helper, you can also sort the list of datasets by color group.

[API Reference ↗](#)

[Send feedback](#)



You can also auto-layout sections of your graph by clicking the layout button at the top right. By default, auto-layout arranges the entire graph, but you can also select specific nodes and click auto-layout to arrange only those nodes.



Exporting to Code Repository

If you have finished prototyping a pipeline in Code Workbook and would like to move your code to Code Repositories, you can use the **Export to Code Repository** helper. [Learn more about exporting to Code Repositories.](#)

[API Reference](#) ↗

Export your code to Code Repositories for a variety of reasons:

[Send feedback](#)

- Code Repositories provides full Git support, allowing users to view and revert to previous commits.
- Code Repositories supports incremental transforms and multi-output transforms.
- Code Repositories supports [unmarking workflows](#) and provides control over when PRs can be merged using [branch settings](#).

PREVIOUS

← [Duplicate nodes](#)

NEXT

[Transforms / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

— [Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

Transforms

Transforms are pieces of logic that take one or more inputs and return a single output. Inputs and outputs can be Foundry datasets or models.

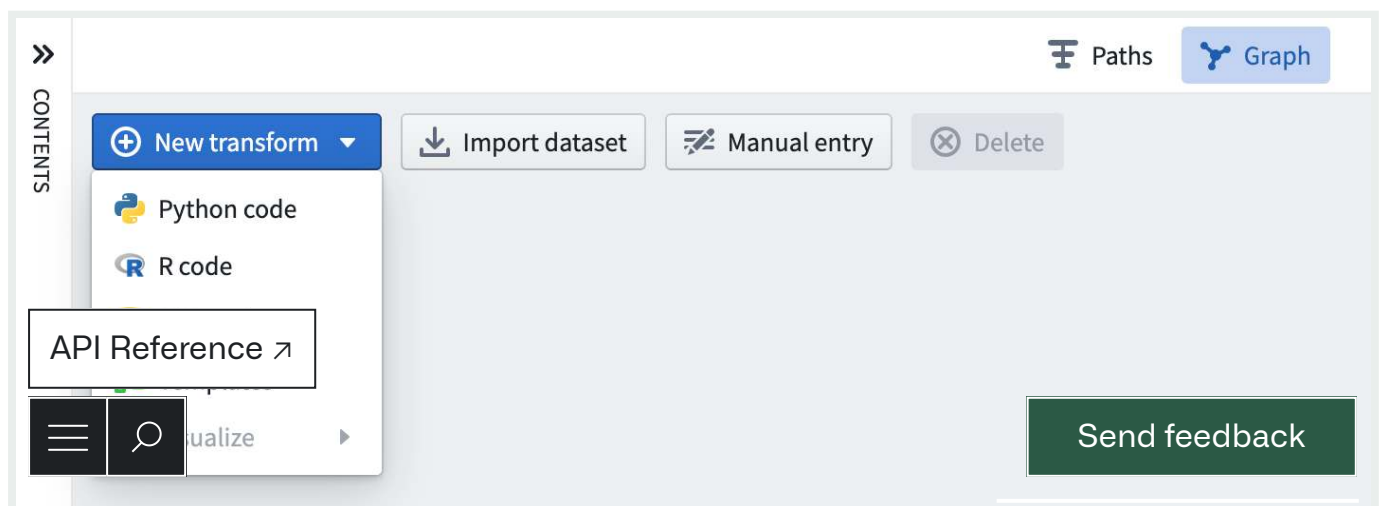
Code Workbook currently supports three types of transforms:

- **Code** transforms allow users to author arbitrary logic in their language of choice.
- **Template** transforms abstract away code behind a simple, form-based interface that can be reused between workbooks and users.
- **Manual entry** transforms allow users to input data into a node.

Adding transforms

There are several ways to add new transforms in Code Workbook.

Use the **New Transform** button in the top-left of the graph, which will allow you to select the type of transform to add. For example, select **Python code** to add a new Python code node, or **Templates** to open the template browser.



Alternately, hover over an existing transform on the graph and click on the blue + button that appears to open the selection dropdown. The newly created transform will be a child of the selected transforms.

The screenshot shows a 'DATASET' view with a table of Titanic data. The table has columns: PassengerId, Survived, Pclass, Name, Sex, and Age. The first five rows are visible. A blue '+' button is overlaid on the table, and a dropdown menu is open, showing options for adding transforms: Python code, R code, SQL code, Templates, and Visualize.

PassengerId	Survived	Pclass	Name	Sex	Age
1	0	3	Braund, Mr. Owen ...	male	22
2	1	1	Cumings, Mrs. Joh...	female	38
3	1	3	Heikkinen, Miss. La...	female	26
4	1	1	Futrelle, Mrs. Jacq...	female	35
5	0	3	Allen, Mr. William H...	male	35

titanic_dataset as titanic_dataset

12 columns • Row count not computed

- Python code
- R code
- SQL code
- Templates
- Visualize

Editing transforms

Editing names

Transforms in Code Workbook can be saved as datasets. Read more about [data persistence](#).

Transforms that are not saved as datasets are identified by a workbook-specific alias that appears in the transform in code. For input datasets, and transforms that are saved as datasets, Code Workbook shows two different names for each dataset: the name of the underlying Foundry dataset, and a workbook-specific alias.

API Reference ↗

Send feedback



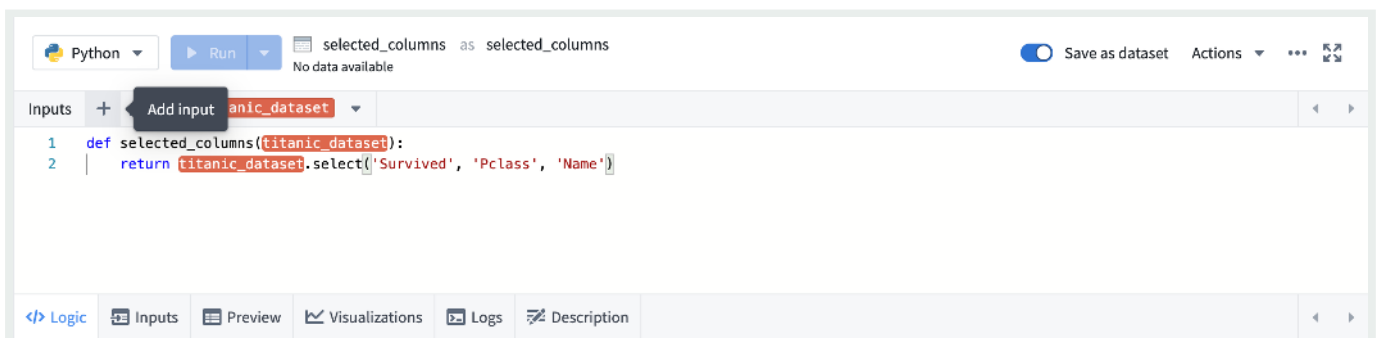
There are a few benefits to having aliases:

- You can refer to datasets in a way that is intuitive for the logic in your Workbook. For example, if you had two datasets called “Titanic” in different folders corresponding to different update frequencies, you could alias one of them as `titanic_historical` and the other as `titanic_daily_feed`.
- You can freely edit dataset aliases without changing the dataset names for anybody using the same datasets elsewhere in Foundry.
- You can change the alias of a dataset on a branch and merge this change across branches.

To edit the dataset name or its alias, click on the name in the logic panel, enter the new name, and use the Enter key when finish editing. If you change a dataset name or alias, you will be prompted to automatically update the other name if you wish.

Adding inputs

Clicking on any transform opens the logic panel. To add input, click the **+** button in the top input bar, and then click the desired input.



To add multiple inputs, use the button next to the **+**.

API Reference ↗

Code

Send feedback

Code transforms appear as a code editor prepopulated with basic structure for the language you are using in the current Workbook. See the [Languages documentation](#) for specific details about writing code in each language.

Global code

The [Global code](#) pane allows you to define code that executes before each code transform in your Workbook. The Global Code pane can be used to define variables and functions that will be available throughout the transforms in your Workbook.

Templates

For [Template transforms](#), the logic panel displays a form to edit Template parameters.

- After clicking a dataset parameter, click any dataset in the graph to set it as the value for that parameter.
- Column parameters depend on the input dataset being specified. After it is specified, you can choose from a dropdown list of column names.
- For value parameters such as integers and strings, you can type in a custom value.

Manual entry

Add a manual entry node by clicking the **Manual Entry** button on the graph.



Enter data in the **Manual Entry** tab. Currently supported column types are Double, Integer, Boolean and String. **Only 500 rows are supported.**

Copy and paste to easily bring in data from other places.

API Reference ↗

Send feedback

The screenshot shows the Dataiku interface with a 'MANUAL ENTRY' transform configuration window open. The window displays a table with the following data:

scenario_name	param1	param2
high_valuation	0.5	1.5
low_valuation	0.7	0.9
middle_valuation	0.6	1.2

Below the configuration window, the 'Preview' tab shows a table with 13 rows and 10 columns. The first three rows contain data from the manual entry transform:

	scenario_name STRING	param1 STRING	param2 STRING	D STRING	E STRING	F STRING	G STRING	H STRING	I STRING	J STRING
1	high_valuation	0.5	1.5							
2	low_valuation	0.7	0.9							
3	middle_valuation	0.6	1.2							
4										
5										
6										
7										
8										
9										
10										
11										
12										
13										

Running transforms

Select **Run** in the bottom panel to execute the selected transform. To run all downstream transforms, select the arrow icon and choose **Run all downstream**.

Running a transform requires edit permissions on the workbook, as well as access to all of the [Markings](#) used by any dataset used by the workbook.

There are also several hotkeys available to quickly run code. Type **?** to view the list of hotkeys.

There are two ways to run many transforms at once:

- Select multiple datasets in the graph by using Ctrl + Click or Cmd + Click. Then right-click anywhere on the graph and select **Run N datasets** from the menu.
- Click the settings icon () in the top bar and select **Run all transforms**.

When running, any printed output is streamed to the Logs tab. This allows you to print output in the middle of your transform and see results while your transform is still running, which can be useful for debugging or timing your logic.

[Send feedback](#)

Once saved to a dataset, any sorting of the data that was applied in code may not be persisted. In other words, if you run a transform that sorts data, the data may not be sorted in that order when you view it as a dataset, though other operations in the transform will have been applied.

Tabs

When working in the Graph or Full Screen Editor, use the bottom tabs on a node to navigate inputs and outputs of a transform.

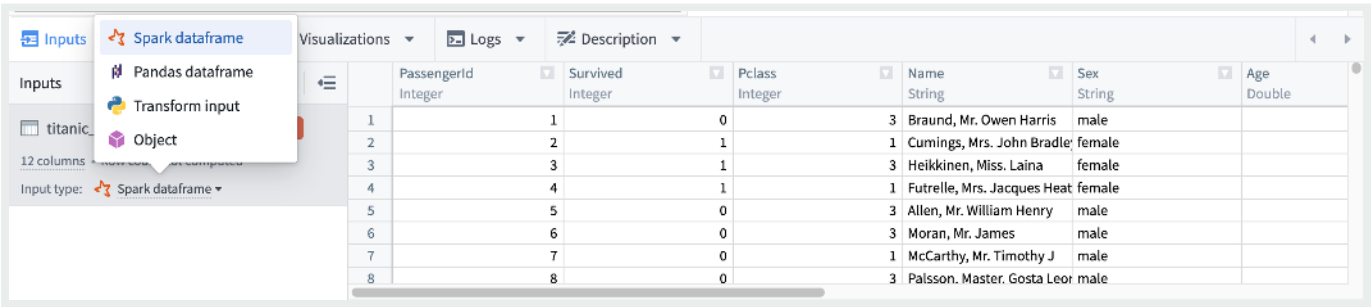
Inputs

Use the Inputs tab to view the inputs to your transform, add additional inputs, or change the input types.



Warning

As with other Spark-based applications, Code Workbook does not by default maintain row order when reading datasets. Sort your data in code if your analysis requires a particular row order.



The following input types are available for each node language:

Node language	Available input types
Python	Spark dataframe, Pandas dataframe, Python transform input, Object
R	Spark dataframe, R transform input, R data.frame

API Reference ↗

Send feedback

When a new transform is added, its input transforms will default to an input type based on the new transform's language and each input transform type, as detailed below:

Input transform type	Downstream transform language	Default input type	Additional details
Import with no schema	Python	Python transform input	
	R	R transform input	
	SQL	Spark dataframe	While you can create a SQL node with an input transform with no schema, SQL nodes only support Spark dataframe inputs and will not be effective in analyzing inputs with no schema.
Import with schema	Python	Spark dataframe	
	R	R data.frame	
	SQL	Spark dataframe	
Import with custom file format, including models	Python	Object	See the model training tutorial for more information. Object input types in Code Workbook are custom file formats and are not related to ontology objects.
	R	Object	See the model training tutorial for more information. Object input types in Code Workbook are custom file formats and are not related to ontology objects.
	SQL	Spark dataframe	While you can create a SQL node with an input transform with no schema, SQL nodes only support Spark dataframe inputs and will not be effective in analyzing inputs with no schema.

API Reference ↗

Send feedback

Input transform type	Downstream transform language	Default input type	Additional details
			support Spark dataframe inputs and will not be effective in analyzing custom file format inputs.
Derived nodes	N/A	Input node's output type	If the output type of the input node is incompatible with the derived node's language, derived nodes will use defaults as defined above for import node types. For example, if Node A returns a Pandas dataframe and derived Node B is an R node, Node B will default to reading Node A as an R data.frame.

Preview

When you run your transform in Code Workbook, a 50-row preview is calculated to see the shape of your result. This preview is displayed in the preview tab before your dataset is finished writing to Foundry. After the dataset is persisted to Foundry, the preview tab will update to the full result.

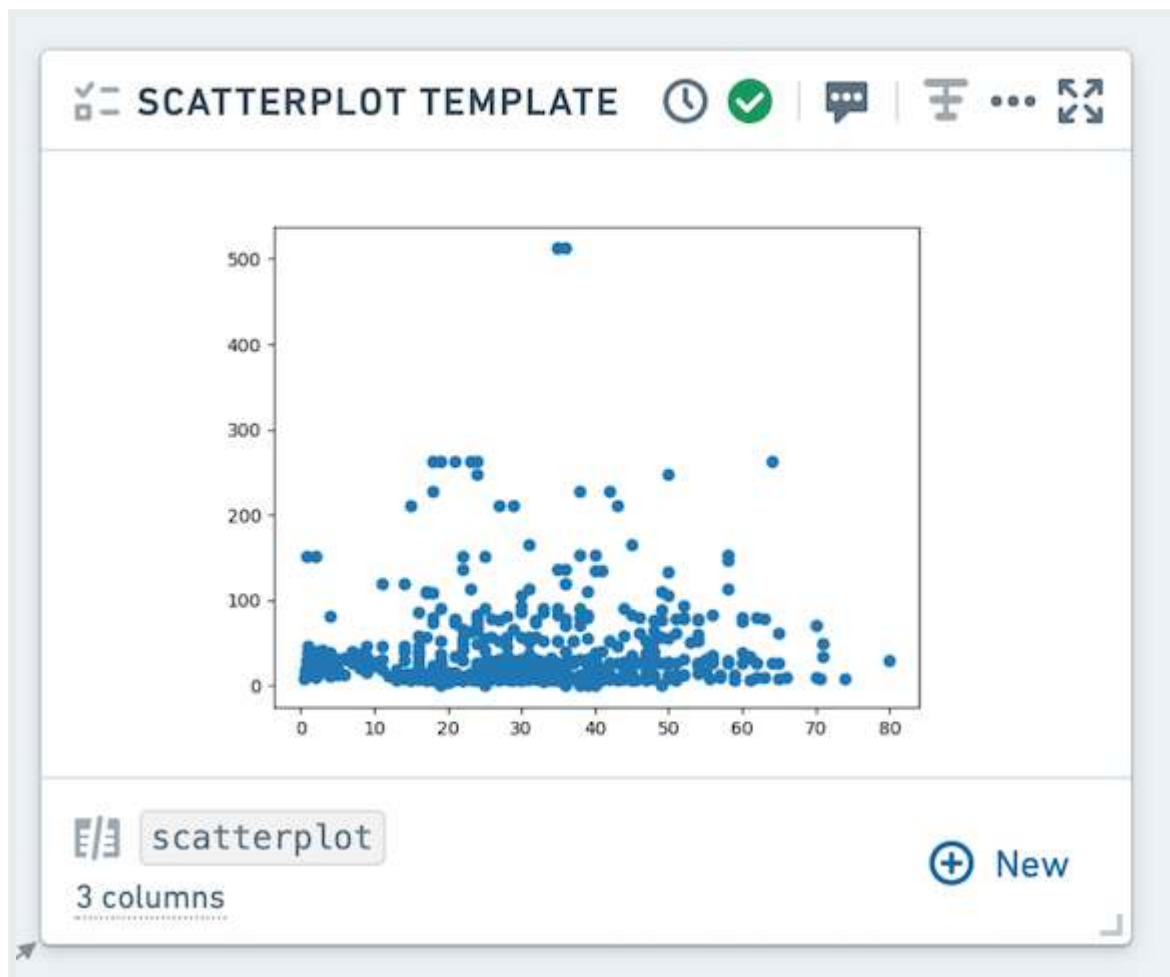
Visualizations

Depending on the language used in your transform, you can use plotting libraries to return visualizations that will propagate to the Code Workbook user interface. See the [Languages documentation](#) for details about how to do this in each language.

Resulting visualizations will be displayed in the graph by default, and are also available in the Visualizations bottom tab.

API Reference ↗

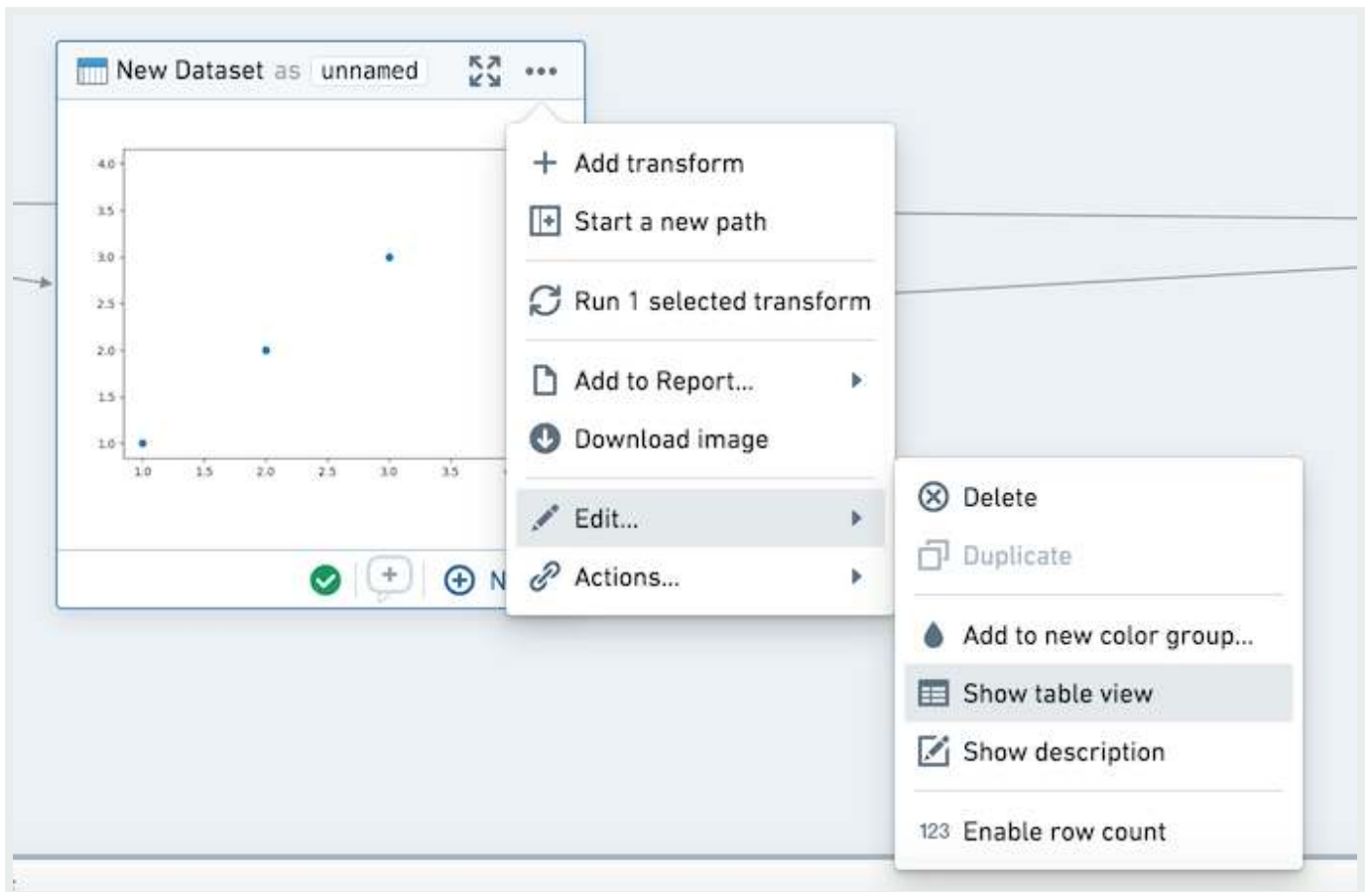
Send feedback



Using the context menu, you can toggle the view in the Graph between table and graph mode.

[API Reference ↗](#)

[Send feedback](#)



In the visualization tab, you can download your graph as an image or add it to a report. These actions are also available in the context menu.

Preview

bar_chart_of_row_counts

2 columns

Save as dataset

Toggle view

Edit template

Actions

</> Transformation Logic

Inputs

titanic_dataset

```

1 def bar_chart_of_row_counts(titanic_dataset):
2     import matplotlib.pyplot as plt
3     from pyspark.sql import functions as F
4     import numpy as np
5
6     input_df = titanic_dataset
7     categorical_column = "Sex"
8
9     # calculate the counts
10    total = input_df \
11        .groupBy(categorical_column) \
12        .agg(F.count("*").alias("count")) \
13        .orderBy("count")

```

Inputs

Preview

Visualizations

Logs

Description

OUTPUT VISUALIZATION

Download image

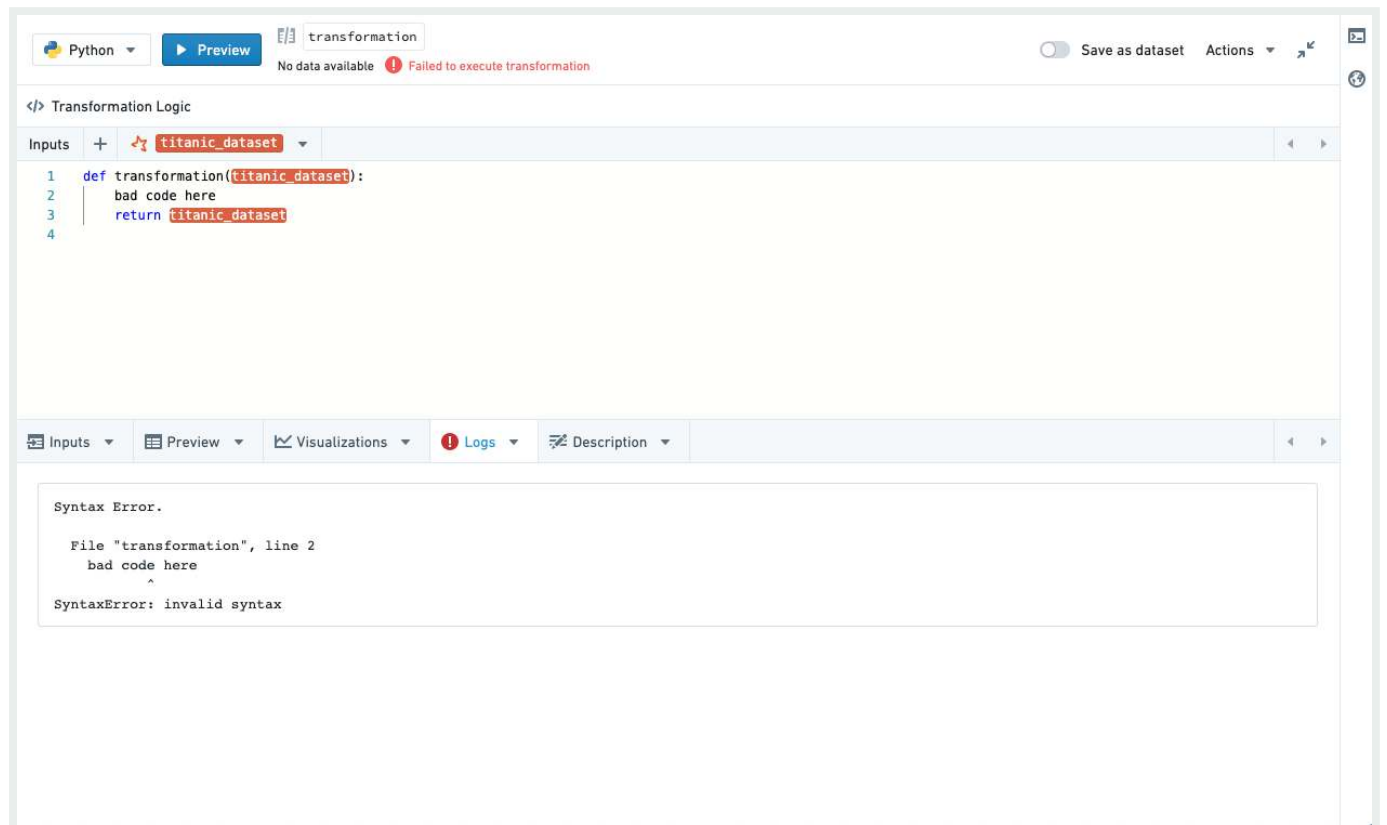
Add to report

API Reference ↗

Send feedback

Logs

View print statements (standard out), warnings and errors (standard error) from running your transform. If there is standard error, the logs tab will be automatically selected and a small callout will appear to notify you there is warning/error output.



Description

Update the description for your dataset. Note that updating the description will update it across all branches - the description will also be available in the **Open Dataset** view.

You can use markdown to style your descriptions, and view them in the graph by clicking enabling **Show Description** in the right-click (action) menu. To show descriptions for all nodes in a workbook, click the Tools cog in the header and select **Show all descriptions**.

Models

API Reference ↗

Transforms that return models will automatically be written to Foundry. To use the Foundry ML interface. Refer to the [Languages documentation](#).

Send feedback

how to use models in each language.

Non-deterministic transforms

Some transformations are non-deterministic — they return different results despite the same input data and transformation logic. Some functions that may lead to non-deterministic transforms include calls to the current timestamp, row number, monotonically increasing ID, and transformations after sorting on a column that contains duplicate values.

In Code Workbook, running a transform will start two jobs: one job calculates a 50 row preview, and the other job calculates the transformation on the full dataset and writes the result to Foundry. If there is a non-deterministic transform, the result you see in the 50 row preview may not match the result in the Foundry dataset.

When running non-deterministic code within a transformation, dataframes are not necessarily materialized. For example, assume you have a non-deterministic transform Transform A, which is the parent of Transforms B and C. If you run each transformation separately from within Code Workbook, each dataset will be written to Foundry sequentially, resulting in consistent results across transforms B and C. However, if you select "Run all downstream" on A, as A is not necessarily materialized when running B and C, you may see conflicting results. Running builds from within Data Lineage can prevent this as well, as each dataset will be materialized and sequentially calculated, as will converting to a Pandas dataframe.

Some examples of functions that result in non-deterministic transformations include `current_date()`, `current_timestamp()`, and `row_count()`, though other functions can result in non-determinism. More information on ways that Spark calculates transformations that may result in unexpected behavior can be found at the [Apache Spark docs ↗](#).

PREVIOUS

← [Workbooks / Moving to production](#)

NEXT

[Visualize data](#) →

[API Reference ↗](#) Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Send feedback](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Transforms / Visualize data

Visualize data

In Code Workbook, you can use open-source visualization libraries to display visualizations of your data. These visualizations can then be shared with others, for instance in [Notepad documents](#).

Python Visualizations

In Python, Code Workbook supports visualizations using Matplotlib, Seaborn, and Plotly.

Using Matplotlib and Seaborn

When using Matplotlib, a call to `matplotlib.pyplot.show()` causes the resulting plot image to be saved in the transform output and returned to the user interface, allowing the creation of customized plots. As with any visualization, you can download this image by right-clicking the transform in the graph and choosing **Download image**.

Here is an example of a transform that uses Matplotlib to render a visualization:

```
1 def viz_plot_univariate_distribution_using_histogram(input_dataset):
2     import matplotlib.mlab as mlab
3     import matplotlib.pyplot as plt
4
5     INPUT_DF = input_dataset
6     SELECTED_COLUMN = "column_to_plot" # Note this should be a numeric column
7     bins = number_of_bins
```

API Reference ↗



```
# Histogram the selected column
bins, counts = INPUT_DF.select(SELECTED_COLUMN).r
```

11

Send feedback

```

12     # Plot the histogram
13     fig, ax = plt.subplots()
14     ax.hist(bins[:-1], bins, weights=counts, density=True)
15
16     ax.set_xlabel(SELECTED_COLUMN)
17     ax.set_ylabel('Probability density')
18     ax.set_title(r'Histogram of ' + str(SELECTED_COLUMN))
19
20     # Tweak spacing to prevent clipping of ylabel
21     fig.tight_layout()
22     plt.show()

```

When using Seaborn, a data visualization library based on Matplotlib, you must call `matplotlib.pyplot.show()` to return the image to the frontend.

You can add Seaborn to your environment by [editing your profile](#) or by [customizing your workbook's environment](#).

```

1  def seaborn_example(pandas_df):
2      import seaborn as sns
3      import matplotlib.pyplot as plt
4
5      sns.set_theme()
6
7      # Create a visualization
8      sns.relplot(
9          data=pandas_df,
10         x="price", y="minimum_nights"
11     )
12
13     # This is necessary to capture the plot
14     plt.show()

```

```

1  def seaborn_violinplot(pandas_df):
2      import seaborn as sns
3      import matplotlib.pyplot as plt
4
5      sns.violinplot(x="col_A", y="col_B", data=pandas_df);
6      plt.show()

```

API Reference ↗

Send feedback

By default, the output of Matplotlib and Seaborn visualizations in Code Workbook will be in PNG format. To output Matplotlib and Seaborn visualizations in SVG format, use the following code before your plot:

```
set_output_image_type('svg')
```

Or, use a hint for better visibility:

```
1 @output_image_type('svg')
2 def chart(input):
3     # create chart here
```

Plotting with different languages and fonts using Matplotlib

To plot labels and text in languages using non-Roman characters (such as Japanese or Korean) or in non-default fonts using Matplotlib, you must specifically specify which font family you would like Matplotlib to use when rendering images. For more information, refer to the [list of available fonts installed by default](#).

Here is an example of how to specify fonts for Matplotlib:

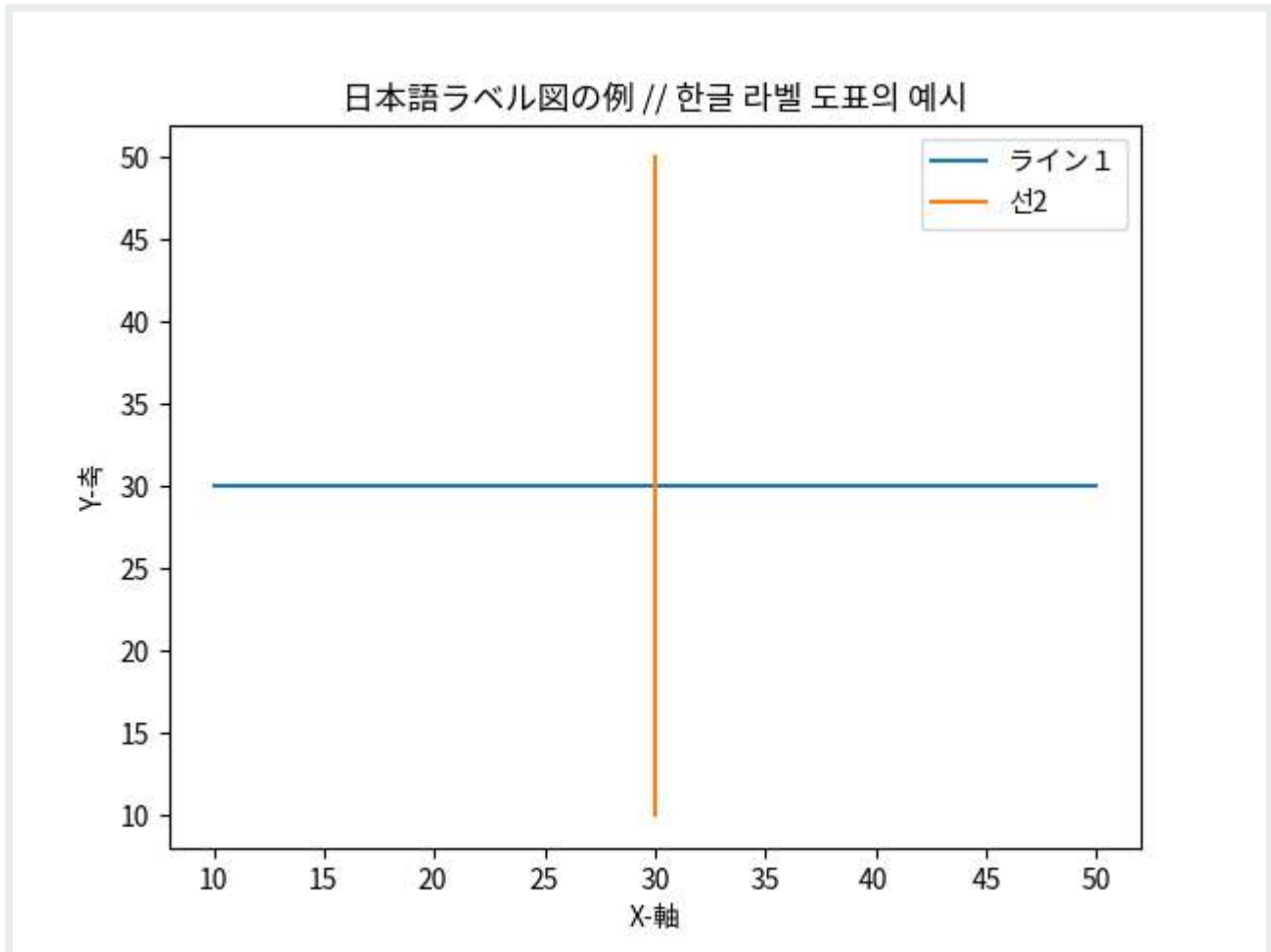
```
def japanese_korean_matplotlib_example():
    import matplotlib.mlab as mlab
    import matplotlib.pyplot as plt
    from matplotlib import rcParams
    # Set font family to the Noto Sans CJK font pack
    rcParams['font.family'] = 'Noto Sans CJK JP'
    # create data
    x = [10,20,30,40,50]
    y = [30,30,30,30,30]
```

API Reference ↗

```
plt.plot(x, y, label = "ライン1")
plt.plot(y, x, label = "선2")
```

Send feedback

```
plt.xlabel("X- ")
plt.ylabel("Y-축")
plt.legend()
plt.title("      ラベル の  // 한글 라벨 도표의 예시")
plt.show()
```



❗ Matplotlib 2.* Font Designation

For Matplotlib 2.*, .ttc font files are not detected by Matplotlib automatically. Either upgrade to 3.* or directly add the font filepath to Matplotlib's font manager.

```
1 from matplotlib import rcParams
2 import matplotlib.font_manager as fm
3 fm.fontManager.ttflist += fm.createFontList(["/usr/share/fonts/opentyp
4 rcParams['font.family'] = 'Noto Sans CJK JP'</pre>
```

[API Reference ↗](#)

Using Plotly

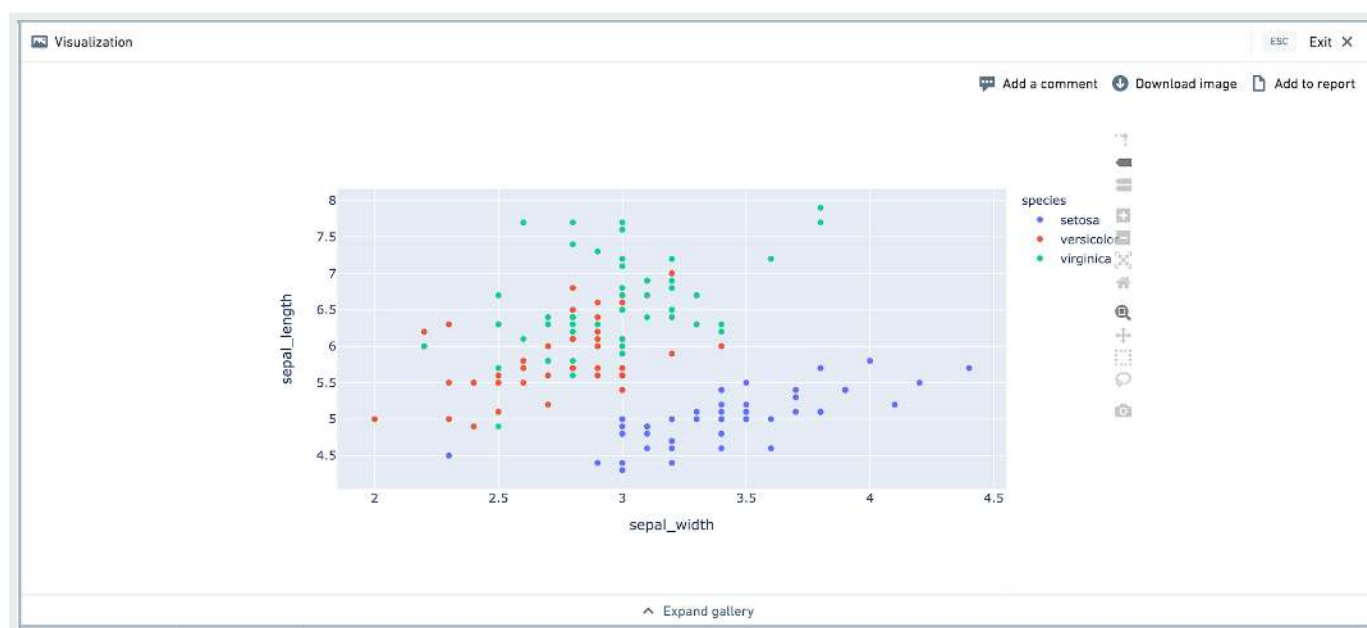
[Send feedback](#)

Plotly is a visualization library that allows you to create interactive images. To use Plotly, first make sure it is included in your environment.

A call to `fig.show()` causes the resulting plot image to be saved as part of the transform output and returned to the user interface. Here is an example of a transform that uses Plotly to render a visualization, using Plotly Express. Plotly Express comes pre-loaded with the iris dataset.

```
1 def plotly_example():
2     import plotly.express as px
3     df = px.data.iris()
4     fig = px.scatter(df, x = "sepal_width", y = "sepal_length", color = "species")
5     fig.show()
```

After running the transform, the Plotly visualization will appear in the visualization tab. We recommend viewing the visualization in full screen mode. You are able to use functionality like zooming in and out, selection on the graph, and so on.



Here is a more complex example, which produces an animated visualization.

API Reference ↗

```
def example_2():
    import plotly.graph_objects as go
    fig = go.Figure(
        data=[go.Scatter(x=[0, 1], y=[0, 1])],
        layout=go.Layout(
```

Send feedback

```

6     xaxis=dict(range=[0, 5], autorange=False),
7     yaxis=dict(range=[0, 5], autorange=False),
8     title="Start Title",
9     updatemenus=[dict(
10         type="buttons",
11         buttons=[dict(label="Play",
12             method="animate",
13             args=[None])]]
14     ),
15     frames=[go.Frame(data=[go.Scatter(x=[1, 2], y=[1, 2])]),
16         go.Frame(data=[go.Scatter(x=[1, 4], y=[1, 4])]),
17         go.Frame(data=[go.Scatter(x=[3, 4], y=[3, 4])]),
18         layout=go.Layout(title_text="End Title"))
19     )
20
21     fig.show()

```

R Visualizations

In R, Code Workbook supports visualizations using ggplot2 and plotly.



Using ggplot2

[API Reference ↗](#)

```

1 fare_distribution <- function(titanic_dataset) {
2     hist(titanic_dataset$Fare)

```

[Send feedback](#)

```
3   return(titanic_dataset)
4 }
```

```
1 example_ggplot <- function() {
2   library(ggplot2)
3   theme_set(theme_bw()) # pre-set the bw theme
4   data("midwest", package = "ggplot2")
5
6   # Scatterplot
7   gg <- ggplot(midwest, aes(x=area, y=poptotal)) +
8     geom_point(aes(col=state, size=popdensity)) +
9     geom_smooth(method="loess", se=F) +
10    xlim(c(0, 0.1)) +
11    ylim(c(0, 500000)) +
12    labs(subtitle="Area Vs Population",
13         y="Population",
14         x="Area",
15         title="Scatterplot",
16         caption = "Source: midwest")
17
18   plot(gg)
19   return(NULL)
20 }
```

By default, ggplot visualizations will be outputted as PNGs. To produce R ggplot visualizations in SVG format, add a hint using a comment:

```
1 fare_distribution <- function(titanic_dataset) {
2   # image: svg
3   hist(titanic_dataset$Fare)
4   return(titanic_dataset)
5 }
```

To customize the PNG or SVG output, you can call the `png()` or `svg()` function using the built-in `graphicsFile` variable as filename.

[API Reference ↗](#)

```
1 unnamed_1 <- function() {
2   png(
3     filename=graphicsFile,
```

Send feedback

```

4     width=800,
5     height=400,
6     units="px",
7     pointsize=4,
8     bg="white",
9     res=300,
10    type="cairo")
11
12    plot(1:10, 1:10)
13 }

```

Note that if you want to use a custom `svg()` function, you'll also need to provide a comment hint described above.

```

1 unnamed_1 <- function() {
2   # image: svg
3   svg(
4     filename=graphicsFile,
5     width=5,
6     height=9,
7     pointsize=4,
8     bg="white")
9
10    plot(1:10, 1:10)
11 }

```

Using Plotly

[Plotly](#) allows you to make interactive graphs. To use Plotly in R, add the `r-plotly` package to your environment. Plot graphs with `plot()` or `print()` to show them on the frontend. Here's a simple example:

```

1 plotly_example <- function() {
2   library(plotly)
3
4   plotly <- plot_ly (
5     x = rnorm(1000),
6     y = rnorm(1000),
7     mode = "markers",

```

[API Reference](#)

Send feedback


```

8         type = "scatter"
9     )
10    plot(scatter_plotly)
11 }

```

Plotly limitations

The following notes apply to both Python and R.

- In the console, Plotly visualizations will be converted to images and displayed as PNGs. They will not be interactive. To view an interactive visualization, write the code in a transform.
- When creating Plotly visualizations, visualizations with more than 20,000 points are not recommended due to degraded browser performance. If creating a scatterplot with a large number of points, use `scattergl` for better performance.

Matplotlib limitations

The following limitations on Matplotlib apply to Python.

- [Matplotlib is not thread safe.](#)
 - When running multiple nodes, Spark will run these computations in parallel. As a result, unintended behavior may be revealed in the form of Matplotlib Runtime Exceptions or visualizations created in incorrect nodes.
- When using multiple Matplotlib visualizations in separate nodes within a single Code Workbook, you must lock each node. You can lock each node with the thread-safe decorator `@synchronous_node_execution`, as shown below.

```

1 import matplotlib.mlab as mlab
2 import matplotlib.pyplot as plt
3 from matplotlib import rcParams
4 @synchronous_node_execution
5     safe_node():
6         # Set font family to the Noto Sans CJK font pack
7         rcParams['font.family'] = 'Noto Sans CJK JP'
8         # create data
9         x = [10,20,30,40,50]

```

API Reference ↗

Send feedback

```
10     y = [30,30,30,30,30]
11
12     # plot lines
13     plt.plot(x, y, label = "라인 1")
14     plt.plot(y, x, label = "선2")
15     plt.xlabel("X- ")
16     plt.ylabel("Y-축")
17     plt.legend()
18     plt.title("      라벨  の  // 한글 라벨 도표의 예시")
19     plt.show()
```

PREVIOUS

← Overview

NEXT

Present visualizations →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Transforms / Present visualizations

Present visualizations

Once you have created a visualization in Code Workbook, you may want to present it to collaborators outside the context of a Code Workbook.

Add Visualizations to Notepad

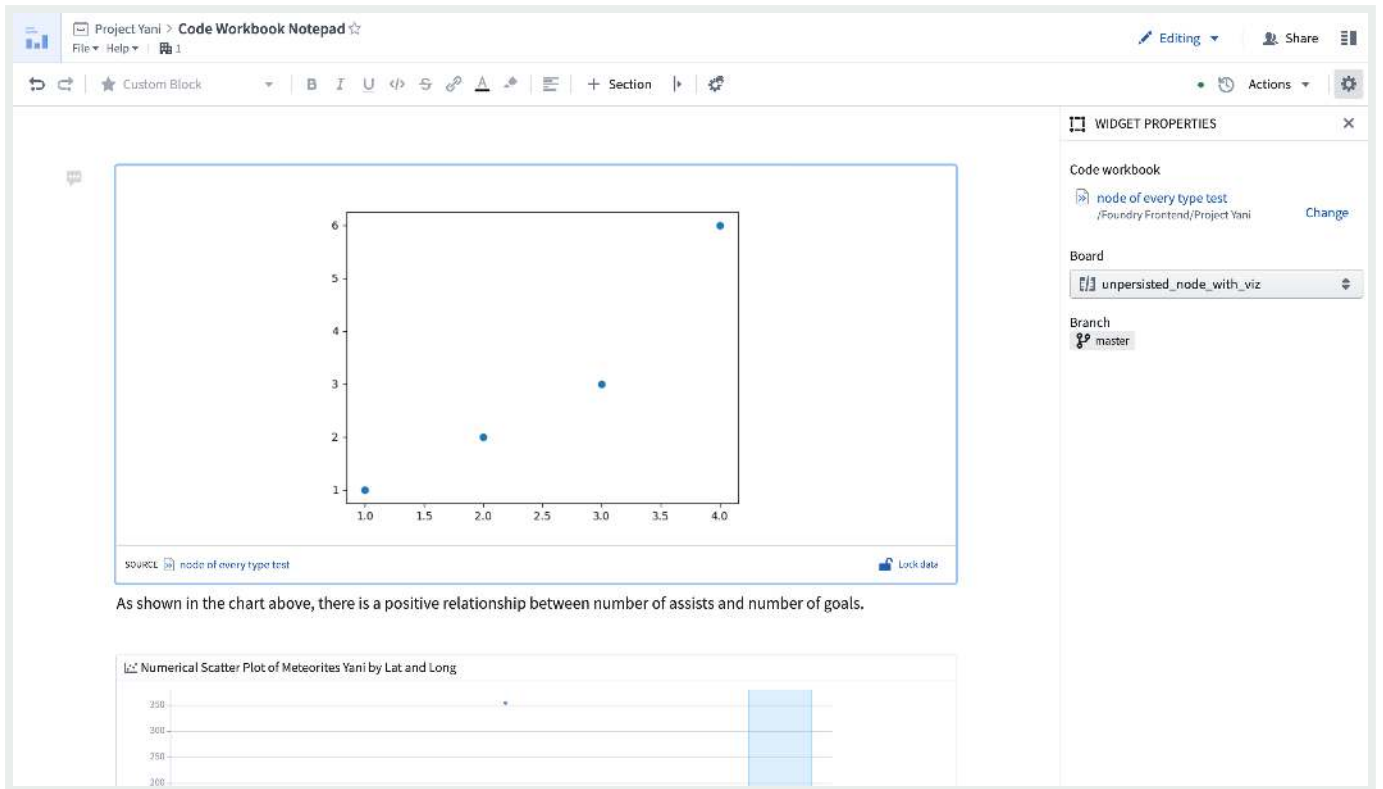
Visualizations generated in a Code Workbook can be presented in a Notepad document. You can embed various resources from across Foundry into a Notepad document, including Contour boards, Quiver canvases, and Code Workbook visualizations, and can annotate the document using text, tables, and images. You can also create Notepad templates, allowing users in a Workshop application to generate new Notepad documents based on a given blueprint. [Learn more about Notepad.](#)

You can add a Code Workbook visualization to a Notepad document by right-clicking a node on the graph and clicking **Copy for Notepad**. Now, the image is copied to your clipboard and can be pasted into any Notepad document.

API Reference ↗



Send feedback



Adding Visualizations to Reports

⚠ Foundry Reports availability

As of October 3rd, 2022, Foundry Reports is no longer available to new customers by default. Contact your Palantir representative if you have a use case that requires access to Foundry Reports. For existing users, Foundry Reports is still supported for creating dashboards with visualizations produced in a Code Workbook.

Visualizations generated in a Code Workbook can be added to a Foundry Report for presentation and collaboration. You can add a visualization to a Report by opening the Visualizations tab and clicking on **Add to Report**, or by right-clicking a node on the graph and clicking **Add to Report**.

PREVIOUS



API Reference ↗

NEXT

Access unstructured files



Send feedback

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Access unstructured files

In addition to operating over Foundry datasets that have a defined tabular schema, Code Workbook supports accessing unstructured files in a dataset. This can be useful for analyzing and transforming unstructured data such as images and other types of media, semi-structured formats such as XML or JSON, compressed formats such as GZ or ZIP files, or R data formats like RDA and RDS.

Unstructured files in Python

Reading files

You can read files in a Python transform by reading an upstream dataset as a `Python transform input`. This API exposes a `FileSystem` object that allows file access based on the path of a file within the Foundry dataset, abstracting away the underlying storage. [Learn more about the `FileSystem`](#). Other information, including the branch and RID (as detailed in the [transform input documentation](#)), is also exposed.

Change the type of your input using the input helper bar, or in the inputs tab.

i Only imported datasets and persisted datasets can be read in as Python transform inputs. Transforms that are not saved as a dataset cannot be read in as Python transform inputs.

Datasets with no schema should be read in as a transform input automatically.

API Reference ↗



Send feedback



Example: Reading CSVs inside a ZIP file

For example, the following code will read the CSVs inside of a ZIP file and return the CSV contents as a dataframe.

```

1 import tempfile
2 import zipfile
3 import shutil
4 import io
5 from pyspark.sql import Row
6
7 # datasetOfZippedFiles is a dataset with a single zipped file that contains
8 def sample(datasetOfZippedFiles):
9     df = datasetOfZippedFiles
10    fs = df.filesystem() # This is the FileSystem object.
11    MyRow = Row("id", "name")
12    def process_file(file_status):
13        with fs.open(file_status.path, 'rb') as f:
14            with tempfile.NamedTemporaryFile() as tmp:
15                shutil.copyfileobj(f, tmp)
16                tmp.flush()
17                with zipfile.ZipFile(tmp) as archive:
18                    for filename in archive.namelist():
19                        with archive.open(filename) as f2:
20                            br = io.BufferedReader(f2)
21                            tw = io.TextIOWrapper(br)
22                            tw.readline() # Skip the first line of each CSV
23                            for line in tw:
24                                yield MyRow(*line.split(","))
25
26    rdd = fs.files().rdd
27    rdd = rdd.flatMap(process_file)
  
```

API Reference ↗

Send feedback

```
27     df = rdd.toDF()
28     return df
```

Writing Files

It is possible to write to an output FileSystem. This can be useful to write non-tabular data formats including images, PDFs, text files, etc.

Call `Transforms.get_output()` to instantiate a `TransformOutput`. [Learn more about the TransformOutput API.](#)

- i** You can only write files using `TransformOutput` in nodes that are saved as datasets. You cannot write files using `TransformOutput` in the console.

- i** Once you have instantiated a `TransformOutput` and used it by calling `filesystem()` or other methods, returning anything other than the `TransformOutput` object will be ignored.

Example: Writing a text file or dataset

The following code is an example of how to write a text file:

```
1 def write_text_file():
2     output = Transforms.get_output()
3     output_fs = output.filesystem()
4
5     with output_fs.open('my text file.txt', 'w') as f:
6         f.write("Hello world")
7         f.close()
8
```



[API Reference ↗](#)

The following code is an example of how to write a dataset and specify output format.

[Send feedback](#)


```

1 def write_dataset(input_dataset):
2     output = Transforms.get_output()
3     output.write_dataframe(input_dataset, partition_cols = ["colA", "colB"]

```

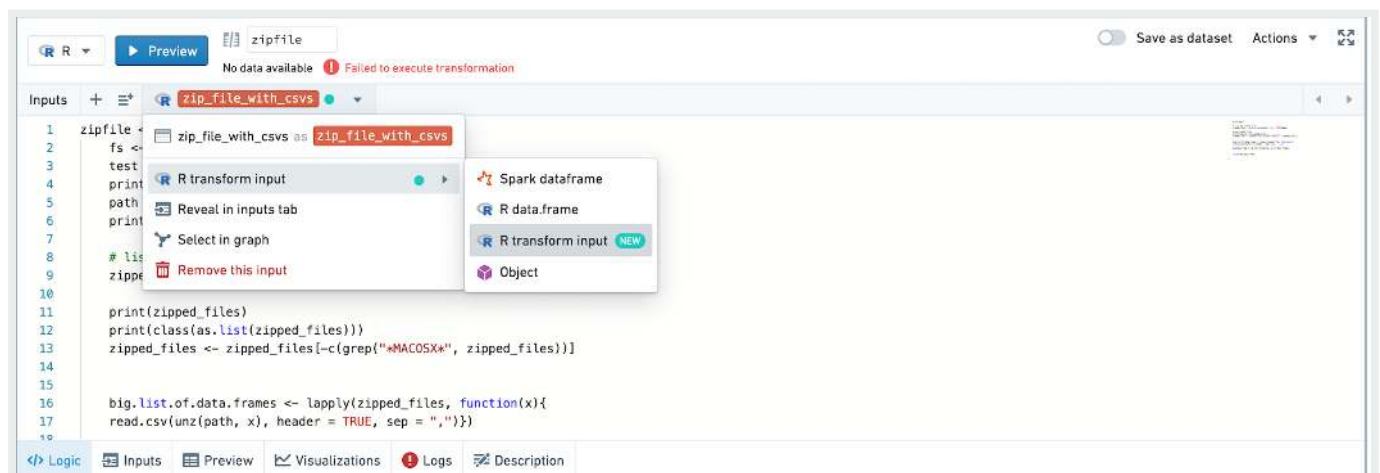
Unstructured files in R

Reading files

You can read files in an R transform by reading an upstream dataset as an **R transform input**. The **TransformInput** object is exposed which allows file access based on the path of a file within the Foundry dataset. [Learn more about the **FileSystem** API.](#)

Change the type of your input using the input helper bar, or in the inputs tab.

- i** Only imported datasets and persisted datasets can be read in as R transform inputs. Transforms that are not saved as a dataset cannot be read in as R transform inputs.
By default, datasets without schemas should be set to input type R transform input already.



API Reference ↗

g an RDS

Send feedback

Use the code below to load an RDS that is a file in an imported dataset. The RDS contains an R data.frame.

```
1 RDS_reader <- function(RDS_dataset) {  
2   fs <- RDS_dataset$fileSystem()  
3  
4   ## The name of the file is test_loading_RDS.rds  
5  
6   path <- fs$get_path("test_loading_RDS.rds", 'r')  
7   rds <- readRDS(path)  
8   return(rds)  
9 }
```

Example: Using rbind on the contents of a set of zipped CSVs

Use the code below to `rbind` the contents of a set of zipped CSVs.

```
1 result <- function(zip_file_with_csvs) {  
2   fs <- zip_file_with_csvs$fileSystem()  
3  
4   ## Get the remote path (name) of the zipfile  
5   zipfile_name <- fs$ls()[[1]]$path  
6  
7   ## Get the local path of the zipfile  
8   path <- fs$get_path(zipfile_name, 'r')  
9  
10  # List the zipped files  
11  zipped_files <- as.list(unzip(path, list = TRUE)$Name)  
12  
13  # For every element on the list, return a dataframe  
14  list_of_data_frames <- lapply(zipped_files, function(x){read.csv(unz(  
15  
16  # Bind all of the dataframes together  
17  rbind_df <- do.call(rbind,list_of_data_frames)  
18  
19  return(rbind_df)  
20 }
```

[API Reference ↗](#)

[Send feedback](#)

Writing files

It is possible to write to an output FileSystem. This can be useful to write non-tabular data formats including images, PDFs, text files, and so on.

Call `new.output()` to instantiate a TransformOutput. [Learn more about the FileSystem API.](#)

-  You can only write files using TransformOutput in nodes that are saved as datasets. You cannot write files using TransformOutput in the console.

Example: Saving an R data.frame to an RDS file

Use the code below to save an R data.frame to an RDS file.

```
1 write_rds_file <- function(r_dataframe) {  
2   output <- new.output()  
3   output_fs <- output$filesystem()  
4   saveRDS(r_dataframe, output_fs$get_path("my_RDS_file.rds", 'w'))  
5  
6 }
```

Example: Saving a plot to a PDF

Use the code below to save a plot to a PDF.

```
1 plot_pdf <- function() {  
2   library(ggplot2)  
3   theme_set(theme_bw()) # pre-set the bw theme  
4   data("midwest", package = "ggplot2")  
5  
6   # Scatterplot  
7   ggplot(midwest, aes(x=area, y=poptotal)) +  
8     geom_point(aes(col=state, size=popdensity)) +  
9     geom_smooth(method="loess", se=F) +  
10    xlim(c(0, 0.1)) +  
11    ylim(c(0, 500000)) +
```

API Reference ➤

Send feedback

```

12     labs(subtitle="Area Vs Population",
13           y="Population",
14           x="Area",
15           title="Scatterplot",
16           caption = "Source: midwest")
17
18     output <- new.output()
19     output_fs <- output$fileSystem()
20     pdf(output_fs$get_path("my pdf example.pdf", 'w'))
21     plot(gg)
22 }

```

Example: Writing a TXT file using a connection

Use the code below to write a TXT file using a connection.

```

1 write_txt_file <- function() {
2   output <- new.output()
3   output_fs <- output$fileSystem()
4   conn <- output_fs$open("my file.txt", 'w')
5   writeLines(c("Hello", "world"), conn)
6 }

```

Example: Uploading a TXT file to a remote path

Use the code below to take the text file at the local path `output.txt`, and upload it to the remote path `output_test.txt`. In the saved dataset, you will see one file named `output_test.txt`

```

1 upload <- function() {
2   output <- new.output()
3   output_fs <- output$fileSystem()
4   fileConn<-file("output.txt")
5   writeLines(c("Header 1"), fileConn)
6   close(fileConn)
7   output_fs$upload("output.txt", "output_test.txt")
8 }

```

API Reference ↗

Send feedback

```
1 write_txt_file <- function() {  
2   output <- new.output()  
3   output_fs <- output$fileSystem()  
4   conn <- output_fs$open("my file.txt", 'w')  
5   writeLines(c("Hello", "world"), conn)  
6 }
```

Example: Writing a Spark dataframe with partitions

Use the code below to write a Spark dataframe that is partitioned by columns A and B.

```
1 write_partitioned_df <- function(spark_df) {  
2   output <- new.output()  
3  
4   # partition on colA and colB  
5   output$write.spark.df(spark_df, partition_cols=list("colA", "colB"))  
6 }
```

PREVIOUS

← Present visualizations

NEXT

Spark →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—Cookies Settings

[API Reference ↗](#)

Send feedback

Analytics / Code Workbook [Legacy] / Transforms / Spark

Spark

[Apache Spark ↗](#) is a commonly-used open-source engine used for data engineering, data science, and machine learning workflows on large-scale data.

Code Workbook natively supports accessing data in Spark across all languages. For detailed information, refer to the following pages:

- [PySpark Reference](#)
- [SparkR Reference](#)

PREVIOUS

← [Access unstructured files](#)

NEXT

[Transforms FAQ](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)



Send feedback

Transforms FAQ

The following are some frequently asked questions about transforms.

For general information, see our [transforms documentation](#).

- [Is it possible to save a CSV file in `transforms-python` rather than saving Parquet?](#)
- [Can I build multiple output datasets from one Python transform?](#)
- [How can I open a GZIP file with transforms?](#)
- [How can I unzip a file as part of a Foundry pipeline? In parallel?](#)

Is it possible to save a CSV file in `transforms-python` rather than saving Parquet?

Below are examples of how to do this in each transform language:

Java

```
1 foundryOutput.getDataFrameWriter(dataFrame)
2 .setFormatSettings(DatasetFormatSettings.builder().format("csv").build())
```

API Reference ↗



Send feedback

Python

```
1 from transforms.api import transform, Input, Output
2 @transform(
3     output=Output("/path/to/python_csv"),
4     my_input=Input("/path/to/input")
5 )
6 def my_compute_function(output, my_input):
7     output.write_dataframe(my_input.dataframe(), output_format="csv")
```

SQL

```
1 CREATE TABLE `/path/to/sql_csv` USING CSV AS SELECT * FROM `/path/to/input`
```

[Return to top](#)

Can I build multiple output datasets from one Python transform?

If you want multiple transforms/datasets, you can create them using a `for` loop:

[API Reference ↗](#)



Send feedback


```

1 from transforms.api import transforms_df, Input, Output
2
3 def transform_generator(sources):
4     #type: (List[str]) -> List([transforms.api.Transform])
5     transforms = []
6     # This example uses multiple input datasets. You can also generate
7     # from a single input dataset.
8     for source in sources:
9         @transforms_df(
10             Output('/sources/{source}/output'.format(source=source)),
11             my_input=Input('/sources/{source}/input'.format(source=source))
12         )
13         def compute_function(my_input, source=source):
14             # To capture the source variable in the function, you pass it
15             return my_input.filter(my_input.source == source)
16
17         transforms.append(compute_function)
18
19     return transforms
20
21 TRANSFORMS = transform_generator(['src1', 'src2', 'src3'])

```

You can now import the `TRANSFORMS` attribute of the module and manually add each transform to your pipeline:

```

1 import my_module
2
3 my_pipeline = Pipeline()
4 my_pipeline.add_transforms(*my_module.TRANSFORMS)

```

To have a single transform that takes in one input and outputs multiple datasets in the same build, you can also do this programmatically as below:

[API Reference ↗](#)

```

1 # Using the `/examples/students_hair_eye_color` dataset
2 students_input = foundry.input('/examples/students_hair_eye_color')
3 students_input.dataframe().sort('id').show(n=3)

```

[Send feedback](#)

```

4  +---+-----+-----+-----+
5  | id| hair| eye| sex|
6  +---+-----+-----+-----+
7  |  1|Black|Brown|Male|
8  |  2|Brown|Brown|Male|
9  |  3| Red|Brown|Male|
10 +---+-----+-----+-----+
11 Note that this example only shows the top three rows.
12
13 from transforms.api import transform, Input, Output
14
15 @transform(
16     hair_eye_color=Input('/examples/students_hair_eye_color'),
17     males=Output('/examples/hair_eye_color_males'),
18     females=Output('examples/hair_eye_color_females'),
19 )
20 def brown_hair_by_sex(hair_eye_color, males, females):
21     # type: (TransformInput, TransformOutput, TransformOutput) -> None
22     brown_hair_df = hair_eye_color.dataframe().filter(hair_eye_color.data
23
24     males.write_dataframe(brown_hair_df.filter(brown_hair_df.sex == 'Male
25     females.write_dataframe(brown_hair_df.filter(brown_hair_df.sex == 'Fe

```

For more help and information on transforms, review the documentation for:

- [Python transforms](#)
- [Java transforms](#)
- [Spark SQL transforms](#)

[Return to top](#)

How can I open a GZIP file with transforms?

Since [TransformInput](#) is a file-like object that is backed by a stream, you can process it as a file. This means you do not need to be concerned about reading the entire file into memory or copying it on to a disk, allowing for usage of much larger files.

[API Reference ↗](#)

[Send feedback](#)

Use the `gzip` and `io` packages included in Python 3:

```
1 import gzip, io
2
3 def process_file(file_status):
4     fs = input_dataset.filesystem()
5     with fs.open(file_status.path, 'rb') as f:
6         gz = gzip.GzipFile(fileobj=f)
7         br = io.BufferedReader(gz)
```

And if you want reads to return strings, you can wrap it:

```
1 tw = io.TextIOWrapper(br)
```

If your file has an encoding you can specify it:

```
1 tw = io.TextIOWrapper(br, encoding='CP500')
```

For more help and information on transforms, review the documentation for:

- [Python transforms](#)
- [Java transforms](#)
- [Spark SQL transforms](#)

[Return to top](#)

How can I unzip a file as part of a Foundry pipeline?

In parallel?

API Reference ↗

This uses Java and Spark to unzip each file within the zip archive in a parallel fashion. If you want to parallelize decompression within a single compressed file format like `.bz2`.

Send feedback

```

1 package com.palantir.transforms.java.examples;
2
3 import com.google.common.io.ByteStreams;
4 import com.palantir.transforms.lang.java.api.Compute;
5 import com.palantir.transforms.lang.java.api.FoundryInput;
6 import com.palantir.transforms.lang.java.api.FoundryOutput;
7 import com.palantir.transforms.lang.java.api.ReadOnlyLogicalFileSystem;
8 import com.palantir.transforms.lang.java.api.WriteOnlyLogicalFileSystem;
9 import com.palantir.util.syntacticpath.Paths;
10 import java.io.IOException;
11 import java.util.zip.ZipEntry;
12 import java.util.zip.ZipInputStream;
13
14 /**
15  * This is an example of unzipping files in parallel using Spark.
16  * <p>
17  * The work is distributed to executors.
18  */
19 public final class UnzipWithSpark {
20
21     @Compute
22     public void compute(FoundryInput zipFiles, FoundryOutput output) throws IOException {
23         ReadOnlyLogicalFileSystem inputFileSystem = zipFiles.asFiles().getReadOnlyLogicalFileSystem();
24         WriteOnlyLogicalFileSystem outputFileSystem = output.getWriteOnlyLogicalFileSystem();
25
26         inputFileSystem.filesAsDataset().foreach(portableFile -> {
27             // "processWith" gives you the InputStream for the given input file
28             portableFile.processWithThenClose(stream -> {
29                 try (ZipInputStream zis = new ZipInputStream(stream)) {
30                     ZipEntry entry;
31                     // For each file in the zip file, write it to the output
32                     while ((entry = zis.getNextEntry()) != null) {
33                         outputFileSystem.writeTo(
34                             Paths.get(entry.getName()),
35                             outputStream -> ByteStreams.copy(zis, outputStream)
36                         );
37                     }
38                     return null;
39                 } catch (IOException e) {
40                     throw new RuntimeException(e);
41                 }
42             });
43         });
44     }

```

API Reference ↗

Send feedback

For more help and information on transforms, review the documentation for:

- [Python transforms](#)
- [Java transforms](#)
- [Spark SQL transforms](#)

[Return to top](#)

PREVIOUS
← Spark

NEXT
Templates / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

API Reference ↗

Send feedback

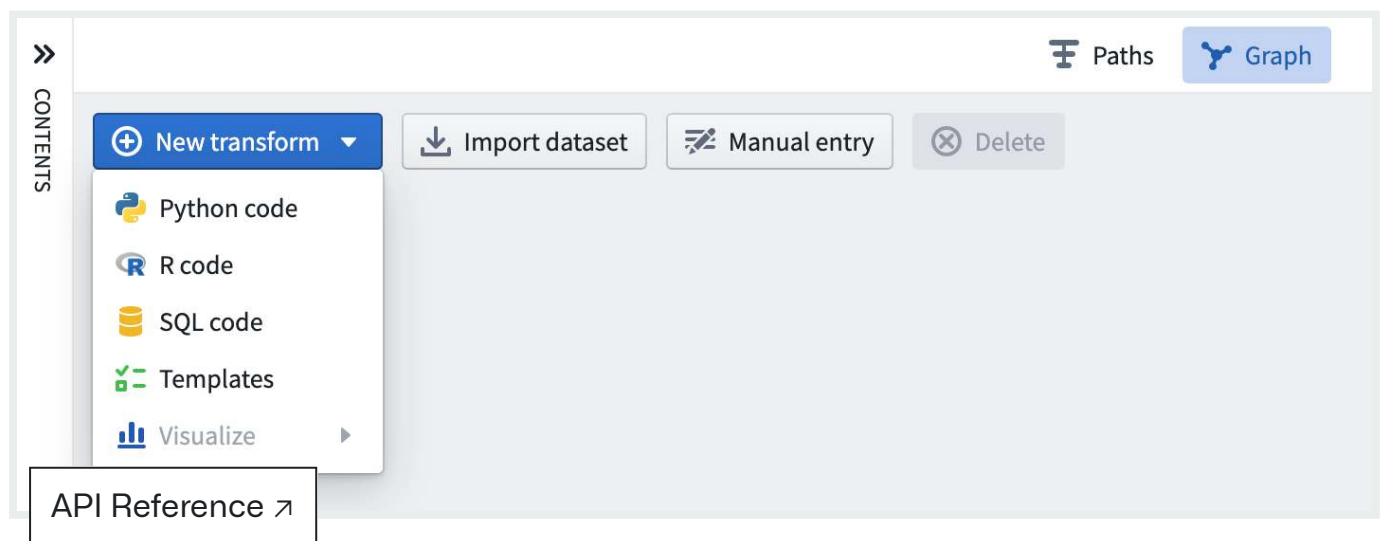
Templates

Code templates enable users with a range of technical experience to collaborate by abstracting code away behind a simple form-based interface. Values selected by users are substituted into a code template, which can then be run like any other transform in the Workbook.

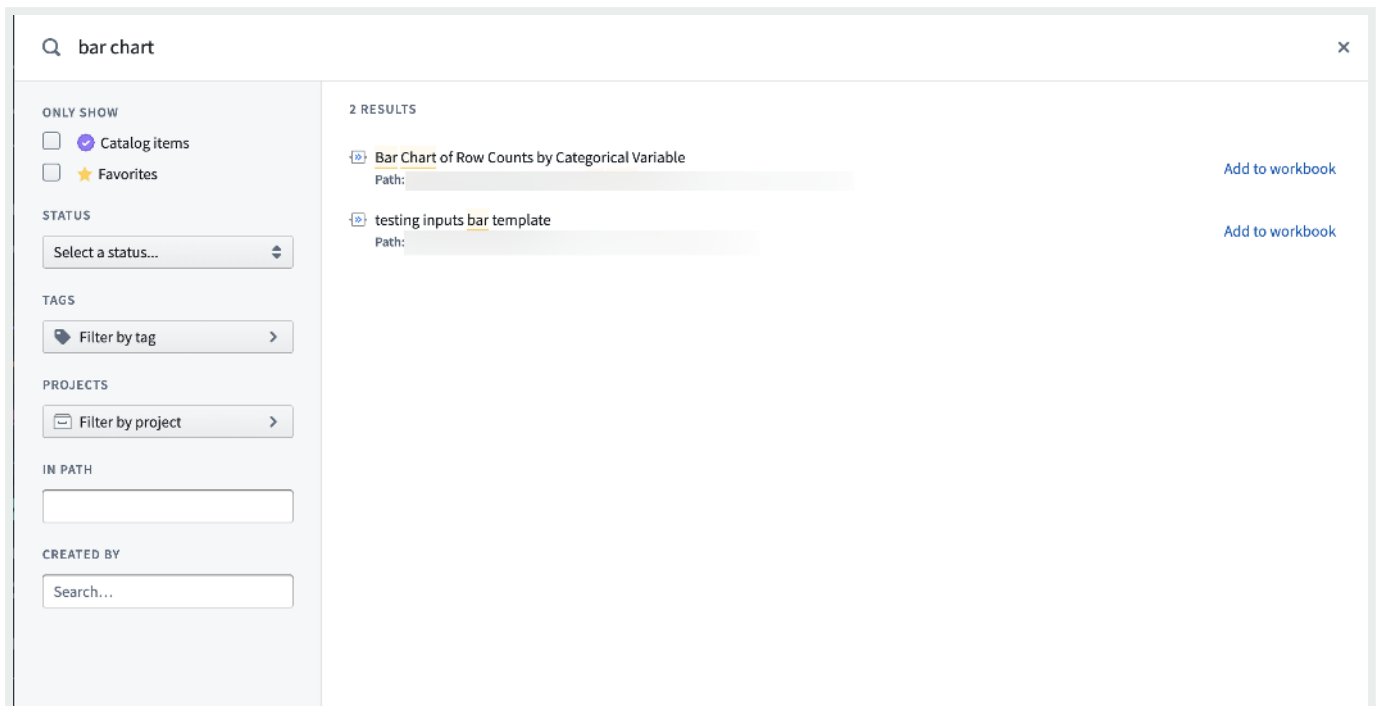
Any transform that can be created with code, including Matplotlib or Plotly visualizations, can be created with code templates. To try using templates hands-on, follow the [Templates tutorial](#).

Search and discovery

All available templates can be found in the template browser. To open the template browser, click **+ New Transform** at the top left of the screen and select **Templates**, or select the **+** button on hover of an existing transform and select **Templates**.



The browser displays recently used templates, and provides search based on keyword, template author, tags, and more.



Using code templates

When a code template is selected on the graph, the user is presented with a simple form input on the bottom of the screen rather than a code editor. Parameters in the template code are rendered as inputs to be filled in by the user.



Input types

API Reference [↗](#) The value for a dataset field must be a valid dataset in the current workbook. When a user clicks the dataset input field, they will be prompted to select an input dataset from the graph. Dataset inputs can also be configured to

[Send feedback](#)

same input dataset; for example, a code template might always rely on a particular ontology dataset. For more, see the [Creating and editing templates](#) section below.

- **Column:** The user is prompted to select one or more columns from one of the input datasets. Column parameters can be configured to select a single column or a list of columns.
- **Text:** A text input field that allows users to enter free text values.
- **Number:** An input field that must be filled by a numerical value.
- **Select:** Allows users to choose from a pre-defined list of possible values.
- **Multiselect:** Allows users to choose any number of values from a pre-defined list of possible values.
- **Boolean:** A switch that lets users toggle a value between True and False.
- **List:** Allows users to enter any number of values, which are passed to the underlying code as a list of values.

Once the input fields have been filled, a template transform can be run in the same way as a code transform: by selecting the **Run** button, or by opening the context menu on the graph and selecting **Run**.

Use **Toggle View** to preview the underlying code generated by the template.

Creating and editing templates

Any code transformation can be turned into a code template to allow the same transformation to be applied without re-writing any code. To create a code template or edit an existing one, select the **Create/Edit template** option on the code editor header.

Code templates are created by designating values within a code transformation as parameters, and adding metadata about what types of input a user should be allowed to enter. When you choose to create a template, you will be brought into a split-screen view in the Full Screen Editor. On the left-hand side of the screen, the code editor allows you to select pieces of code to parameterize. On the right-hand side of the screen, the template editor allows you to configure metadata about each parameter.

[API Reference ↗](#)

[Send feedback](#)

</> Transformation Logic

View templateActionsESCExit full screen

Edit "bar_chart_example"

```
1 def bar_chart('{{titanic_dataset}}'):
2     input_df = '{{titanic_dataset}}'
3     test_df2 = input_df
4     categorical_column = '{{param1}}'
5
6     # calculate the counts
7     total = test_df2 \
8         .groupBy(categorical_column) \
9         .agg(F.count('*').alias("count")) \
10        .orderBy("count")
11
12    # convert summarized dataset to pandas
13    total_pdf = total.toPandas()
14
15    # plotting code
16    fig = plt.figure()
17    ax = fig.add_subplot(111)
18
19    y_pos = np.arange(len(total_pdf[categorical_column]))
20    ax.set_yticks(y_pos)
21    ax.barh(y_pos, total_pdf["count"])
22    ax.set_yticklabels(total_pdf[categorical_column])
23    plt.xlabel("count")
24    plt.ylabel(categorical_column)
25
26    plt.tight_layout()
27    plt.show()
28
29    # return the aggregated dataframe to save it as a dataset
30    return total
31
32
33 #####
34 ## Global imports and functions included below ##
35 #####
36
37 # Functions defined here will be available to call in
38 # the code for any table.
39 from pyspark.sql import functions as F
40 import matplotlib.pyplot as plt
41 import numpy as np
```

Update "bar_chart_example"

Title*: bar_chart_example

Description: Add a helpful description about this template...

Parameters: Highlight variables in the code to create parameters below

titanic_dataset

(x) param1

+ Add new parameter

Status: Unreleased

☐ Save as default version

CancelUpdate template

By default, when a new template is created, the inputs are converted into dataset-type parameters. These parameters can be edited to give more descriptive names and descriptions. The template itself should also be given a title and detailed description.

To create new parameters, select the **Add new parameter** option on either the template editor or the code editor. You will be prompted to highlight a portion of text to be substituted by the input value of the parameter. If you wish to add multiple instances of the same parameter, you can do so by selecting the **Add** button next to the list of instances in the code on the template editor.

API Reference ↗

Send feedback

```

22     ax.set_yticklabels(total_pdf[categorical_column])
23     plt.xlabel("count")
24     plt.ylabel(categorical_column)
25
26     plt.tight_layout()
27     plt.show()
28
29     # return the aggregated dataframe to save it as a dataset
30     return total
31
32
33     #####
34     ## Global imports and functions included below ##
35     #####
36
37     # Functions defined here will be available to call in
38     # the code for any table.
39     from pyspark.sql import functions as F
40     import matplotlib.pyplot as plt
41     import numpy as np

```

 Highlight text to parameterize as param1

 GLOBAL CODE

Once a parameter has been added, you must select the parameter type: dataset, column, or variable. If **variable** is selected, you should also select a param type for that variable: text, number, select, multiselect, boolean, or list. Column inputs can optionally be configured to create a list of multiple columns, and datasets can be configured to use a specific input dataset by default. We recommend that you include descriptions for all parameters and default values for all variable-type parameters.

The global functions that are available in the Workbook where a template is first created will be automatically appended to the content of the template code so that it will be available in other Workbooks. Templates do not have access to global code in Workbooks where they are applied.

The version history of templates is saved, and new edits to a template are always saved as a new version of that template. Edits to a template do not automatically update instances of that template; each instance of the template will include a prompt to update to the latest version if they are using an outdated version of the template.

[API Reference ↗](#)

Best practices

[Send feedback](#)

- Include default values whenever possible.
- Create detailed descriptions for templates and for template parameters.
- Find an organizational scheme that makes it easy for users to find the most relevant templates for their workflows. This likely means creating a dedicated public folder with high-quality, vetted templates. Templates may be grouped in folders by workflow. A default template folder can be configured.

PREVIOUS
← Transforms / Transforms FAQ

NEXT
Getting started →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

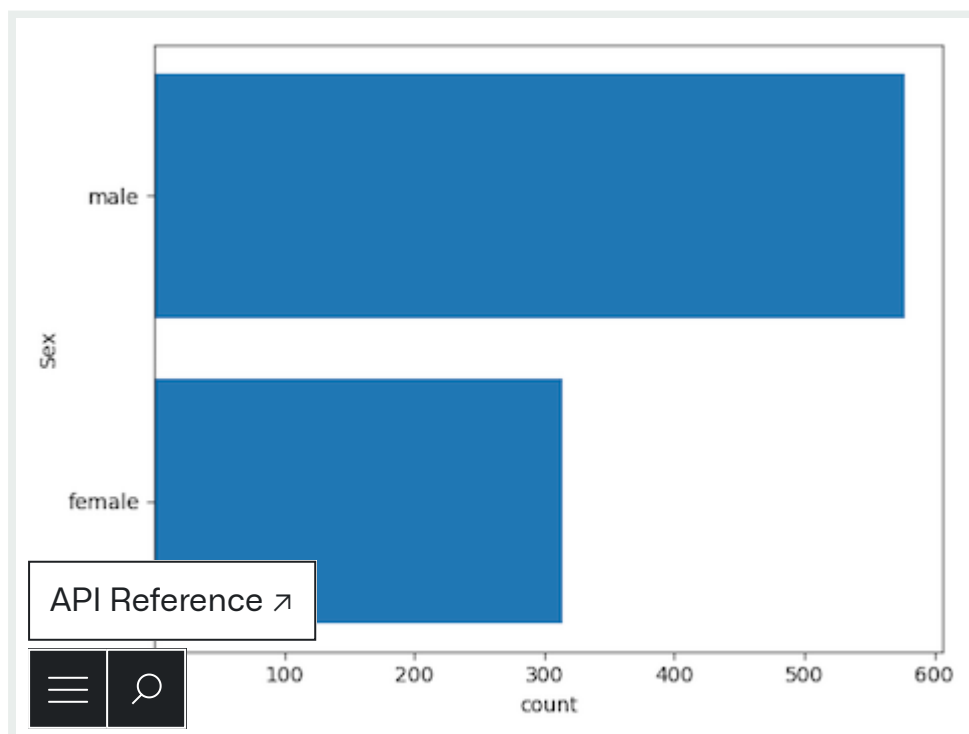
Getting started

In this tutorial, we will run through:

- [Creating a chart](#)
- [Creating a Template](#)
- [Using Templates](#)

Creating a chart

Since `titanic_dataset` contains rows (representing passengers) and columns (representing information about the passengers), we can visualize passenger count for a given passenger property (such as Sex or Pclass) using a bar chart. For example, here is a breakdown of passengers by sex:

[Send feedback](#)

In this example, we'll create a transform that creates the desired chart. Create a Python transform named `bar_chart_of_row_counts` and insert the following code:

```
1 def bar_chart_of_row_counts(titanic_dataset):
2     import matplotlib.pyplot as plt
3     from pyspark.sql import functions as F
4     import numpy as np
5
6     input_df = titanic_dataset
7     categorical_column = "Sex"
8
9     # calculate the counts
10    total = input_df \
11        .groupBy(categorical_column) \
12        .agg(F.count("*").alias("count")) \
13        .orderBy("count")
14
15    # convert summarized dataset to pandas
16    total_pdf = total.toPandas()
17
18    # plotting code
19    fig = plt.figure()
20    ax = fig.add_subplot(111)
21
22    y_pos = np.arange(len(total_pdf[categorical_column]))
23    ax.set_yticks(y_pos)
24    ax.barh(y_pos, total_pdf["count"])
25    ax.set_yticklabels(total_pdf[categorical_column])
26    plt.xlabel("count")
27    plt.ylabel(categorical_column)
28
29    plt.tight_layout()
30    plt.show()
31
32    # return the aggregated dataframe to save it as a dataset
33    return total
```

When you run this transform, the chart shown above will appear in your Workbook in the transform node in the graph. You can also jump into a full screen image view by hovering over the [API Reference](#) transform node and selecting **View Image**. This image viewer can also be reached from the contents sidebar and the Visualization tab.

[Send feedback](#)

To create the chart as an SVG, use the following code before creating your plot:

```
1 set_output_image_type('svg')
```

Or use the decorator for better visibility:

```
1 @output_image_type('svg')
2 def bar_chart_of_row_counts(titanic_dataset):
3     # ...
```

Creating a Template

Next, we'll convert this transform into a Template so it can be generalized and reused. Click on the **Actions** button in the top-right of your code editor and click **Create template**.

The screenshot displays the Databricks workspace interface. At the top, there are tabs for 'Paths' and 'Graph'. Below these, a 'New Transform' button is visible. The main area shows a transform named 'bar_chart_of_row_counts' with a 'New Dataset' button. A table of Titanic data is shown on the left, and a bar chart visualization is on the right. The code editor at the bottom contains the following code:

```
1 def bar_chart_of_row_counts(titanic_dataset):
2     import matplotlib.pyplot as plt
3     from pyspark.sql import functions as F
4     import numpy as np
5
6     input_df = titanic_dataset
7     categorical_column = "Sex"
8
9     # calculate the counts
10    total = input_df \
11        .groupBy(categorical_column) \
12        .agg(F.count("*").alias("count")) \
13        .orderBy("count")
```

An 'Actions' menu is open in the top right corner, showing options: 'Add multiple input datasets', 'Create template', 'Sever parent permissions', and 'Cache'.

Y API Reference ight into a template creation view in the full screen editor. In the Template editor, you can edit the Template's name, description, and parameters. Let's name this Template Bar Chart of Row Counts by Categorical Variable

Send feedback

description as follows: Create a transform with a bar chart of the row counts of 1 categorical column in any input dataset.

The screenshot shows the Databricks Transformation Logic editor. On the left, a Python function `bar_chart_of_row_counts` is defined, which takes `titanic_dataset` as input, calculates row counts for the 'Sex' column, and generates a bar chart. On the right, the 'Create template' dialog is open. It has a title 'Bar Chart of Row Counts by Categorical Variable' and a description 'Create a transform with a bar chart of the row counts of 1 categorical column in any input dataset'. The parameters section shows `titanic_dataset` as a dataset parameter. The status is 'Released' and there is a 'Commit Message' field.

Any input datasets — in this case, just `titanic_dataset` — are automatically added as parameters of type `dataset` for the Template. Click on `titanic_dataset` in the Template editor to change it. Since we want this template to be generic, let's change the parameter name from `titanic_dataset` to `input_dataset` and add a description.

This screenshot shows the 'Create template' dialog with the parameter `input_dataset` selected. The title is 'Bar Chart of Row Counts by Categorical Variable' and the description is 'Create a transform with a bar chart of the row counts of 1 categorical column in any input dataset'. The parameters section shows `input_dataset` as a dataset parameter. The 'Type' is set to 'Dataset'. The 'Always use "titanic_dataset" for this parameter?' toggle is turned off. The description field contains 'Input dataset.'.

API Reference ↗

Send feedback

Two instances of `{{input_dataset}}` will be highlighted in the transform code. Next, let's parameterize the input column. To assign a variable in the code body as an input parameter for the template, click **Add new parameter** in the top-right of the transform and highlight the appropriate variables in the code. Highlight the string `"Sex"` as shown below:

The screenshot displays the Databricks Transformation Logic editor. On the left, a code editor shows a PySpark function `bar_chart_of_row_counts` that takes `input_dataset` and `selected_column` as parameters. The code calculates row counts for a categorical column and generates a bar chart. On the right, the 'Create template' panel is open, showing the configuration for the `selected_column` parameter. The parameter is of type 'Column', sourced from 'input_dataset', and has a description: 'This column will be used to create the bar chart.' The status is set to 'Released'.

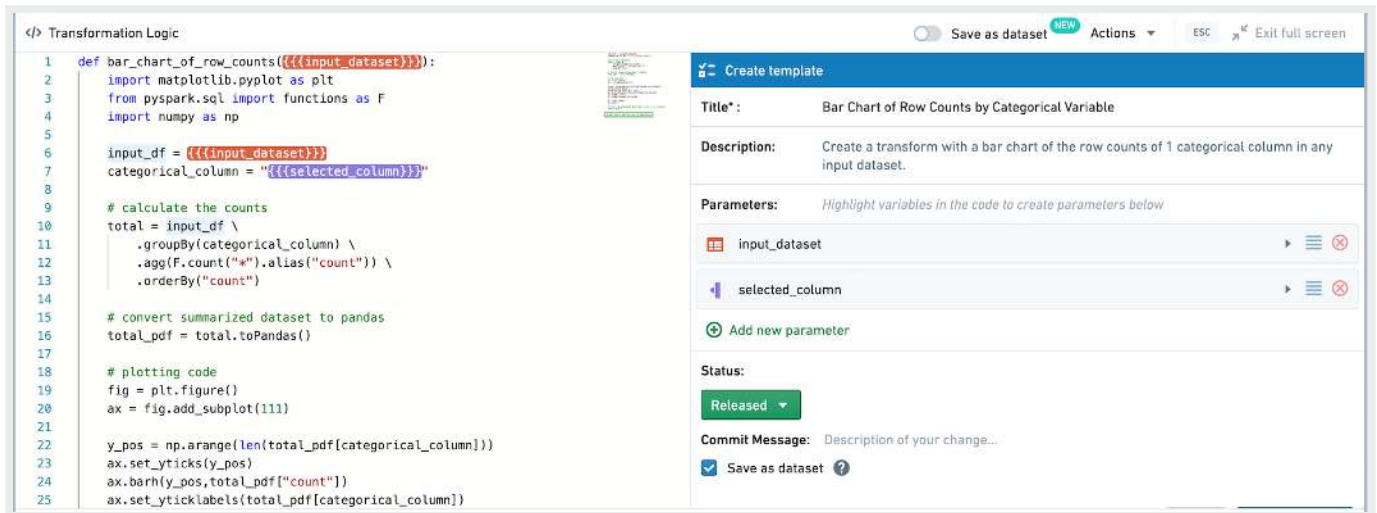
This adds this part of the code as a parameter of type `column` with the Source Dataset as `input_dataset`. In the Template editor, edit the `param1` parameter name and rename it to `selected_column`.

In this example code, we defined the column name as a variable at the top of the transform so that we only need to parameterize it once. When you're templating other transforms, you can use the **Add** button to add more instances of the same parameter.

Next, choose whether this template should be saved as a dataset by default. By checking the **Save as dataset** box, when added the template will be added as a persisted transform by default. If **Save as dataset** is left unchecked, the template will be applied as an unpersisted transform by default. In this case, let's choose to Save as Dataset by default as we want to use the output in other applications.

[API Reference ↗](#)

[Send feedback](#)

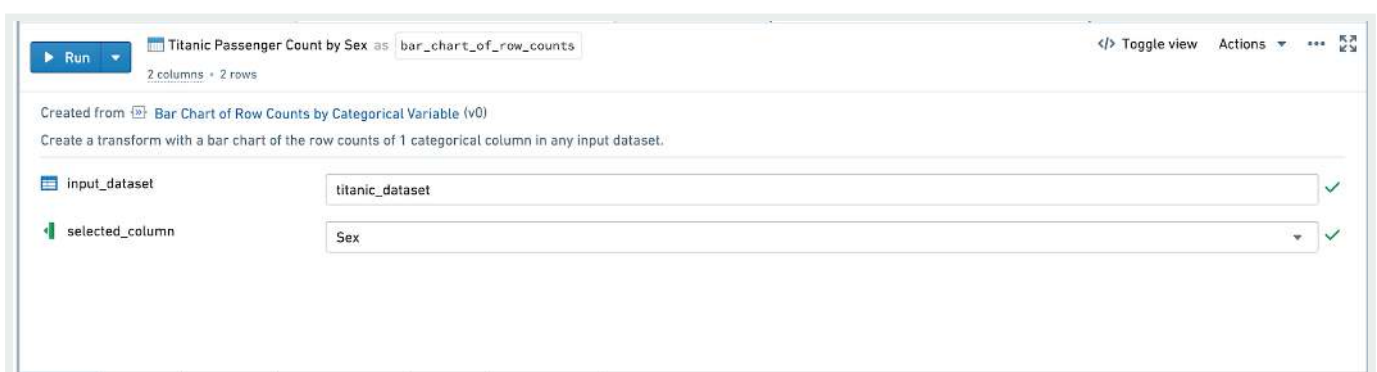


Finally, click the **Create template** button to create and save the Template. Whenever you create a new Template, you will have to choose a folder to save it in. For this example, you can save the Template in your home folder.

Templates can only be discovered and used by users who have access, so you can save a Template in your home folder while you are still working on it, and move it to a shared folder once you want to promote it for broader use. Templates can also be added to the [Data Catalog](#).

Using Templates

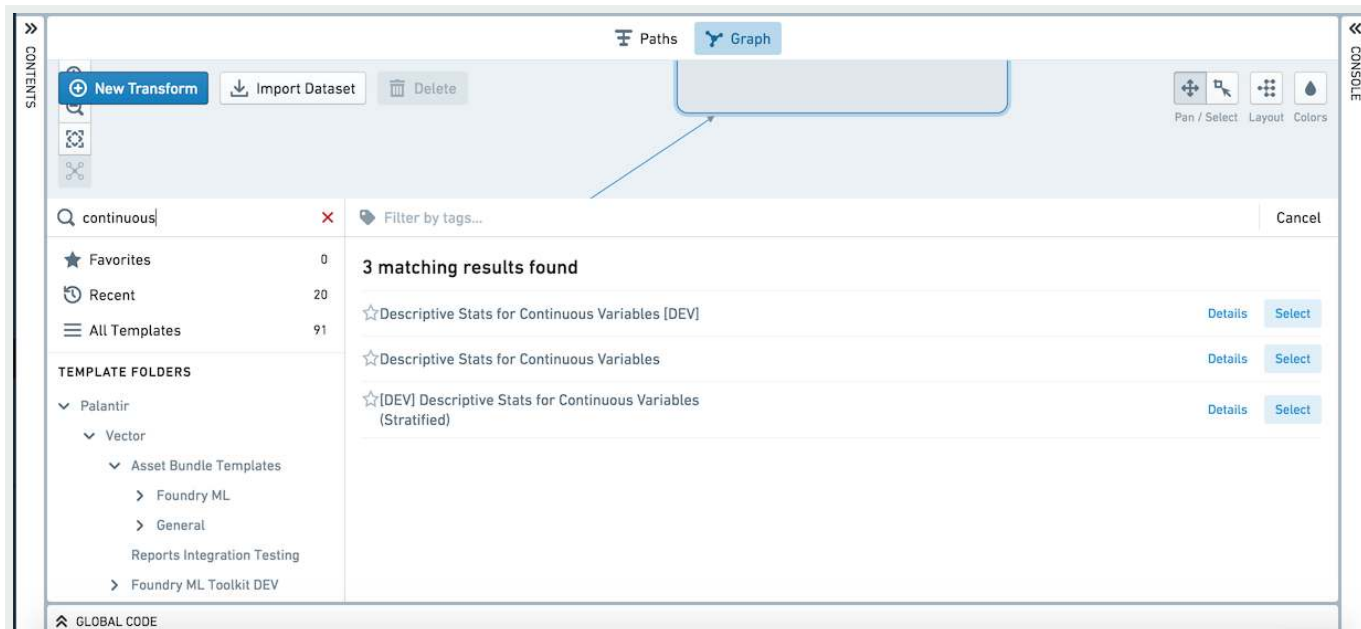
After creating and saving a template, you can use the Template in a point-and-click manner.



To view available templates, click **Browse all templates** in the transform creation view.

[API Reference ↗](#)

[Send feedback](#)



Browse Templates supports searching for templates by name, description, and tags, or browsing based on Favorites, Recently used templates, or the Files structure. To apply a template, click its name and choose "Select". Let's add the template we just created.

After adding the template to the graph, you can rename the transform and fill out the inputs. Assign the `input_dataset` parameter by clicking **Click here to add dataset** and clicking the `titanic_dataset` in the graph. Now you can select any column as the `selected_column` parameter to create charts based on that column.

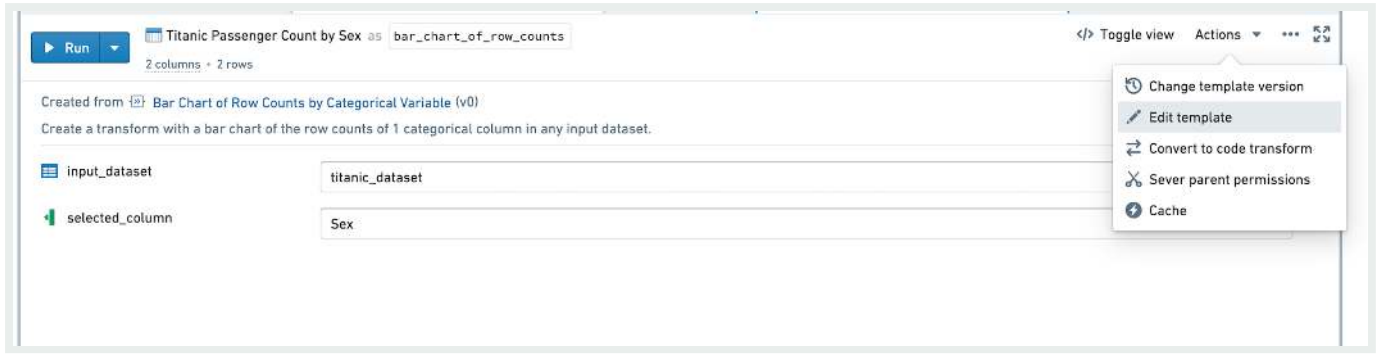
Select **Run** to compute the transform. For transforms that output a visualization, graph view is the default in the Graph. You can right-click, choose **Edit** and then **Show table view** to view the node as a table.

Editing Templates

If you would like to update the code backing a template, click on **Actions** and then **Edit Template** to enter the code editor and edit the template.

API Reference ↗

Send feedback



PREVIOUS
← Overview

NEXT
Multi-node templates →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

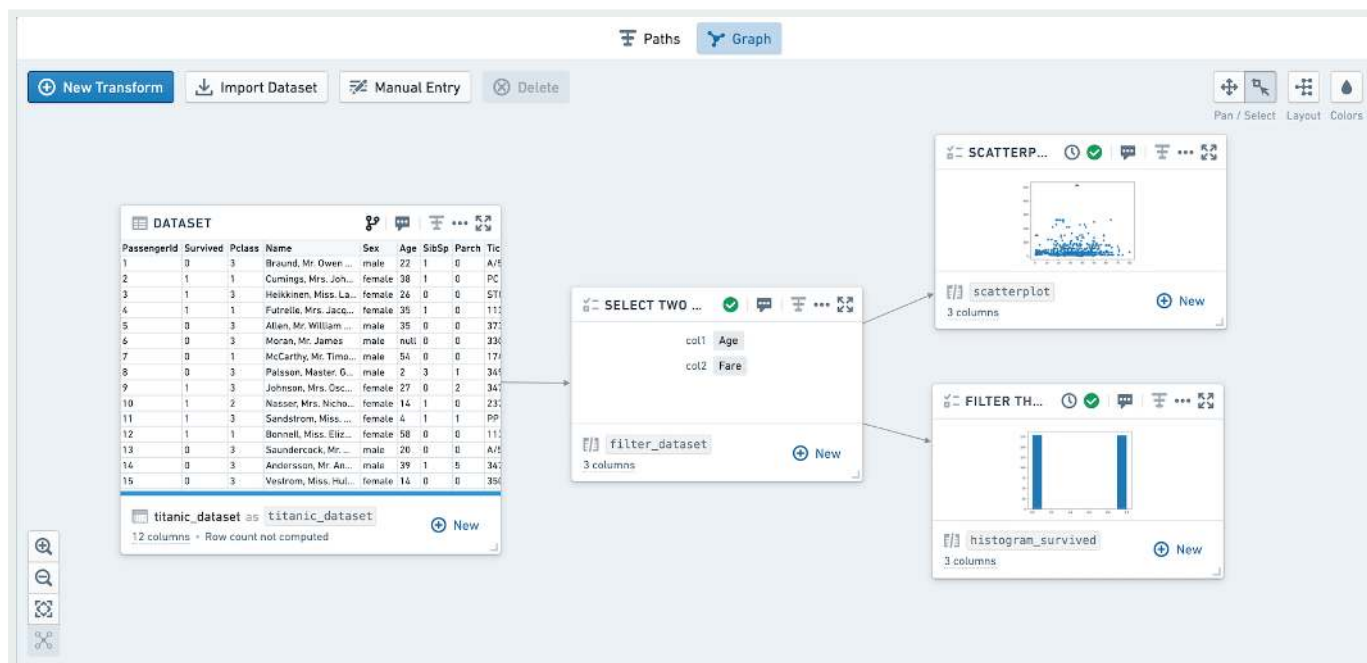
[Send feedback](#)

Multi-node templates

Code Workbook supports multi-node templates for templated workflows. A template can be created from multiple other templates, and you can bind the values of parameters in these templates together.

Let's walk through an example using `titanic_dataset`. We will templatize a workflow selecting two numeric columns and plotting them in two graphs. The first graph is a scatterplot of the two numeric columns. The second plot is a histogram of whether or not the passengers survived, based on filtering on the numeric columns.

Here's a visual overview of the workflow we are templating:



First, create a template selecting two numeric columns from the input dataset, along with `Survived`. Title this template `Filter`.

The screenshot shows the Databricks workspace interface. On the left, a code editor displays the following Python code:

```
1 def filter_dataset({{titanic_dataset}}):
2     return {{titanic_dataset}}.select('{{col1}}', '{{col2}}', 'Survived')
3
4 #####
5 ## Global imports and functions included below ##
6 #####
7
8
```

On the right, the 'Update "Select two numeric columns"' configuration panel is open. It includes the following fields:

- Title*:** Select two numeric columns
- Description:** Add a helpful description about this template...
- Parameters:** Highlight variables in the code to create parameters below
- Parameter list:** A table with one entry: `titanic_dataset` (2 instances in code, Type: Dataset, Input type: Spark dataframe, Always use "" for this parameter?: ☐, Input tags: Select..., Description:).

Second, create a template plotting a scatterplot of the two numeric columns. Title this template `Scatterplot`. Note that the input dataset is set to be read in as a Pandas dataframe.

The screenshot shows the Databricks workspace interface. On the left, a code editor displays the following Python code:

```
</> Transformation Logic
1 def scatterplot({{filter_dataset}}):
2     filter = {{filter_dataset}}
3     import matplotlib.pyplot as plt
4     plt.scatter(filter['{{col1}}'], filter['{{col2}}'])
5     plt.show()
6
7     return filter
8
9 #####
10 ## Global imports and functions included below ##
11 #####
12
13
```

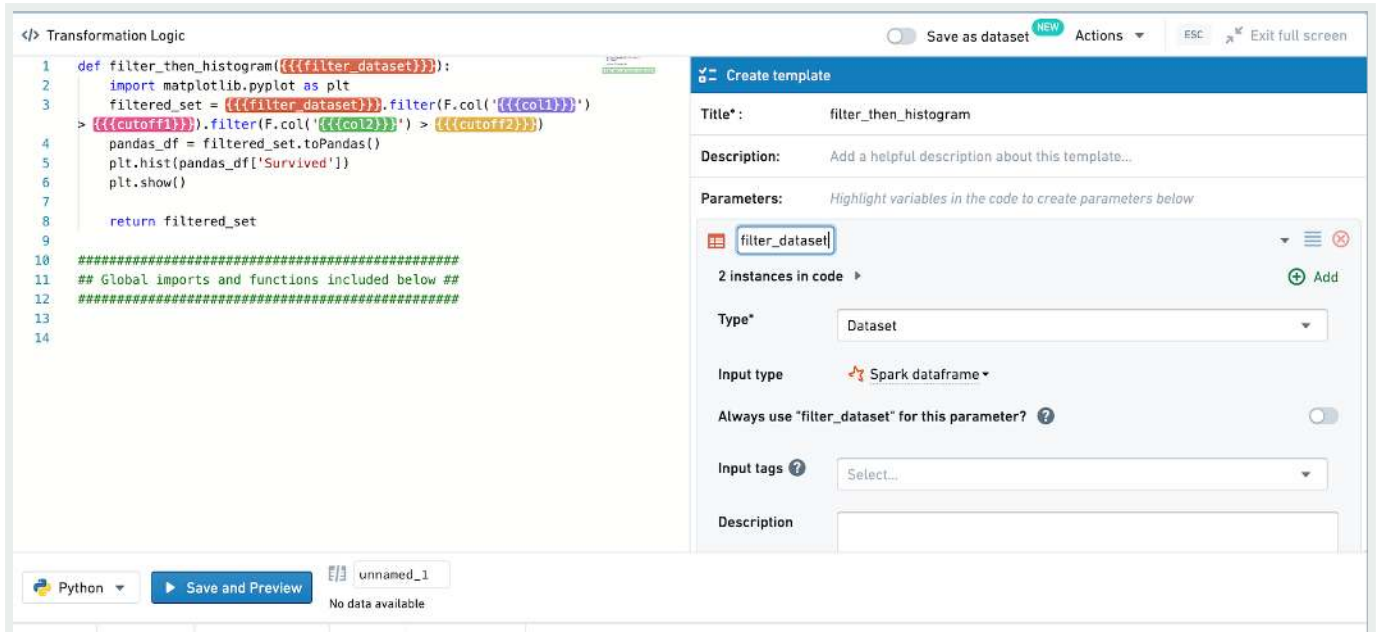
On the right, the 'Create template' configuration panel is open. It includes the following fields:

- Title*:** scatterplot
- Description:** Add a helpful description about this template...
- Parameters:** Highlight variables in the code to create parameters below
- Parameter list:** A table with one entry: `filter_dataset` (2 instances in code, Type: Dataset, Input type: Pandas dataframe, Always use "filter_dataset" for this parameter?: ☐, Input tags: Select..., Description:).
- Group:** A dropdown menu.
- Buttons:** OK, Cancel.

API Reference ↗

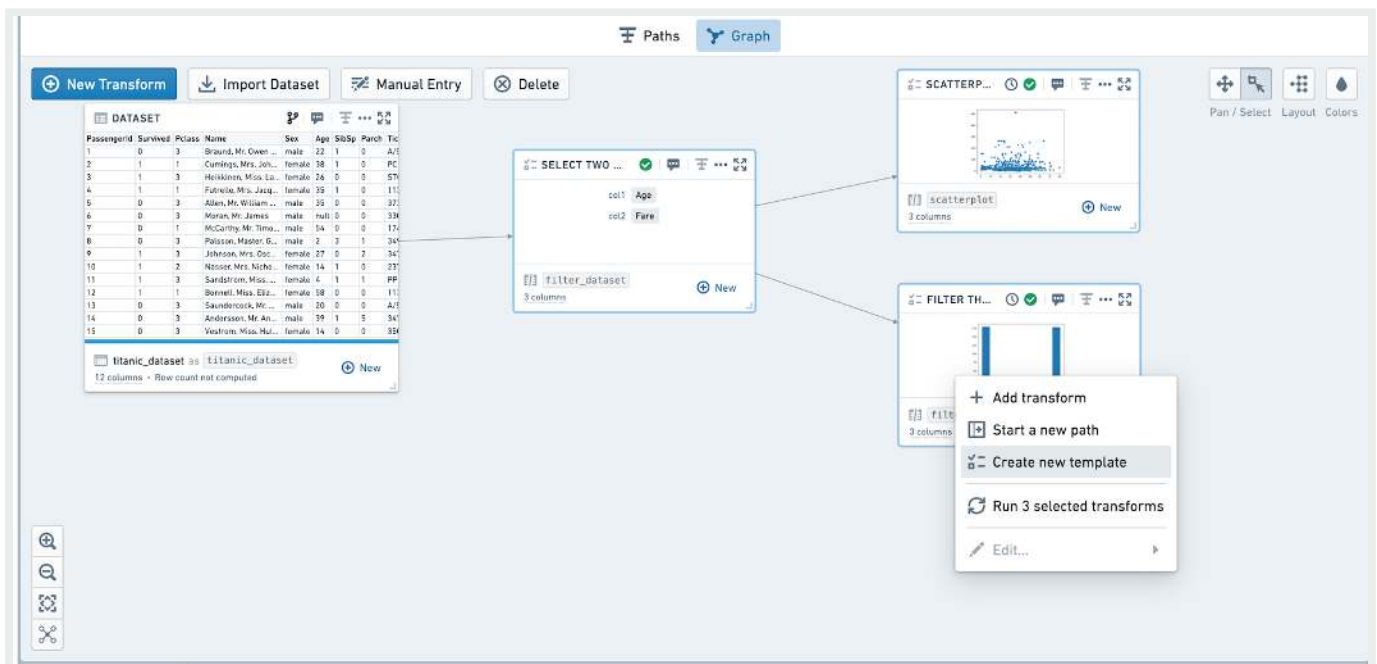
Finally, create a template that filters the input dataset based on the two numeric columns and two templated inputs. Note that the input dataset is set to be read in as a Pandas dataframe.

Send feedback



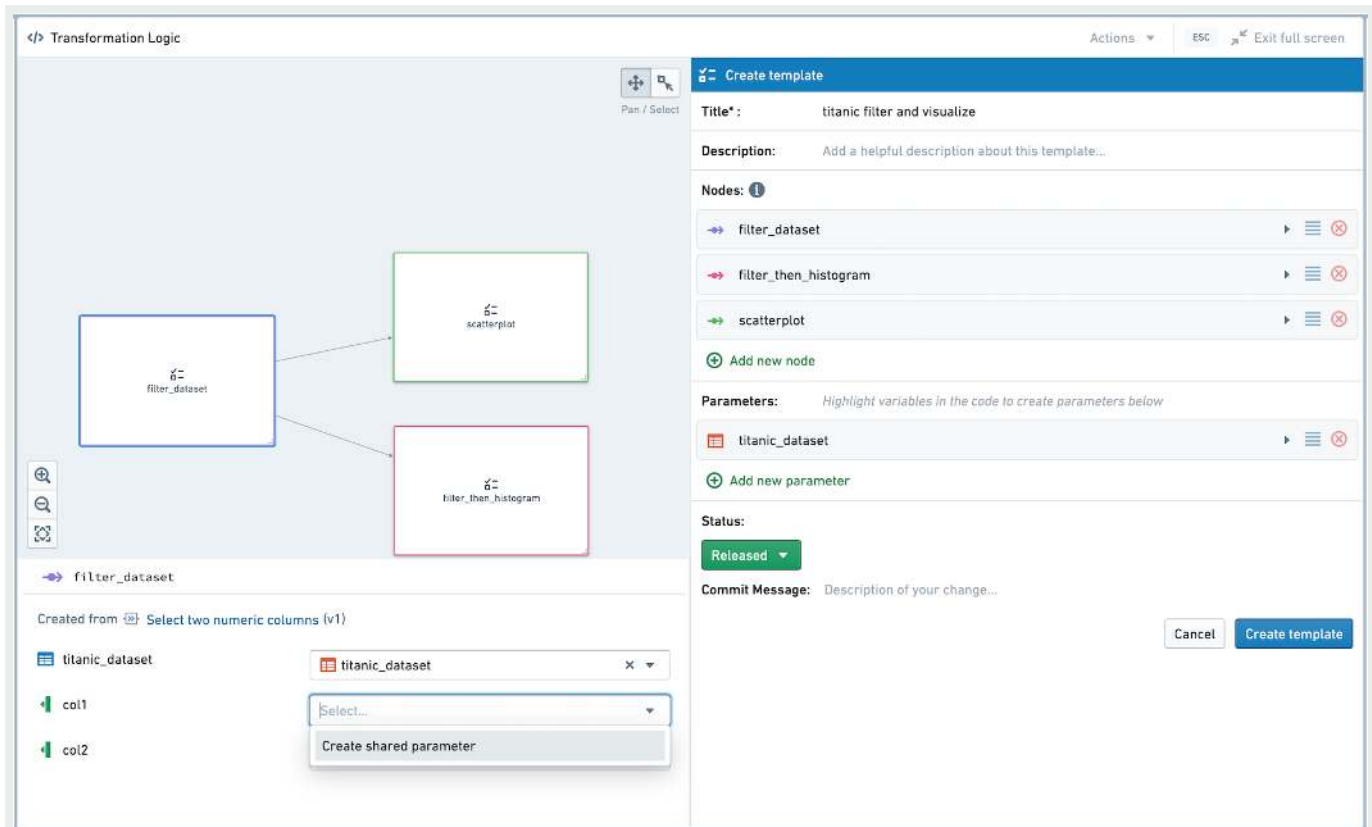
Creating Multi-Node Templates

Select all three templates and right-click to open the menu, then select **Create new template**. You should now see the template editor.



We want to link the values of the `col1` parameters, and link the values of the `col2` parameters. First, click into the `Filter` template. Click into `col1`, and select **Create shared API Reference** down.

Send feedback



On the right-hand side, a new parameter titled `col1` has been created. Select `titanic_dataset` as the source dataset in the right-hand pane. Then, click into the two other templates and choose to link `col1` to the new `col1` Multi-Node Template parameter. Repeat for `col2`, and then save the multi-node template.

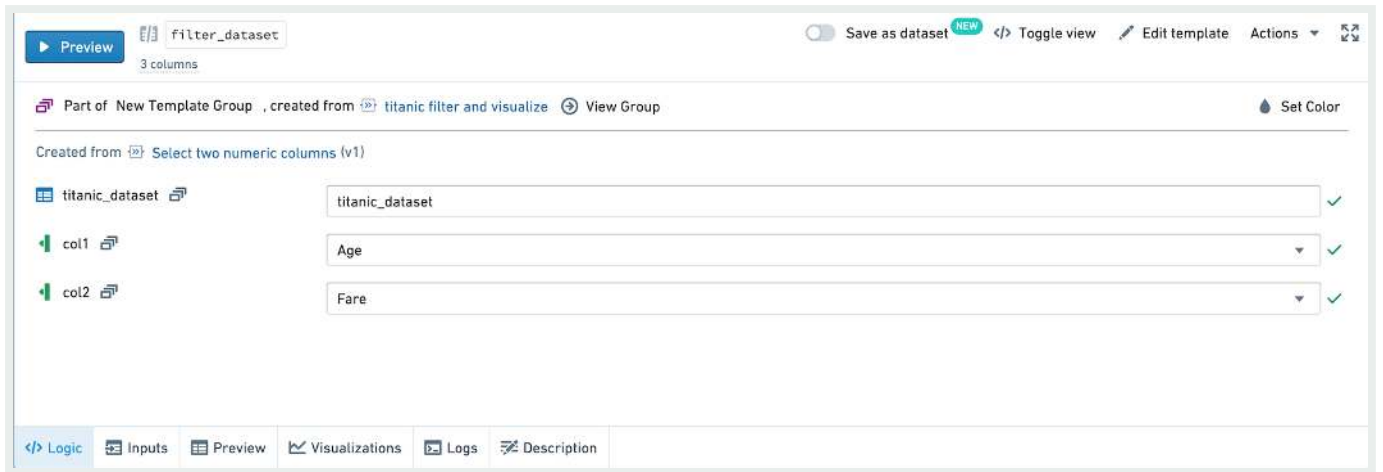
Using Multi-Node Templates

The three templates we previously created are now part of a multi-node template.

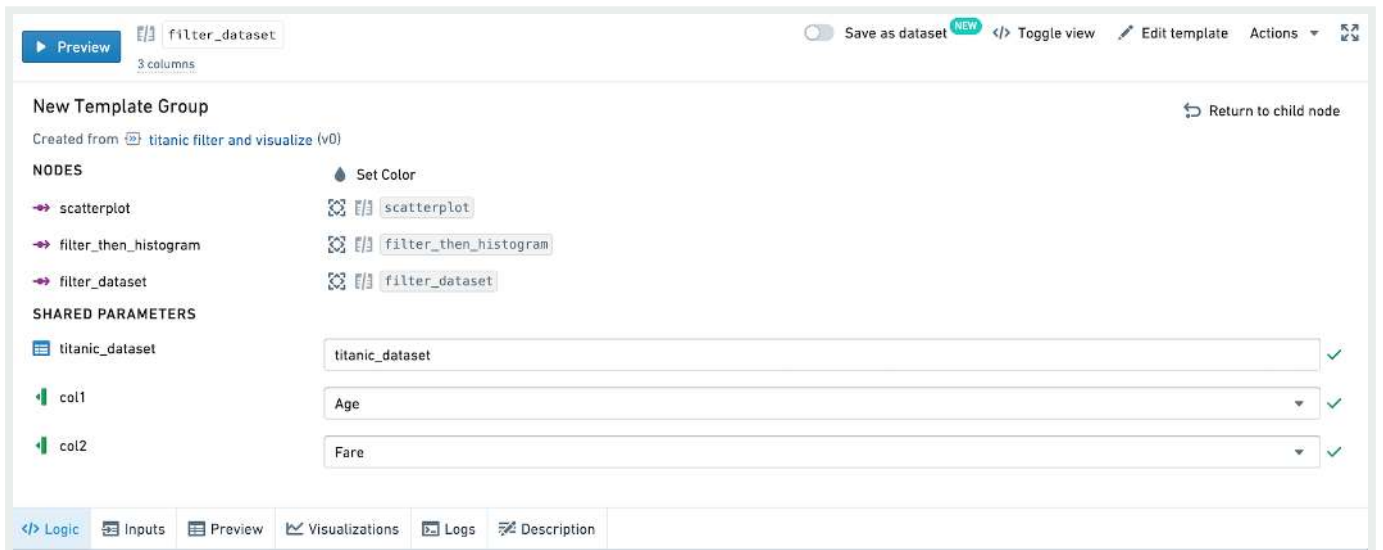
Click into the `Filter` template. Next to the `col1` and `col2` parameters, there is an icon indicating this parameter value is controlled by the multi-node template parameter.

[API Reference ↗](#)

[Send feedback](#)



Select **View Group** at the top of the pane. You now see a view highlighting the nodes in the Multi-Node Template, and listing the shared parameters in the template. You can change the value of **col1** and **col2** in this view, and all instances in the three nodes will also be changed.

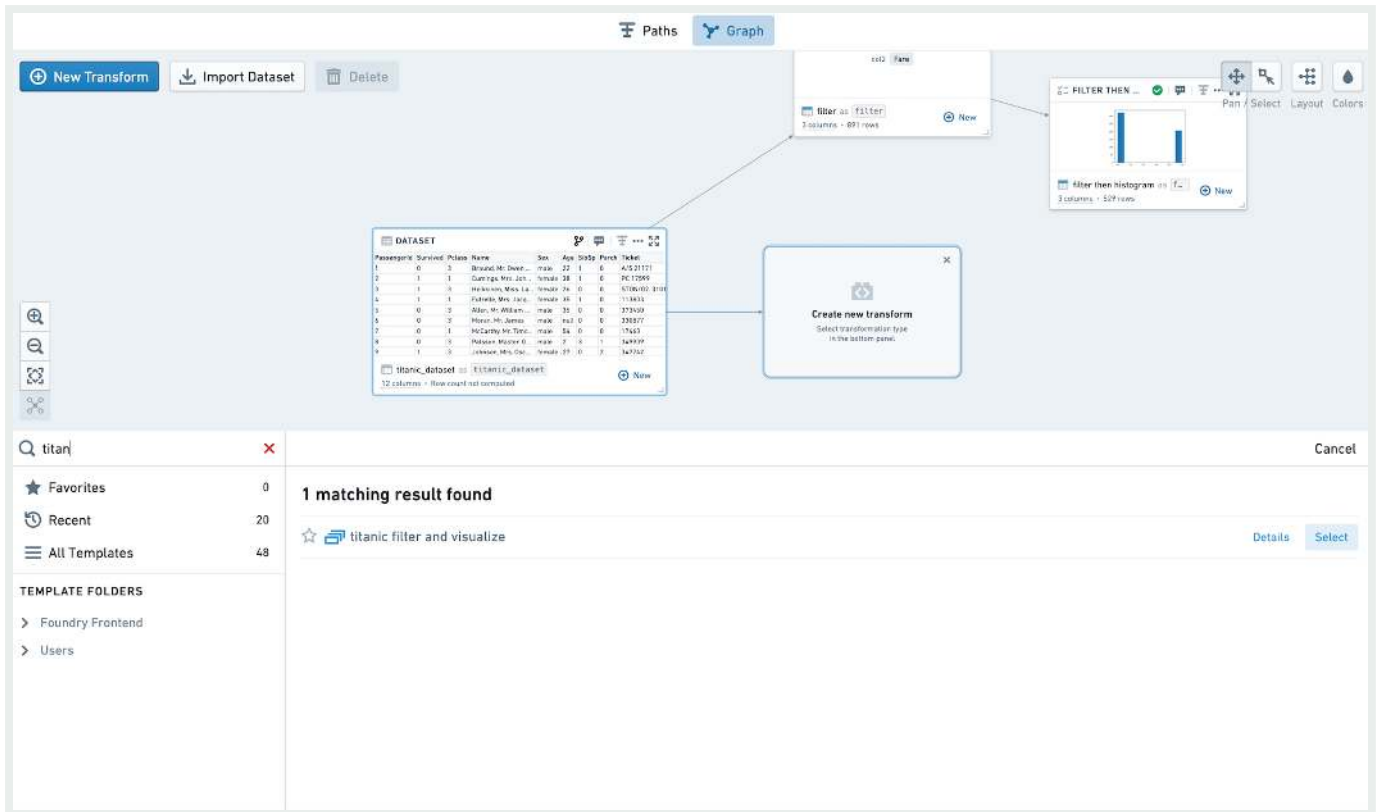


If you change the value of a shared parameter in the child node pane, the value will also be changed for all instances in the multi-node template.

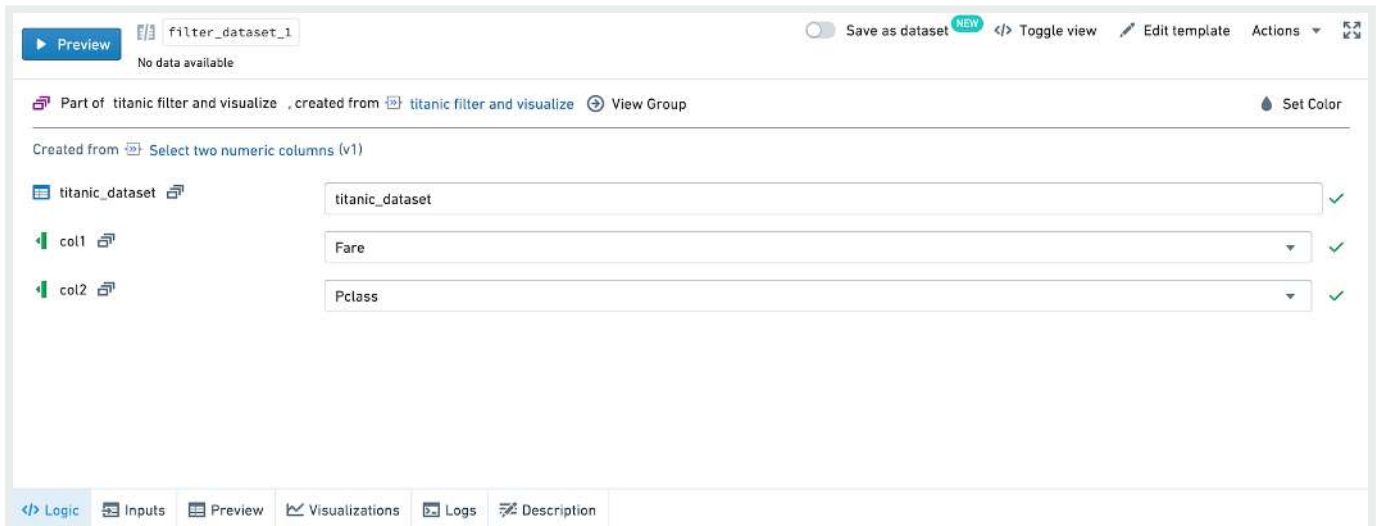
Let's add a new instance of this template.

[API Reference ↗](#)

[Send feedback](#)



Let's analyze `Fare` and `Pclass`. By selecting these two columns in the view for the child node, notice that you're setting the column values across the group.



Then, update the cutoff values in `Filter then histogram`. Run the templates, and we've now created the same graphs for a different set of numeric columns.

API Reference ↗

← Getting started

NEXT
Suggested templates →
Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

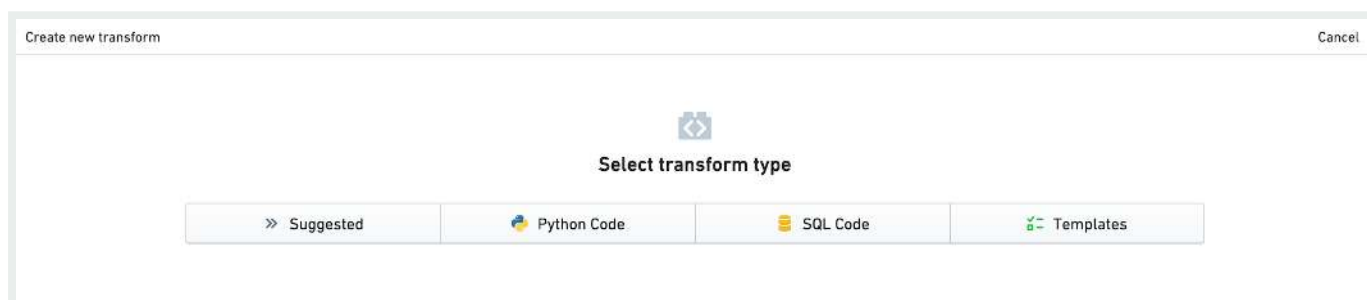
— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Suggested templates

For groups of similar datasets, you may want to suggest templates to apply. For example, for a group of datasets containing monthly stock returns, you may want to suggest a template that calculates the annualized return. Suggested templates are shown in a separate button within the New Transform panel, making them more discoverable.



Configuring input tags in a template

Below, we see a template that calculates annualized returns from monthly returns. For any dataset with the tag **Finance**, this template will appear in the **Suggested** tab when the user creates a new transform with the dataset as a parent.

Configure the desired input tags in the **Input Tags** field when creating or editing a template. You can add multiple input tags.

Create template

Title* :

calculate annualized returns

Description:

Calculates Annualized returns from monthly returns

Parameters:

Highlight variables in the code to create parameters below

monthly_returns_SP

▼

☰

✕

1 instance in code ▶

⊕ Add

Type*

Dataset ▼

Input type

 Spark dataframe ▼

Always use "monthly_returns_SP" for this parameter?

?

☐

Input tags ?

Finance:MonthlyReturns ✕ | ▼

Description

Group

✕ ▼

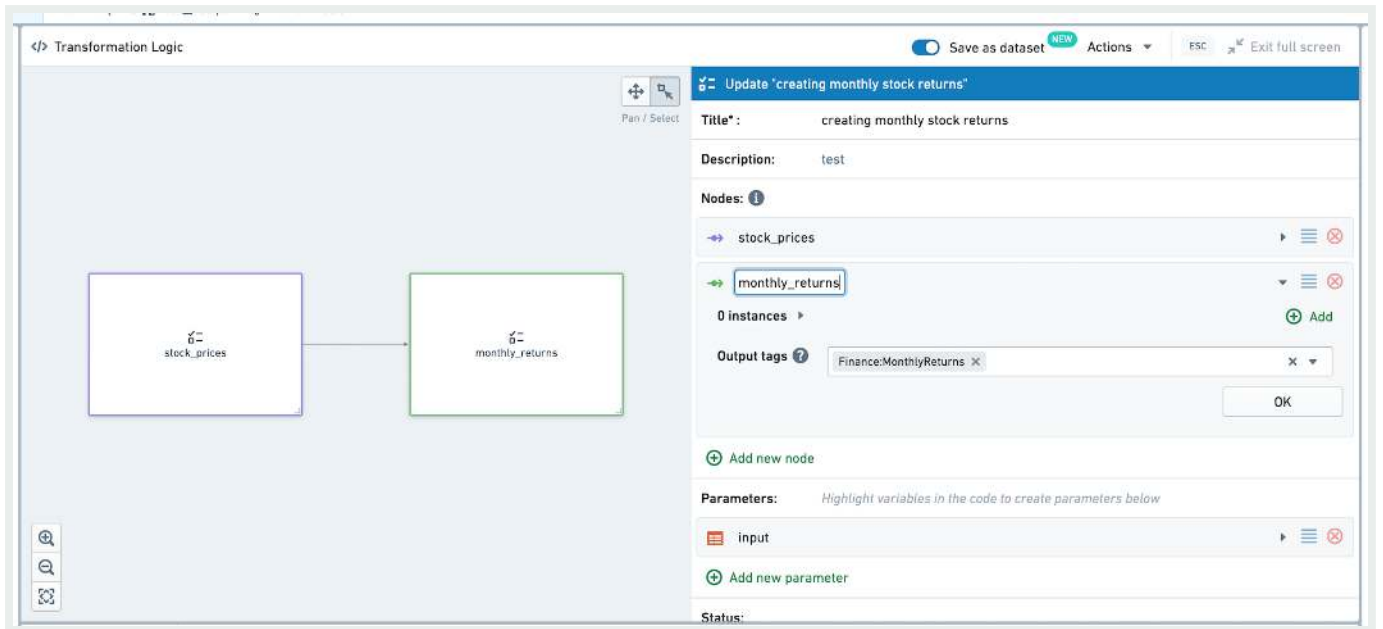
OK

Adding tags to a dataset

One way to add input tags to a dataset is to open the dataset in Dataset-app. Note that because tags are tied to datasets, tags cannot be added to unpersisted transforms in Code Workbook. This means that Suggested Templates are not supported for unpersisted transforms.

[API Reference ↗](#)

[Send feedback](#)



PREVIOUS
← Multi-node templates

Environment / Overview →
NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Environments

Each Code Workbook is associated with an environment. An environment includes a set of Conda packages and Spark settings installed on the Spark module backing computation in the Workbook. On opening a Workbook, you may see **Waiting for resources** or **Initializing Environment** as the Workbook obtains a Spark module. You will not be able to use the console or run transforms until a Spark module is acquired.

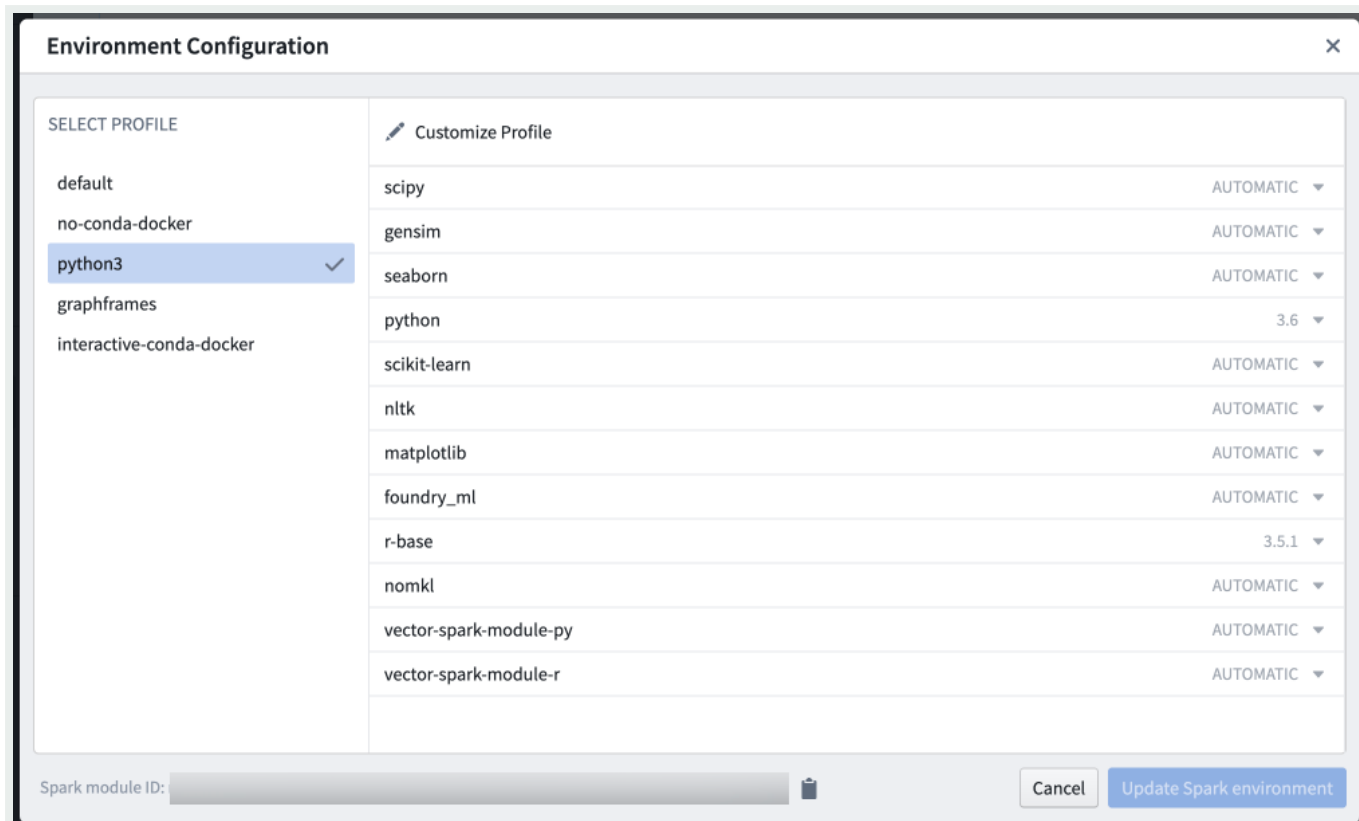
Each user is assigned one Spark module that is used across workbooks in the same project with the same environment. Spark modules are never shared between users.

Select a profile

You can configure your environment by clicking on **Environment** and then **Customize Spark environment**. You will see a list of [predefined profiles](#). These have been configured by your administrator as useful sets of defaults for particular workflows or user groups.

To use a predefined profile, select it in the left-hand panel, then click **Update Spark environment**.

[API Reference ↗](#)[Send feedback](#)



Modify a profile

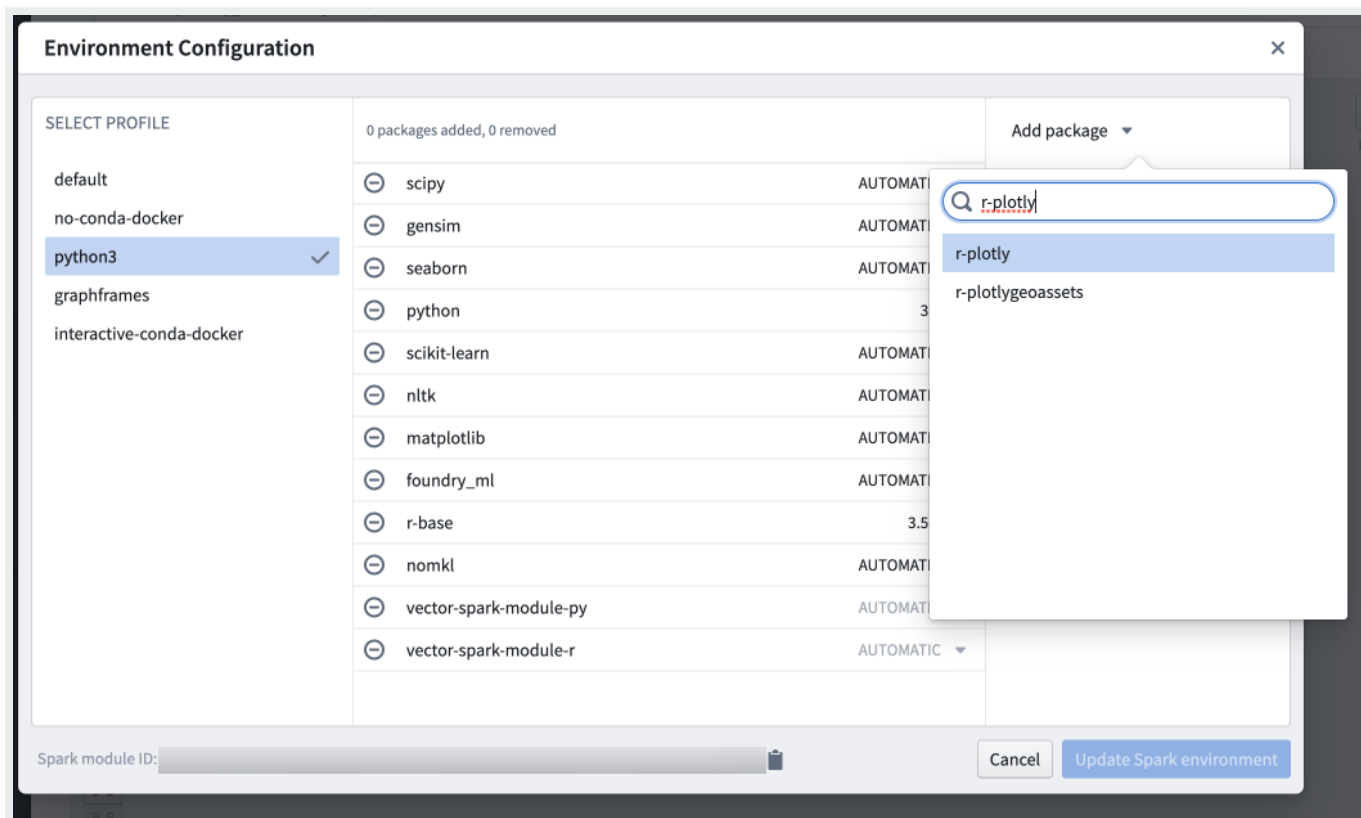
If none of the predefined profiles suit your use case, you can add or remove packages to modify a profile by editing the predefined profile in Control Panel or customizing the predefined profile in a workbook. The main difference between these options is the scope of your changes. If you edit a predefined profile in Control Panel, packages installed for all workbooks with that profile will change accordingly. Conversely, if you customize a predefined profile in a workbook, only the packages installed for that particular workbook will change.

To edit a predefined profile in Control Panel, you can [configure the packages for a Code Workbook profile](#).

To customize a predefined profile in a workbook, open the desired workbook, click on **Environment**, select **Configure environment**, and click on the **Customize profile** button. In the customization view, you can remove existing packages by clicking the minus sign, or change their requested versions. To add a new package, search for it in the **Packages** sidebar, click the plus sign. Once it's added to the profile, you can choose a specific version, choose **AUTOMATIC**, or specify a custom Conda version.

Send feedback

 Customized environments will be slower to initialize than predefined profiles.



Add a profile

To create a new predefined profile for use across workbooks, you can [configure Code Workbook profiles](#) in Control Panel.

Custom Conda Versions

A Custom Conda Version can be:

- A version (such as `3.6`)
- A comparison operation and a version (such as `>=3.6`) (Accepted comparison operators are `=`, `==`, `>=`, `>`, `<=` and `<`) For more details, see the [Conda package](#)

[API Reference ↗](#)

[Send feedback](#)

[PREVIOUS](#)



[NEXT](#)



© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Code Workbook profiles

A Code Workbook profile is a predefined set of Conda packages and Spark settings that serves as a useful default environment for a use case or group of users. For a given Code Workbook profile, you can also configure pre-warmed modules to reduce user wait times.

For example, a data science group may want to configure a data science profile with Conda packages like `tensorflow` and `keras`, along with a higher amount of driver memory.

In Code Workbook, users can choose from a list of Code Workbook profiles in the Environment Configuration dialog. Users can then further customize the Conda packages. Users are unable to customize the Spark settings for a given Code Workbook profile.

For a given profile, the permissions boundary is the project. Users with Viewer access to the profile can import the profile into a project. Once the profile has been imported into a project, anyone using Code Workbook in the project can use the profile.

Artifacts profiles

[New Code Workbook profiles created in Control Panel](#) use Artifacts. Artifacts-backed profiles allow use of libraries securely produced by Artifacts, including Python libraries authored in Foundry that are not published to the `shared` channel. Refer to the [Control Panel documentation](#) to learn more about creating Artifacts profiles.

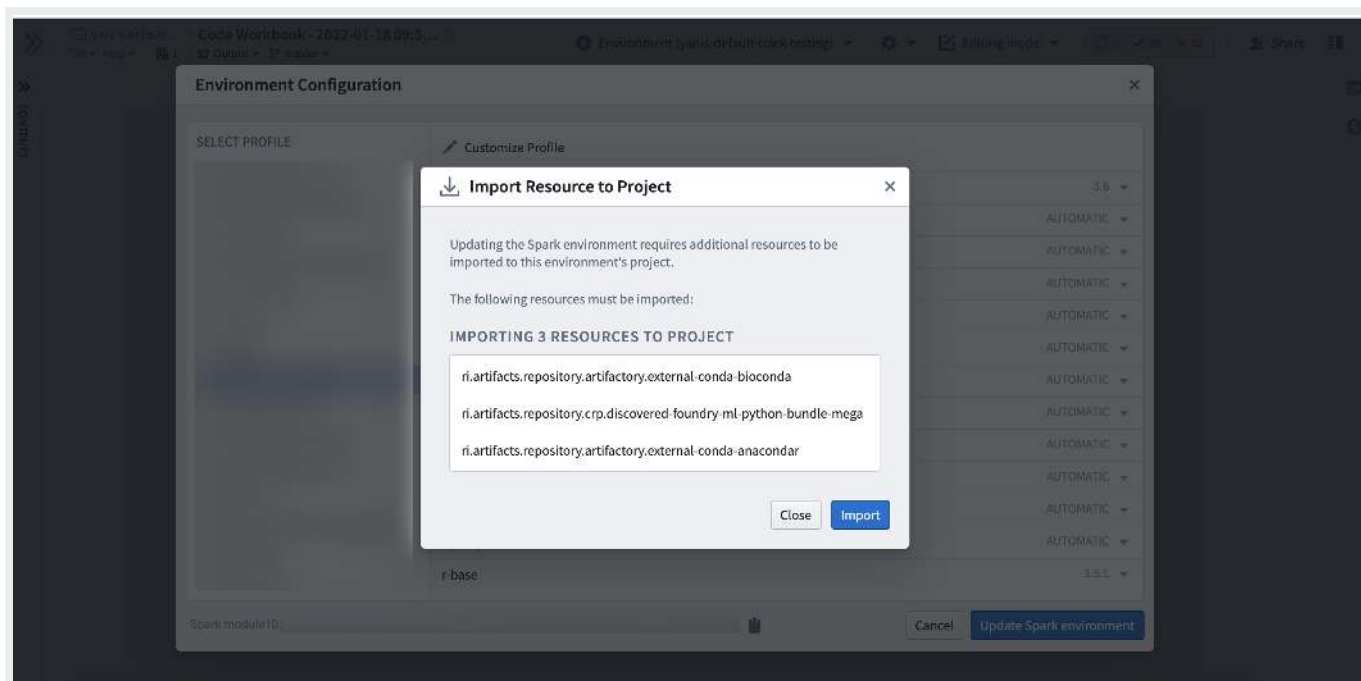
Using an Artifacts profile

[API Reference ↗](#)

After an Artifacts profile is created in Control Panel, users can use that profile in Code

W   An Artifacts profile contains a list of requested packages [Send feedback](#) repositories that provide those packages. To use the profile, all backing repositories on the

profile must be added as a project import in the workbook's project. If you do not have permissions to import all of the backing repositories on the profile, you will not be able to set the environment with that profile.



If at any point, a backing repository of the profile is no longer imported into the project, you will no longer be able to obtain an environment until it is added as an import, which you will be prompted to do in the UI.

Customizing an Artifacts profile

When using a customized Artifacts profile in Code Workbook, the list of backing repositories used is that of the Code Workbook rather than that of the profile. Newly created Code Workbooks are initialized with no backing repositories, and the list of backing repositories is automatically populated as users use customized Artifacts environments in the Workbook. Note this means that all users of customized Artifacts environments, across all branches in the Workbook, are adding to and using the same list of backing repositories.

When using a customized Artifacts environment in Code Workbook, all backing repositories of the Workbook must be imported into the project. If at any point, a backing repository of the workbook is no longer imported into the project, you will no longer be able to obtain an environment until it is added as an import, which you will be prompted to do in the UI.

[API Reference ↗](#)

[Send feedback](#)

Future plans

In the future, all existing profiles and environments will be migrated to use Artifacts. This means that regardless of base profile, users will be able to use libraries securely produced from Artifacts.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

View resolved environment

After obtaining a Spark environment, you can view the exact packages installed in the Spark environment in the Resolved Dependencies dialog. To open the dialog, select **Environment** > **View resolved packages**. The dialog will show a list of the direct and transitive dependencies.

218 TOTAL DEPENDENCIES

12 DIRECT DEPENDENCIES

	Version	Build string
nomkl	1.0	h5ca1d4c_0
python	3.6.7	h381d211_1004
scipy	1.5.3	py36h976291a_0
scikit-learn	0.23.2	py36hb6e6923_3
gensim	4.1.2	py36hc4f0c31_0
r-base	3.5.1	hea2b9be_1007
matplotlib	2.2.4	py36_0
nltk	3.6.7	pyhd8ed1ab_0
seaborn	0.11.2	hd8ed1ab_0
foundry_ml	2.46.2	py_0
vector-spark-module-py	5.168.0	py_0
	5.168.0	r35_0

API Reference ↗

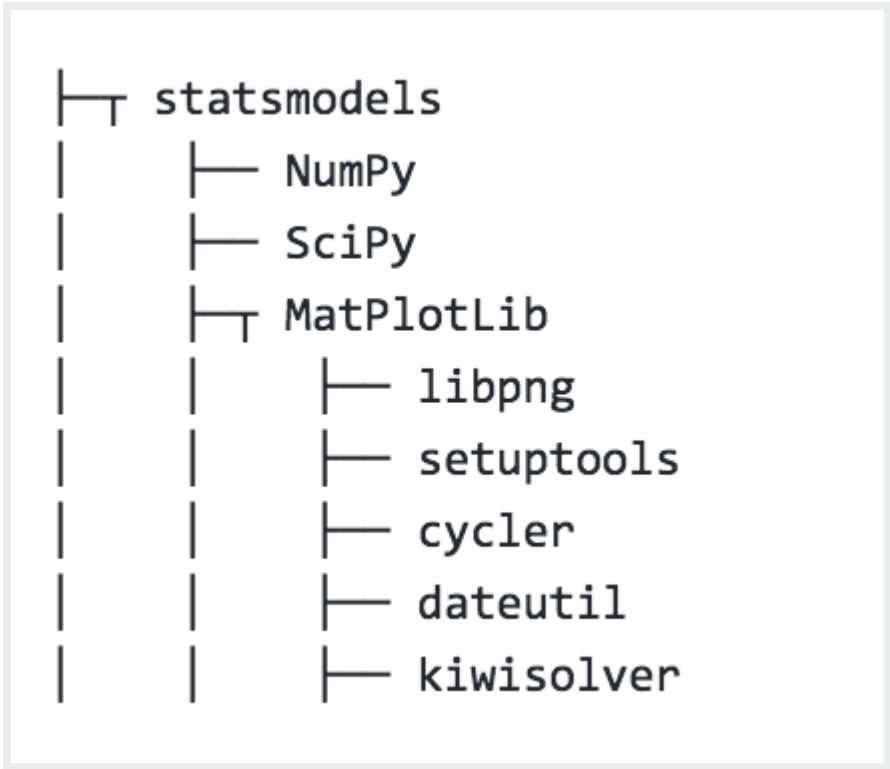
TRANSITIVE DEPENDENCIES

	Version	Build string
--	---------	--------------

Send feedback

A direct dependency is a package explicitly specified by the user to include in the Spark environment. You can specify direct dependencies in the **Customize Spark Environment** menu.

A transitive dependency is a package relied upon by a direct dependency. For example, depending on `statsmodels` transitively imports `NumPy`, `SciPy`, `Matplotlib`, and their dependencies as well.



Analytics / Code Workbook [Legacy] / Environment / Batch builds and interactive builds

Batch builds and interactive builds

When using Code Workbook interactively by running jobs from within the workbook interface, all jobs will use the Spark module associated with the workbook. As seen in the image below, you can view the Spark module ID [in the session history](#) dialog for any session.

The screenshot shows the 'SESSION HISTORY' dialog. On the left, a list of sessions is shown with their IDs and timestamps. The session ID `8fe59113-2aa3-4e17-88db-ae3bfa4ae075` is highlighted. On the right, the 'Compute information' tab is active, displaying a JSON configuration for the selected session. Two red boxes highlight the Spark module ID in the configuration: one in the session ID list and another in the `"sparkModuleId"` field of the `"moduleLaunchInfo"` object.

```

1 {
2   "moduleAssignmentInfo": {
3     "sessionType": {
4       "type": "interactive",
5       "interactive": {
6         "userId": "4663c4c3-a04a-48b0-81cc-a6f0ff822442"
7       }
8     }
9   },
10  "moduleLaunchInfo": {
11    "sparkModuleId": "ri.spark-module-manager.main.spark-module.8fe59113-2aa3-4e17-88db-ae3bfa4ae075",
12    "profileId": "ri.vector.main.profile.default.c1346225-a10b-4040-a345-586b50046fbf",
13    "isCustomEnv": true,
14    "jarDependencies": [],
15    "moduleLaunchType": "ON_DEMAND_MODULE",
16    "moduleResources": {
17      "type": "staticAllocation",
18      "staticAllocation": {
19        "driverCores": 1,
20        "driverMemoryInMib": 4096,
21        "gpuResources": null,
22        "executorCores": 0.6,
23        "executorMemoryInMib": 4096,
24        "executorInstances": 2

```

Because each user is assigned one Spark module that is used across workbooks in the same project with the same environment, your interactive job may queue on other jobs from a different user completing. For example, up to five Python jobs can run on the same module. The sixth job will appear as "Queueing in Code

API Reference ↗

Send feedback

Within a batch build (for example, a scheduled build or a build from Dataset Preview), one build will use one Spark module per environment in the build. For example, if a scheduled build contains datasets created within multiple Code Workbooks, and these Code Workbooks all use the same environment, the build will use the same Spark module for all jobs. No interactive jobs will be routed to that Spark module.

We recommend using batch builds when the desired outputs are saved as datasets, and you are not running transforms one-by-one to iterate on code. This may include the following cases:

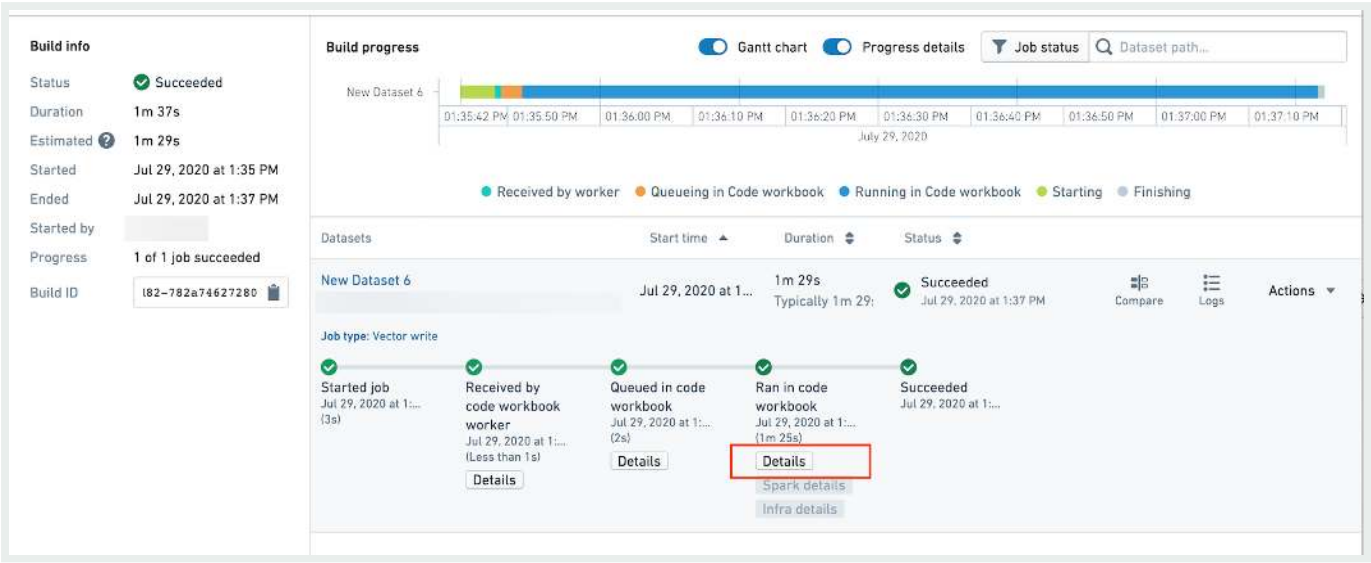
- You would like to build some long-running transforms in the same workbook or other workbooks using the same interactive Spark module.
- You want the datasets to be updated on some regular cadence. For example, if you'd like to see updated data when there is new input data, or daily at 9AM, you should use a scheduled batch build.

From a workbook, use **Open Dataset** to view a dataset in Dataset Preview and build using the **Build** button in the top right of the screen. To build multiple datasets, navigate to the cog at the top of the page, then choose **Explore Data Lineage**. Then, select the datasets you want to build and choose to build them in the right sidebar.

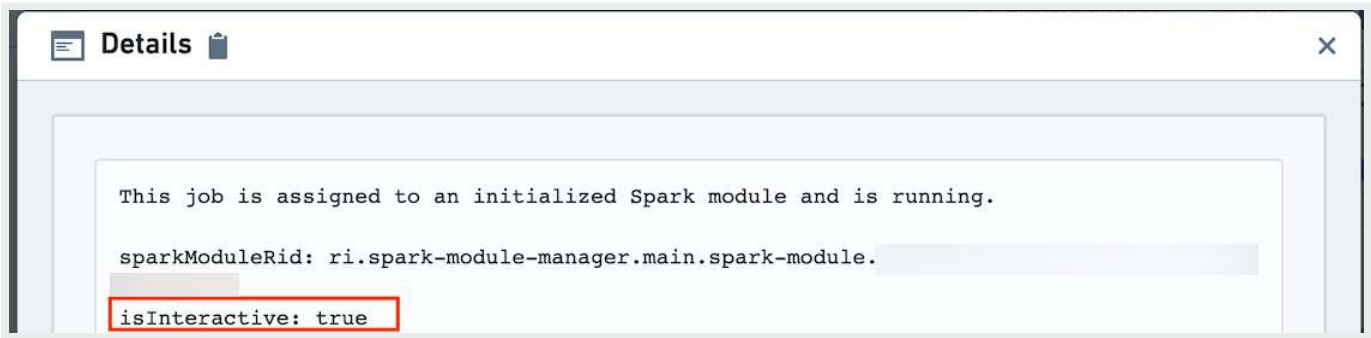
The screenshot displays the 'Data lineage' interface. The top toolbar includes icons for Tools, Layout, Undo/redo, Clean, Select, Expand, Color, Find, Remove, Align, Layout by color, Group by color, Legend, and Node color options. A legend below the toolbar identifies 'Uploaded Dataset (1)' as pink and 'Code Workbook Dataset (2)' as green. The main workspace shows a flow from 'dataset_1' to 'dataset_2'. On the right, a sidebar titled '2 datasets selected' contains a 'Builds' section with three radio button options: 'Selected dataset(s) only' (which is selected), 'All transforms in between selected dataset(s)', and 'All ancestor datasets'. A 'Next (View preview)' button is located at the bottom of this sidebar. At the bottom left of the interface, there is a link labeled 'API Reference'. At the bottom right, there is a dark green button labeled 'Send feedback'.

Alternately, use the same sidebar and click on the calendar icon to set a schedule to build the datasets. [Learn more about setting up schedules.](#)

To tell if a job is being built in batch build or interactive mode, navigate to the Builds application and click on the "Details" button.



The details will list the ID of the Spark module, and whether `isInteractive` is true. If true, the job is running in interactive mode. If false, the job is running in batch build mode and does not share a Spark module with any interactive jobs.



Analytics / Code Workbook [Legacy] / Environment / Environment creation overview

Environment creation

i This is an advanced guide that goes into detail about the environment initialization process. It is intended for users who are interested in the technical considerations that can affect initialization performance.

For general guidance regarding common environment-related issues, see the [Environment Troubleshooting Guide](#).

Conda is an open-source language-agnostic package and environment manager. Mamba is an open-source reimplementation of the Conda package manager. Code Workbook has moved to using Mamba to resolve package dependencies and install sets of packages into independent environments. Mamba offers several advantages in package resolution over Conda, most notably in increased speed and increased legibility of error messages.

This page introduces the most important concepts and outlines the environment creation process; for more information, consult the [official Conda documentation ↗](#) and [Mamba documentation ↗](#).

Important Terms

- [Package](#)
- [Channel](#)
- [Environment](#)

API Reference ↗



Send feedback

A **package** is a collection of files that commonly contains metadata, libraries, and/or binaries. Code Workbook provides access to a broad selection of packages (`numpy` , for example) to supplement the core language features.

A package is *versioned*, and nearly always has a set of dependencies—other packages that must also be installed for it to function properly. A dependency might be a specific version of a package, a range of acceptable versions, or any version at all.

Channel

A **channel**, sometimes called a repository, is any location where packages are stored. One channel might be a directory in the local file system, and another might be a directory hosted on a web server.

Regardless of type, each channel is a directory tree that separates packages by platform architecture. Each platform subdirectory contains a file called `repodata.json` , which is an index of all packages in that subdirectory.

Conda searches a set of preconfigured channels whenever it needs to fetch a package. For more information about channel management in Foundry, see the [Code Repositories documentation](#).

Environment

A Conda **environment** is a directory that contains a specific collection of packages. An environment is created for each Workbook by passing to Conda the packages specified in the Environment Configuration panel. Conda constructs a set of packages that satisfies the configuration and all dependencies, and installs these packages onto the Spark module that backs the Workbook.

Performance

The [API Reference](#) section of performance draws on [this Anaconda blog post](#) , which discusses Conda performance at length, but will similarly apply to a Member implementation. The next two sections summarize this material and performance factors that are most relevant to Code Workbook:

Send feedback

- [Creating an Environment](#)
- [Limitations](#)

Creating an Environment

Environment creation comprises two major steps: the solve step and the install step.

Solve step

In the *solve step*, the designated package manager, either Conda or Mamba, attempts to find packages and versions that satisfy all transient dependencies. Transient dependencies comprise the dependencies of the packages specified in the Environment Configuration panel, those dependencies' dependencies, and so on. This step contains four stages:

1. Download and process package indices. Conda will download relevant `repodata.json` files from each configured channel, and will convert the index entries into objects in memory.
2. Reduce the index. Conda builds up a set of all packages that could possibly be used for the environment. To do this, the algorithm begins with the provided package specification and recurses through all dependencies. All unneeded packages—primarily those that are not in the dependency graph—are pruned away.
3. Express dependency constraints as a Boolean satisfiability (SAT) problem. Conda prefers certain types of solutions (e.g. use the newest possible versions of packages), and these biases are baked into clause construction.
4. Run the SAT solver.

Install step

If the solve succeeds, the *install step* is next. Here, each artifact is retrieved from the proper channel and uses them to construct the environment. This step contains three stages:

1. Download and extract all packages in the solved environment.
 2. Verify package contents. Depending on configuration, Conda will either use a checksum or the file size is correct.
- to the environment.

API Reference ↗

Send feedback

Limitations

All of the steps outlined above can be susceptible to slowness in certain situations. Causes of slowness usually fall into one of three categories:

- [Upstream changes](#)
- [Environment specification](#)
- [Package size](#)

Upstream changes

A significant portion of slowness is caused by factors external to Foundry.

- Downloading and processing channel indices scale with total size of index files; the more channels that are needed to consider and the larger these channels are, the longer these steps will take.
- Index reduction also scales with the number of transitive dependencies; the number of transitive dependencies is determined by which dependencies packages choose to declare.

Because these factors are external and opaque, it can be difficult to perform root cause analysis on performance regressions. An environment may suddenly take longer to load because a channel recently increased in size, or because a package declared several new dependencies in its newest release.

Environment specification

More commonly, slow initialization is directly tied to the **environment specification** itself. The solve step scales superlinearly with environment size, so as a general rule of thumb, environments with more packages will take disproportionately longer to initialize.

There are two ways to remediate these situations.

- First, remove unneeded packages from the environment definition. It is much more [API Reference ↗](#) to have small, specialized environments than to have large, general-purpose ones.
- Second, try adding version constraints to some of the packages in the [Configuration panel](#). It is most effective to pin versions for packages with many extant

[Send feedback](#)

builds like `python` and for packages with complex dependency graphs like `scipy`. Doing so will allow Conda to more aggressively reduce the indices, meaning that the SAT solve will not need to account for as many package versions.

Package size

Package size is typically less problematic than the other factors in this section, but it is still relevant in some cases.

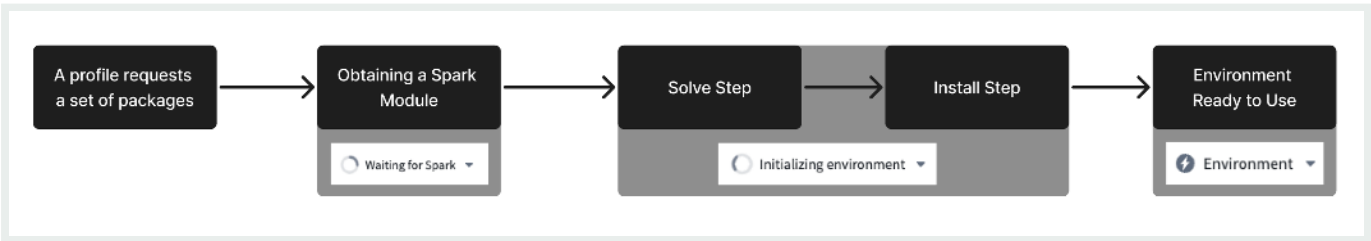
Downloading and extracting even a single package may not be trivial. For example, the `pytorch` package is about 460 MB in size, and can take 35+ seconds to extract.

Download, extraction, and verification all scale linearly with the size and number of packages in the environment. Due to transitive dependencies, the solved environment typically contains many more packages than were explicitly specified in the environment definition, and the increase in packages may cause slowness.

Remediation in this case is similar to the suggestions for [environment specification](#): try to keep environments as small as possible.

Environment optimizations

The previous sections of this page describe the [environment initialization process](#) and [factors that affect initialization performance](#). The following diagram summarizes the standard process of obtaining an environment:



To decrease time spent in the “Waiting for resources” and “Initializing environment” stages of the process, Code Workbook uses a series of optimizations designed to obtain an

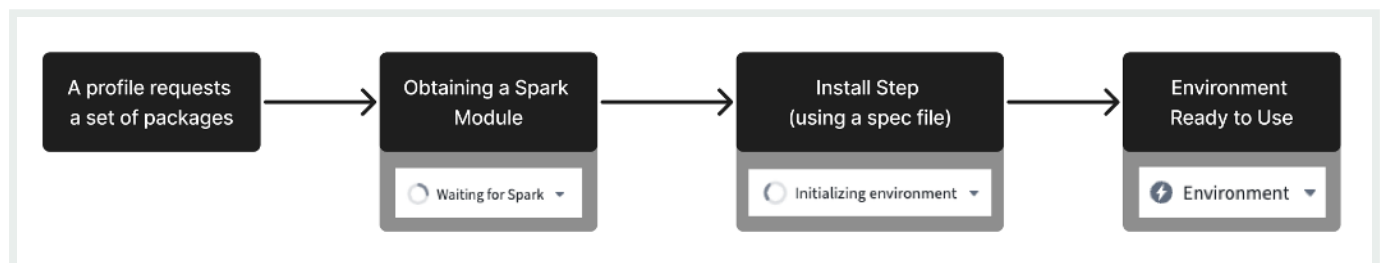
e [API Reference ↗](#) kly as possible.

Send feedback

Solve step optimization: Spec files

If no changes are made to the set of requested packages of the environment, there should be no need to solve the environment again, as that would lead to the same resolved environment. As a result, Code Workbook avoids the [solve step](#) of the environment initialization by storing the result of a successful solve in a *spec file*. The next time that an identical environment needs to be initialized, Code Workbook skips the solve step by referring instead to the spec file. This allows for a strictly better initialization time, since Code Workbook only has to perform the install step. As a precautionary measure for the rare cases where previously successful environments stop working, Code Workbook will, by default, invalidate a spec file after 24 hours. After that, a new spec file will be generated on the next successful solve of the environment.

As a result, with spec files, every subsequent initialization attempt of a known environment will look like the following diagram:



- i** Initializing a new environment that Code Workbook has never seen before will naturally lead to slower initialization times. However, once the environment initializes successfully, all subsequent initialization attempts should succeed much faster.

Initialization optimization: Conda Docker

- i** Conda Docker is an optimization only available on Kubernetes-enabled environments. More information on Kubernetes can be found in the blog post [Introducing Rubix: Kubernetes at Palantir ↗](#).

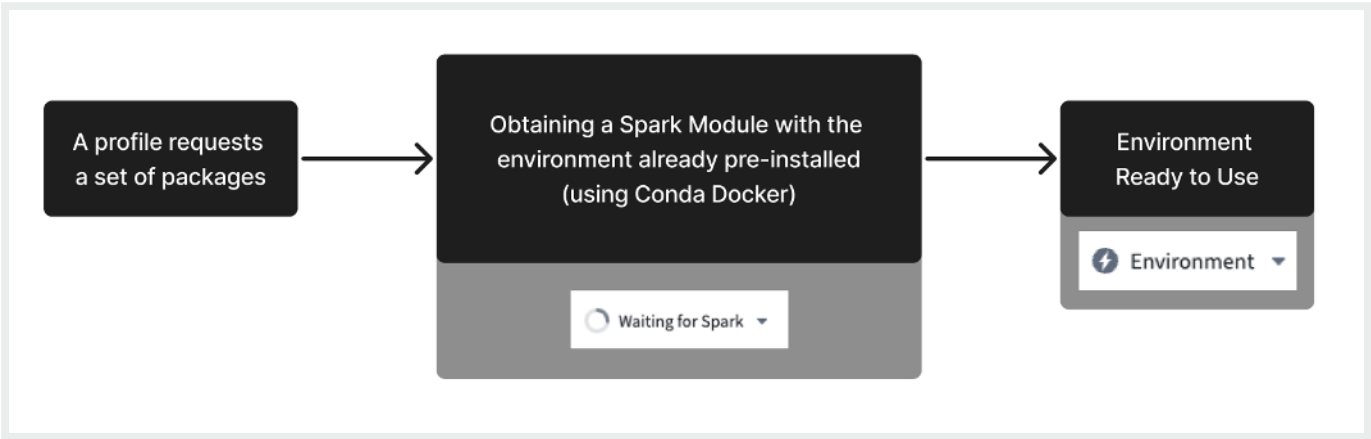
As shown in the diagrams above, to initialize an environment, Code Workbook must first obtain a Spark module, then install packages onto the module. Similarly, Code Workbook stores the result of a successful solve in a spec file, valid for 24 hours. This spec file can be stored as a Docker image. Using this image, Code Workbook can request subsequent

[API Reference ↗](#)

[Send feedback](#)

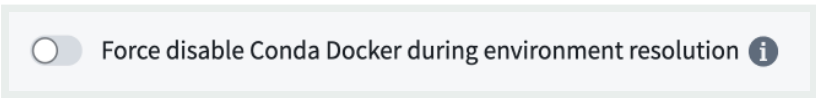
Spark modules with all necessary packages already present on the module. This bypasses both the solve step and the on-module Conda install step, leading to much faster environment initialization times. Code Workbook will, by default, invalidate a Docker image after 24 hours. A new image will be generated on the next successful solve of the environment.

The following diagram illustrates how Conda Docker simplifies the initialization process:



Disable Conda Docker

In rare cases, environments using Conda Docker may fail at runtime if their packages contain [pre-link or post-link scripts ↗](#). If an environment containing such scripts appears to be failing, you can manually disable this optimization in the bottom left of Code Workbook’s environment configuration tab. After disabling Conda Docker, environments will still benefit from the other available optimizations. This configuration will persist for the combination of the profile used and the workbook branch on which it was toggled.



Prewarming

While the optimizations mentioned above can shorten the initialization time of an environment, you can avoid waiting for an environment altogether by proactively initializing some number of modules. Those modules are referred to as “warm” modules. When configuring a warm module queue, Code Workbook ensures that a set of Spark modules are always ready to be assigned. For more information on how to configure

[API Reference ↗](#)

[Send feedback](#)

warm modules, see the [Prewarming](#) section of the Code Workbook configuration documentation.

i If you regularly encounter long times waiting for an environment to initialize, consider implementing a warm module queue for the profile in question.

! The optimizations mentioned above benefit environments that Code Workbook has already stored. When requesting a custom environment, Code Workbook must perform the standard initialization process, which will not benefit from any optimizations. As a result, custom environments will generally have slower initialization times. To troubleshoot a slow initialization process of a custom profile, consult the [troubleshooting](#) documentation.

PREVIOUS

← [Batch builds and interactive builds](#)

NEXT

[Session history and session pinning](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

—[Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Environment / Session history and session pinning

Session history and session pinning

Sessions

When you initialize a Code Workbook environment within a Spark module, a series of metadata is attributed to the module. This metadata can be divided in two categories: the compute information and the environment information. A **session** is an instantiation of these settings as part of a Spark module lifecycle or, informally, “what was true about a given Spark module during its lifetime”.

The compute information includes details about the Spark settings attributed to the Spark module, as well as other relevant information such as jar dependencies, resource identifiers, and the type of module being launched. On the other hand, the environment information can be further broken down into two categories: the requested and resolved environments. A **requested environment** is a set of conda specifications, such as requesting `pandas=2.*` or `python<3.10`, while the **resolved environment** is a set of resolved packages that satisfy the constraints established by the requested environment. It is important to note that a requested environment is non-deterministic, while a resolved environment, by definition, is a permanent solution of a given requested environment. As a result, two identical requested environments may lead to different resolved environments. For example, requesting `python>=3.6` in 2017 would likely resolve to Python 3.6, while the same request today could lead to Python 3.10.

Session history

[API Reference ↗](#)

Code Workbook provides information about all the recent sessions of a Workbook. To view the session history of a Workbook, select the **Environment** dropdown menu in the top right corner of the Workbook and select **View session history**.

[Send feedback](#)

 Environment ▾

Profile: **default**

View resolved packages

Restart Spark session

View session history

Configure environment

The **Session history** window provides information through three different tabs: **Compute information**, **Requested environment**, and **Resolved environment**. The left pane provides the ID of the session as well as the timestamp at which the session was initialized. The icon to the left of the session ID will indicate whether it successfully initialized (green), failed (red), or other (blue). A blue session typically means that the session has not finished initializing and therefore reached neither failed or success states.

SESSION HISTORY

Sessions

9d3ef8fd-2841-4386-8f8f-0...

Today at 11:38 AM

c4e6a6e5-4270-4b14-b319-6...

Today at 11:36 AM

67094079-437f-4867-812b-6...

Jul 27, 7:57 PM

bec7bfb9-5379-4e56-b464-b...

Jul 27, 4:47 PM

02083a0f-6cf0-4ecd-a078-c...

Jul 17, 10:16 PM

537528d0-3974-48f6-ab6e-9...

Jul 17, 9:17 PM

ddc1e8cb-6ba1-4fb0-8022-4...

ri.spark-module-manager.main.spark-module.9d3ef8fd-2841-4386-8f8f-0c9275621c57

Compare sessions

Compute information

Requested environment

Resolved environment

1 Initialization mode: Docker

2

3 Environment repository:

4 - ri.vector.main.profile.default.80b6ad82-bdea-43a8-a936-db00252722d1

5

6 Requested packages:

7 - matplotlib=3.*

8 - numpy=1.*

9 - pandas=1.*

10 - python=3.7

11 - r-base=3.6.*

12 - vector-spark-module-py=5.510.0

13 - vector-spark-module-r=5.510.0

14

15 Backing repositories:

16 - ri.artifacts.repository.artifactory.internal-docker-release

17 - ri.vector.runtime.artifacts.repository

API Reference ↗

Send feedback

Compute information

The **Compute information** tab offers information about the type of Spark module that was requested for a given session:

- **moduleAssignmentInfo:** General information about whether an environment was spun up by a specific user for interactive Workbook usage or by a background job, such as a scheduled build. For more information about the difference between an interactive and a build module, see [batch builds and interactive builds](#).
- **moduleLaunchInfo:** Metadata about the properties assigned to the Spark module itself.

This includes the following fields:

- **sparkModuleRid:** The unique identifier of the Spark module. Given that there is a 1:1 mapping between a module and a session, you will notice that the `sparkModuleRid` and the `sessionId` will share the same unique identifier
- **profileRid:** The unique identifier of the profile with which the Spark module was initialized.
- **isCustomEnv:** Whether that session was initialized using a custom environment. Recall that a custom environment in Code Workbook is defined as any standard profile that was modified directly in the Environment configuration menu of the Code Workbook interface.
- **jarDependencies:** A list of all jar dependencies manually added to the profile in [Control Panel](#).
- **moduleLaunchType:** Can either be `WARM_MODULE` or `ON_DEMAND_MODULE`. Indicates whether the session used an already initialized module from the warm module queue, or requested a fresh module to start its initialization process.
- **moduleResources:** All the compute information tied to the module, including driver and executor memory, amount of executors, and more.

Requested environment

The **Requested Environment** tab offers information about the desired environment settings before the start of its initialization:

API Reference ↗

Initialization Type: The type of Code Workbook initialization to be performed for that session. It can either be `solve`, `file`, or `docker`. For more information on the types of initializations in Code Workbook, see documentation about [environments](#).

Send feedback

- **Environment repository:** The key under which the environment in question is stored. For non-custom environments, this will be the `profileRid` used for the session. For custom environments, this will be the `workbookRid` of the workbook in which the profile was customized.
- **Requested packages:** The list of requested packages submitted for the initialization process. This list will include both the packages specifically requested by the profile, as well as any additional packages automatically requested by Code Workbook for it to function.
- **Backing repositories:** The list of channels from which the installed packages were sourced.

Resolved environment

The **Resolved Environment** tab offers information about the environment packages used as part of the initialization. This includes the time it took to initialize the environment, as well as the packages and their versions that ultimately got installed onto the module. This information is particularly important, as the session rollback feature of Code Workbook borrows the resolved environment of a previous session rather than a requested environment.

Compare sessions

It is often helpful to compare two given sessions to understand how a given environment may have changed. The **Session history** window allows you to compare your current session with any historical session. You can access the session history comparison tab by selecting **Compare sessions** at the top right of the session history window.

The session comparison menu will provide two windows next to each other. On the right, the information of the current session is displayed. On the left, any session from the **Sessions** list can be displayed for comparison. To switch the session to be compared on the left, select any session from the Sessions list. You can exit the comparison view at any time by selecting **Exit comparison** at the top right of the window. The left selected session will remain selected.

API Reference ↗

Sessions can be compared across all three available tabs. Comparing the **Resolved environment information** of two sessions may be helpful in understanding changes to the environment available of the module. Comparing the **Requested environment** is helpful to understand

Send feedback

In the examples below, using the various tabs of session comparison reveals the following information about two sessions:

- A different profile was used.
- The profile was customized.
- Smaller memory settings were employed on the module.

Send feedback

- More recent package versions of Python and pandas were requested, and the error got solved

Comparing	✓ f327a1a7-bfd5-48c8-a348-f90ad...	with	✓ 0fa97125-7127-4b99-a553-0bb...	Exit comparison ↺
Compute information	Requested environment			
	Resolved environment			
✓	f327a1a7-bfd5-48c8-a348-f90ad90909e9		✓	0fa97125-7127-4b99-a553-0bb1d69340eb
179	- python-dateutil=2.8.2=pyhd8ed1ab_0		128	- python-dateutil=2.8.2=pyhd8ed1ab_0
180	- python=3.8.15=h257c98d_0_cpython		129	+ python=3.6.15=hb7a2778_0_cpython

- Different versions of Python ended up being installed on the environment.

Session pinning

In certain cases, you may want to temporarily rollback to the exact same settings of a historical session. Code Workbook allows this behavior by providing the opportunity to pin a session. Pinning a session means initializing a brand new session and Spark module using the same metadata as a historical session to reproduce a seemingly identical environment. A pinned session will borrow the **resolved** information from a historical session to initialize a fresh module. This is particularly important, because using the same resolved environment guarantees the installed packages to be the same, while using the same requested environment does not. As a result, pinning a session simulates the effect of rolling back to a previously working environment. A select few pieces of metadata, such as the initialization mode, are not borrowed from the historical session.

How to pin a session

To pin a session, navigate to the **Session history** window and select the session you want to pin. Then select **Pin session** at the bottom right of the panel. The current branch of the Workbook will have a pinned session override that will last up to for 24 hours. A banner will display the remaining time of the override, as well as an option to remove the pin. During that period, every subsequent interactive environment initialization will borrow the pinned session's information. When the period expires or when the session is manually unpinned, the Workbook will revert back to using its currently configured environment.

API Reference ↗

g is designed for debugging purposes, and should not be relied on production-ready pipelines. For that reason, a pinned session will only affect *interactive* environments. Builds executed outside of Workbook interface, such as scheduled builds, will not be affected by the pinned

Send feedback

session override. We recommend that you restrict session pinning to occasional troubleshooting and experimentation. Instead, use the [session history](#) feature to understand the differences between various sessions, and edit the environment definition directly.

Restrictions of session pinning

Certain limitations apply when pinning a session. Pinned sessions do not last infinitely, not every session can be pinned, and not every initialization will be affected by a pinned session. Find below a list of restrictions to be mindful of when considering pinning a session:

- A session pin may last up to 24 hours. To extend this time further, you will need to re-pin the session. Re-pinning a session will cause a new module to be spun up and all local state on the module will be lost.
- A pinned session does not borrow all of the metadata from its attached historical session. Settings such as the initialization mode may differ.
- Only previously successful sessions can be pinned. Unfinished or unsuccessful sessions are disallowed. Additionally, Code Workbook may defensively prevent you from selecting a version for pinning due to incompatible versions, blacklisted package versions, API breaks, and so on. A red banner will inform you of sessions that cannot be pinned.
- Pinned sessions only affect interactive jobs - not build jobs.
- Only sessions using [Artifacts Profiles](#) can be pinned

For the reasons above, pinning a session is a debug feature that should not be relied on for long-term, production-ready pipelines.

Troubleshooting environments using session history

The [session history](#), [session comparison](#), and [session pinning](#) features mentioned above can be instrumental in troubleshooting failing environments. Particularly, they help address failing environments. Follow these steps to remediate such cases:

API Reference ↗

1. Has the environment worked in the past?
 - If no, refer to the [Environment Troubleshooting Guide](#).

Send feedback

- If yes but it is now failing, proceed to the next step.
2. Using the [Compare sessions](#) feature, select the currently failed session with a previously succeeding and observe the differences in the environment.
 - If there are changes in the compute information details, the module settings may not contain sufficient memory for the module to correctly initialize.
 - If there are changes in the requested environment details, a manual breaking change was introduced to the environment which led to the failure.
 - If there are no changes in the requested environment details, but changes in the resolved environments that led to a failure, a new version of one of the packages used may contain a breaking change. These packages generally come from open source. You can investigate the issue in the faulty package directly or modify your environment to avoid requesting the package or version in question.
 - If there are no apparent changes in the environment or the steps above did not help, consult the [Environment Troubleshooting Guide](#) or contact Palantir Support for further assistance.
 3. (Optional) While troubleshooting during the previous step, use the [Session pinning](#) feature to first ensure that the pinned version of the environment works, and to temporarily unblock the functionality of the Workbook while a more permanent solution is found.
 4. After discovering the root cause of the environment failing, adjust your profile's settings accordingly to permanently remediate the situation.

PREVIOUS

← [Environment creation overview](#)

NEXT

[Troubleshooting guide](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Troubleshooting guide

- [My custom environment initializes slowly](#)
- [Environment fails to initialize](#)
 - [Code 1 - Environment resolution error](#)
 - [New Mamba error messages](#)
 - [Legacy Mamba error messages](#)
 - [Code 137 - Module ran out of memory](#)
 - [Other Codes](#)
- [Default environment initialization speed is inconsistent](#)
- [Packages which require both a Conda package and a JAR](#)
- [Other issues](#)

My custom environment initializes slowly

Slow initialization generally indicates that the environment definition is too large or too complex. Initialization time tends to increase superlinearly with the number of packages in the environment. Additionally, Foundry will often pre-initialize commonly used environments, so if you created a custom environment based on a default profile, the slowness is likely because you are no longer getting a pre-initialized environment and must wait for a new environment to be created. See [Introduction to Environment Creation](#) for more details about the environment creation process.

Try the following:

API Reference ↗

☰

🔍

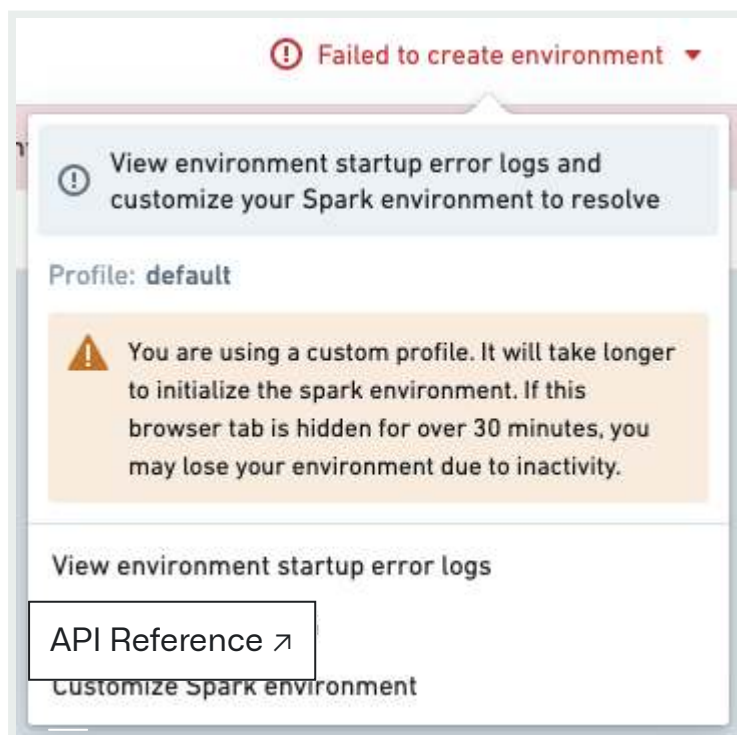
Environment began initializing slowly after a change was made to its configuration. Reverting the change. Adding even a single package can dramatically increase initialization time in certain cases.

Send feedback

2. **Simplify** – Remove any unneeded packages from the environment definition. Small, specialized environments are much more performant than large, general-purpose ones.
3. **Pin versions** – Add version constraints to some of the packages in the Environment Configuration panel, as discussed in [Selecting an Environment](#). It is most effective to do this for packages with many possible versions like `python` and for packages with complex dependency graphs like `scipy`. If adding a particular package caused the environment to become slow, try pinning a version for that package.
 - **Caveat:** Adding version pins may cause an environment to become unsatisfiable. If the environment fails to initialize, try relaxing the pin. For example, `python=3.6.10` will match only that version, but `python=3.6` will match any of `python 3.6.x`, and so is more likely to be satisfiable.
 - One option is to wait for your environment (with no pins) to resolve, then check which versions were selected (see [Viewing Resolved Environment](#)). You can then add pins for those specific versions until initialization performance is sufficiently fast. It is usually optimal to pin versions for packages with many dependencies while leaving all other packages unpinned.

Environment fails to initialize

If your environment fails to initialize, you can view the environment startup error logs to view additional information about what went wrong.



Send feedback

The first line of the log will read `Execution failed with non-zero exit code:` followed by an integer error code. This error code indicates the specific failure mode.

Code 1 - Environment Resolution Error

This error indicates that some subset of the specified packages (and/or their dependencies) are incompatible or could not be installed. If a recent change in the Environment Configuration panel caused initialization to begin failing, try reverting the change. Below is a list of common environment resolution errors.

New Mamba error messages

Palantir [contributed ↗](#) to the open-source Mamba community by providing better formatting of environment initialization error messages. As of February 2023, Foundry services benefit from errors that more closely represent the dependency tree being infringed by environment failures.

For guidance on understanding those new messages, how to interpret them, and advice on how to remediate them, see [New error messages](#).

Legacy Mamba Error Messages

Find below a list of common error messages from the legacy error messages, before the [new Mamba error messages](#) were introduced.

Package not found

In this case, no configured channel provides the package `A` dependency.

```
1 Problem: nothing provides requested A.a.a.a
```

This can happen if the pinned version of your package does not exist. If this is the case, try refer to [API Reference ↗](#) the versioning from your package within the `meta.yaml`; for example, `matplotlib 1.1.1` could become `matplotlib`. Alternatively, this may also mean that the environment manager could not locate the necessary package to install the requested version.

Send feedback

Dependency not found

In this case, package `A` contains a required dependency `B` which was not provided by any channel.

```
1 Problem: nothing provides B-b.b.b needed by A-a.a.a:
```



Note that `B` can be a package the user requested explicitly (in `meta.yml/foundry-mL/vector` profiles) or it can be a transitive dependency.

This may occur if `B` is not available for installation on your enrollment; for example, `B` may have been recalled and therefore is not available for install.

If this is the case, try removing the pinned version of `A` in case there is a version of `A` that has all its dependencies available, or contact your Palantir representative to import the necessary package into Foundry.

Duplicate package error

```
1 cannot install both X.a.b.c and X.d.e.f
```



This error occurs if you try to install the same package `X` and have two different version pinnings for both. For example, you might receive this error if you try to pin both `python 3.6.*` and `python 3.7.*` in your `meta.yml`. You can resolve this issue by removing one of the duplicate package pinnings.

Permission errors

```
1 CondaService:ReadRepositoryPermissionDenied
```



If [API Reference ↗](#) or, it may be because some of the assets and packages you need are not available in your enrollment.

[Send feedback](#)

Package conflict

In this case, package **A** has a requirement on the version of package **B** but this version of **B** conflicts with other packages:

```
1 Problem: package A-a.a.a requires B >=2.2.5,<2.3.0a0, but none of the p
```

Note that the package **B** could refer to a transitive dependency of **A**, which you have not explicitly listed as a requirement in your environment. See [Resolved Environment](#) documentation for more information on transitive dependencies.

Remediating a Code 1 error

To remediate an environment failure, consider the following steps:

1. Check the **STDOUT** in the environment startup logs.
2. Create a minimal example of a failing solve. Remove packages until the environment initializes, and then add packages back in until you have determined which packages are the blockers.
3. Try relaxing constraints by removing package pins.
 - You can open up multiple workbooks or multiple branches of the same workbook in different windows to make this process faster.
 - We recommend using binary search to discover issues. Try removing all constraints in one workbook, removing half of the constraints in another workbook, then the other half in another workbook, and so on.

Code 137 - Module ran out of memory

This error typically indicates that the environment initialization required more memory than was available on the Spark module. If a recent change in the Environment Configuration panel caused initialization to begin failing, try reverting the change. Otherwise, try one or more of the following suggestions:

[API Reference ↗](#)

- Remove unneeded packages from the environment. Memory usage increases superlinearly with the number of packages in an environment, so removing a few packages may be enough to resolve the problem.

[Send feedback](#)

- Add version constraints for some of the packages as described in Step 3 of the [Environment initialization is slow](#) section. Each package version that you pin will reduce the complexity of the environment solve, which will in turn reduce the amount of memory required to initialize the environment.

If your environment still fails to initialize after removing all unneeded packages and pinning some of the remaining packages to specific versions, contact your Palantir representative and include details of all debugging steps you tried.

Other codes

If your environment fails to initialize with an error code not listed above and the environment fails again on retry, contact your Palantir representative and include the full text of the error log.

Default environment initialization speed is inconsistent

We expect that default profiles will sometimes initialize quickly and sometimes take longer. In the latter case, you are waiting for Mamba to resolve your environment specification and install it onto the Spark module backing your Workbook. This is the default behavior. In the former case, either you are getting a pre-initialized module or your environment specification matches a resolved environment in the cache that can be installed right away. If many users are opening Workbooks around the same time, you are less likely to get a pre-initialized module and will need to wait for a new one to be spun up.

Packages which require both a Conda package and a JAR

Some packages require both a Conda package and a JAR in order to be available. A c [API Reference ↗](#) the graphframes package. Such packages require a special setup process, and you may experience the following error during build time if this package has not previously been configured for your instance:

Send feedback

```
1 o257.loadClass.: java.lang.ClassNotFoundException:<Class>
```



JAR Files can be added to a profile's configuration in the **Advanced** section of the Conda Environment configuration tab in Control Panel (see [Configure Code Workbook Profiles](#)). Contact your Palantir representative if you need assistance in configuring such packages.

Other issues

If the guidance above is insufficient to resolve your issue, or if you encounter an issue outside the scope of this guide, contact your Palantir representative and include details of any debugging steps you tried.

PREVIOUS

← [Session history and session pinning](#)

NEXT

[Branching / Overview](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

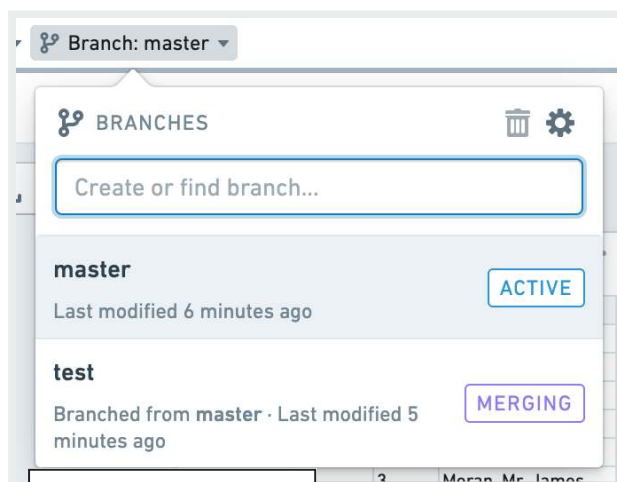
Branching

Version control tools are commonly used in software to allow groups of developers to work together as effectively as possible. The use of version control, along with good development practices, allows engineers to build features quickly while maintaining confidence that the changes they introduce are of high quality.

Branching in Code Workbook provides a version control experience tailored to data transformation, enabling teams to operate on logic and data simultaneously in a Workbook.

Branches

By default, Workbooks are created with a single branch with the same name as the default branch across all of Foundry. Typically, this branch is called `master`. You can use the branch menu to browse and select a branch, create new branches, and change branch settings.

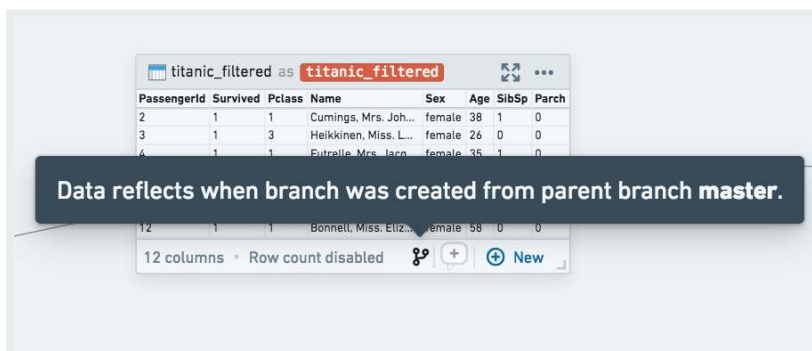
[API Reference ↗](#)[Send feedback](#)

Creating branches

⚠ By default, Code Workbook allows you to create at most 100 branches. If your workbook had over 100 branches before the limit was put in place, all your existing branches will remain, but you will not be able to create additional branches until the number of branches in your workbook is reduced to less than 100.

Type the name of your new branch in the branch menu, then click **Create branch** or use the Enter key to create your new branch. A new branch will be created in the Workbook with the current branch as its parent.

When you create a branch, Code Workbook keeps track of the state of each dataset at the time of branch creation. Any transforms you run on your new branch will use this stored state to load data. This means that changes on the parent branch will not break new branches you have created.



When you run transforms on a branch, Code Workbook creates branches on the associated Foundry datasets so that the results of your logic changes are stored in isolation from other branches. After writing to Foundry has completed, click **Open dataset** in the Output panel to view your result dataset on your branch.

Deleting branches

[API Reference ↗](#)

To delete a branch, navigate to the branch and click the trash can icon in the top-right of the branch menu. Deleting a branch that still has child branches based on it will delete the child branches as well.

[Send feedback](#)

those branches. For example, if you have three branches `master` → `develop` → `feature`, deleting `develop` will result in `feature`'s parent becoming the `master` branch.

Merging branches

Once you're done working on your branch, you can merge it back into the parent branch to incorporate your work. [Learn more about merges.](#)

Branch settings

To edit branch settings, navigate to a branch and click the gear icon in the top-right of the branch menu. In most Workbooks, you will only need to edit branch settings on the `master` branch.

Branch settings can be used to customize the process for making changes to a branch. Currently, branch settings allow you to set a few options:

- Is the branch protected? If a branch is protected, nobody can make edits to the branch directly. Instead, all changes must be merged in through another branch. Note that if a branch is protected, merging into it requires Owner permissions on the Workbook.
- Does the branch allow running? If a branch is protected and this setting is turned off, datasets on this branch must be materialized using Foundry builds. This ensures that nobody can initiate an interactive run that prevents builds from succeeding.

← PREVIOUS
Environment / [Troubleshooting guide](#)

NEXT →
[Getting started](#)

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[API Reference](#) ↗

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Branching / Getting started

Getting started

As you create Workbooks of increasing scale and complexity, you may encounter new challenges:

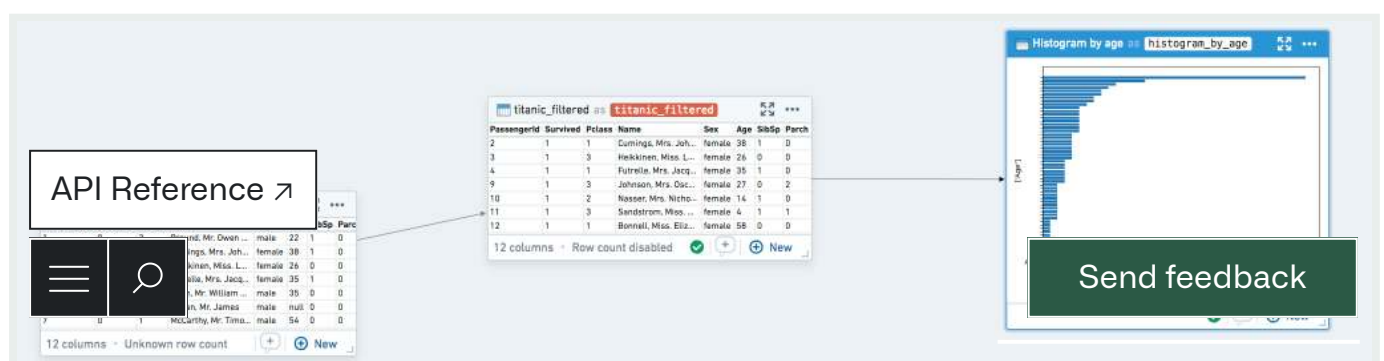
- You may want to experiment with a Workbook by introducing new changes, but must preserve existing logic and avoid breaking production pipelines.
- You may want to collaborate with a colleague on a Workbook, but need to avoid conflicts when making changes to the same Workbook simultaneously.

Branching provides a solution to these challenges. In this tutorial, we explore the following concepts:

- [Creating a branch](#)
- [Previewing a merge](#)
- [Completing a merge](#)
- [Resolving merge conflicts](#)

Creating a branch

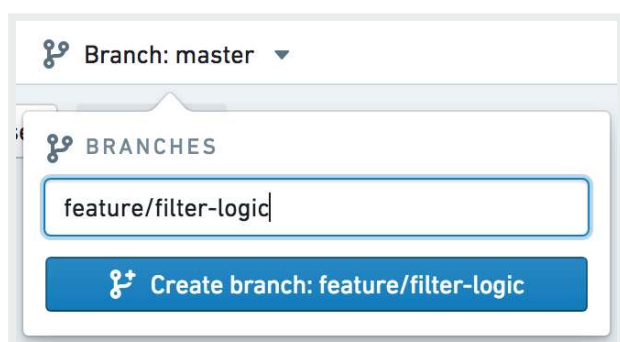
In this example, we're working on a short pipeline that starts with a dataset of passengers on the Titanic and applies a few filters to it.



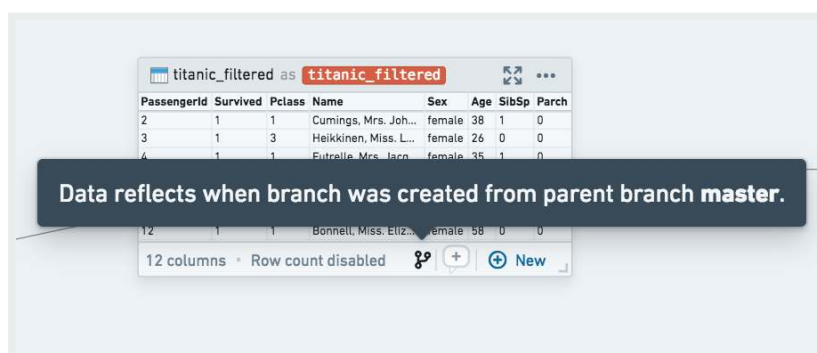
Tip

A **branch** is a working copy of your Workbook that allows you to make changes safely and incorporate them into the master version later. [Learn more about branching.](#)

Click the branch menu in the top left of the Workbook, enter the name of your new branch, and click **Create**. Branch names are commonly prefixed based on the type of change being made (e.g. `feature/` or `bugfix/`), or by your username or initials (e.g. `jdoe/` or `jd/`).



After `feature/filter-logic` is created, transforms reflect data and logic at the time of branch creation. If `master` changes, your branch `feature/filter-logic` will continue to function as before. Likewise, any changes you make on `feature/filter-logic` will not interfere with logic or data changes on `master`.



Make API Reference ↗ in your new branch as you normally would. In this example, we'll change the logic for a time or filtering code.

Send feedback

```

1 def titanic_filtered(titanic_dataset):
2     input_df = titanic_dataset
3     output_df = input_df.filter( (input_df.Survived == 1) & (input_df.Sex == 'female') )
4     return output_df

```

Previewing a merge

Tip

A **merge** is a copy of the work you've done in your branch, combined with the current state of the master copy. This allows you to review changes before incorporating them back into master.

When you're ready to introduce the changes on your branch back into the original branch, click **Preview merge** in the top right.

 Preview merge into master

 Environment (pinned) ▾

 ▾

 ▾

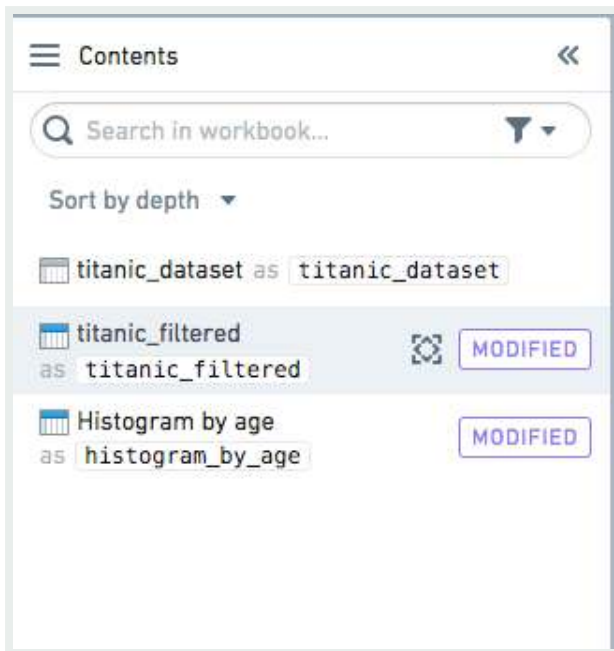
Actions ▾

You're now taken into a merge. In this state, you can continue to make changes to your logic and run transforms until you're satisfied with the changes that will be introduced. The **Run Affected** button at the top allows you to run all affected transforms in this Workbook—those with logic changes and anything downstream of them—with a single click.

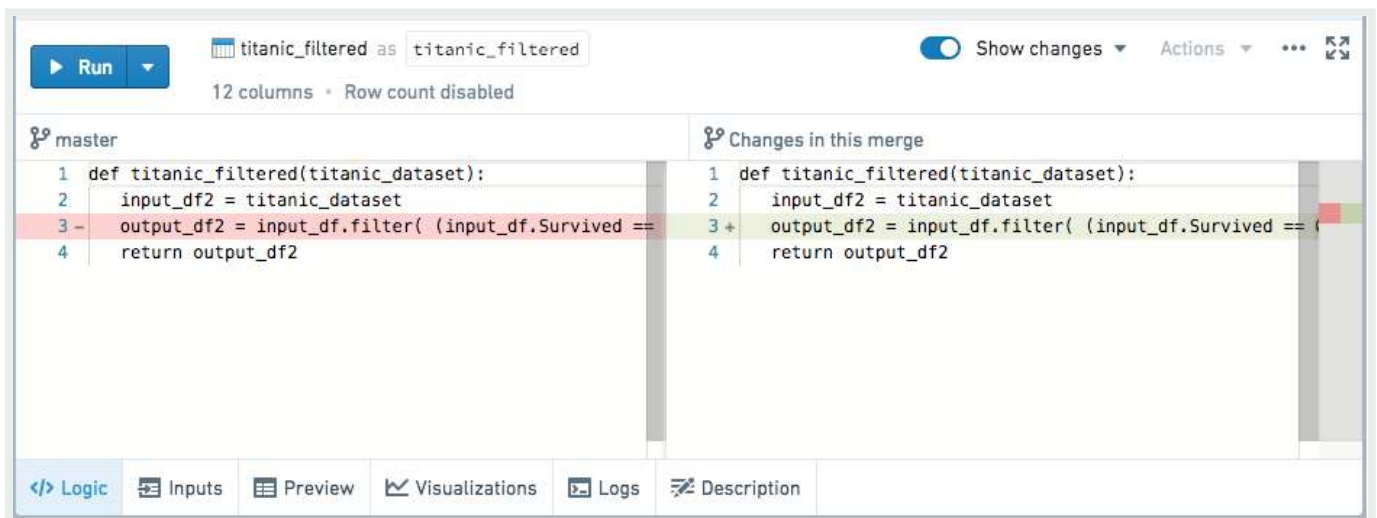


When the **API Reference** sidebar shows changes in row counts or columns that will be introduced through this merge. This can help surface changes your business logic both in transforms you actually edited as well as downstream transforms.

[Send feedback](#)



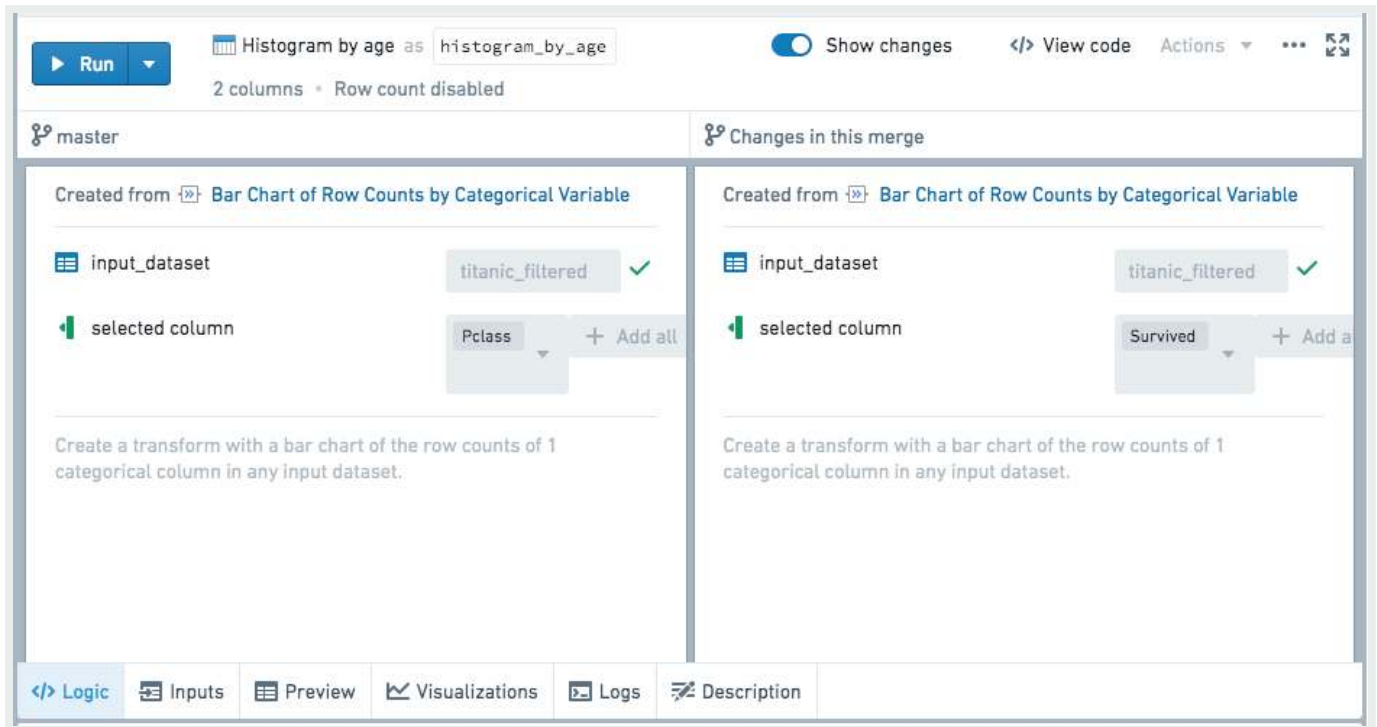
To see what logic changes will be introduced through this merge, select any modified transform and click **Show Changes**. This will show a split-screen view of the logic that has changed.



This also works for templates, as shown below:

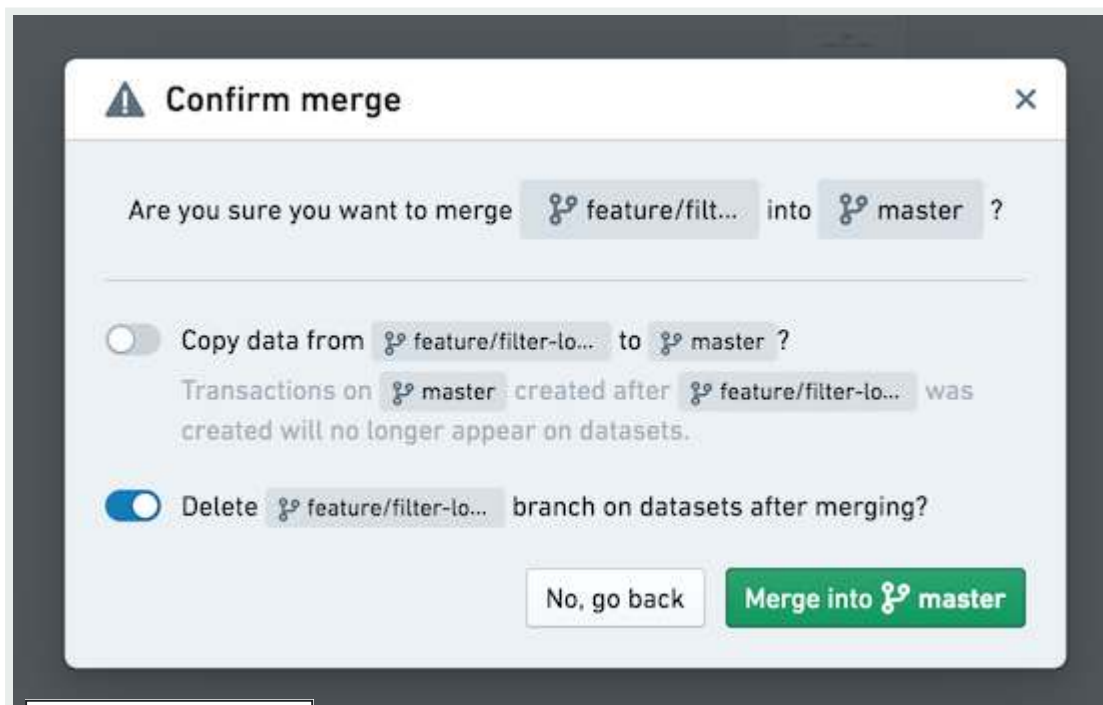
[API Reference ↗](#)

[Send feedback](#)



Completing a merge

When you're satisfied with the changes you're introducing, click **Merge Branch** to finish merging into your `master` branch. You will be presented with a dialog box with two toggles:



[API Reference ↗](#)

The first toggle allows you to choose whether to copy the transactions from the merging branch into the branch you are merging into. Let's imagine that after done additional work on master and committed new transactions on the derived datasets'

[Send feedback](#)

`master` branch. If this toggle is set to True, the transactions created on master since the `feature/filter-logic` branch was created will no longer appear on the dataset after merging.

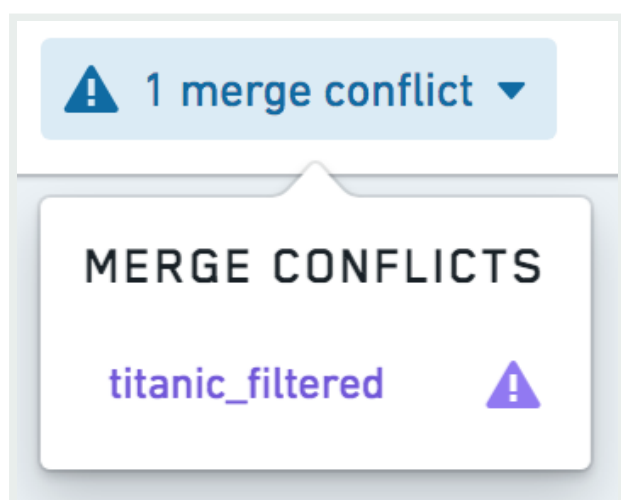
The second toggle allows you to choose whether to delete the merging branch from the datasets. Note this is different than deleting the workbook branch itself, which is always done with a merge and not configurable. If the second toggle is True, the merging branch will be deleted from the derived datasets created in the Workbook.

Let's say as above, you choose not to copy the data from the merging branch into `master`. After you click **Merge into master**, the `master` branch will be updated with the logic from your merge.

Resolving merge conflicts

As you use branching more frequently, especially while collaborating with colleagues, you may end up modifying the same piece of logic in two different branches. Let's walk through what happens in this scenario.

When you click **Preview merge**, if conflicting changes have already been introduced to `master`, a prompt will indicate that you need to resolve conflicts before proceeding with the merge.



[API Reference ↗](#)

Clicking on the conflicting transform will show a merge conflict view. markers allow you to pick which logic you want to use.

[Send feedback](#)

```

1  def titanic_filtered(titanic_dataset):
    Accept edits from master | Accept edits from feature/filter-logic | Accept from Both
2  <<<<<<< master
3      input_df2 = titanic_dataset
4      output_df2 = input_df.filter( (input_df.Survived == 1) & (inp
5      return output_df2
6  =====
7      input_df = titanic_dataset
8      output_df = input_df.filter( (input_df.Survived == 0) & (inp
9      return output_df
10 >>>>>> feature/filter-logic
11

```

If the conflict involves a template, or a transform has been deleted on one of the branches, you can resolve conflicts using a split-screen view.

After resolving conflicts, you can continue to make further edits and run transforms to verify that your merged logic functions as expected. When you're ready, click **Complete merge** to finish merging as usual.

PREVIOUS
← Overview

Merging branches →
NEXT

© 2026 Palantir Technologies Inc. All rights reserved.

[API Reference ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

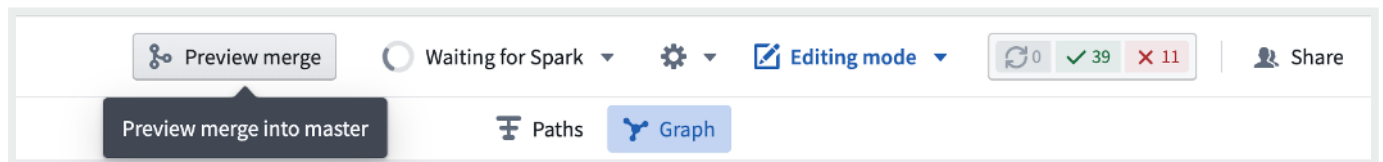
Send feedback

Merging branches

Merges allow you to stage the changes that will be introduced into the parent branch before deciding to copy them over. If there are logical conflicts between your branch and the parent branch, you can resolve conflicts during the merge and rerun your transforms to check that results are correct.

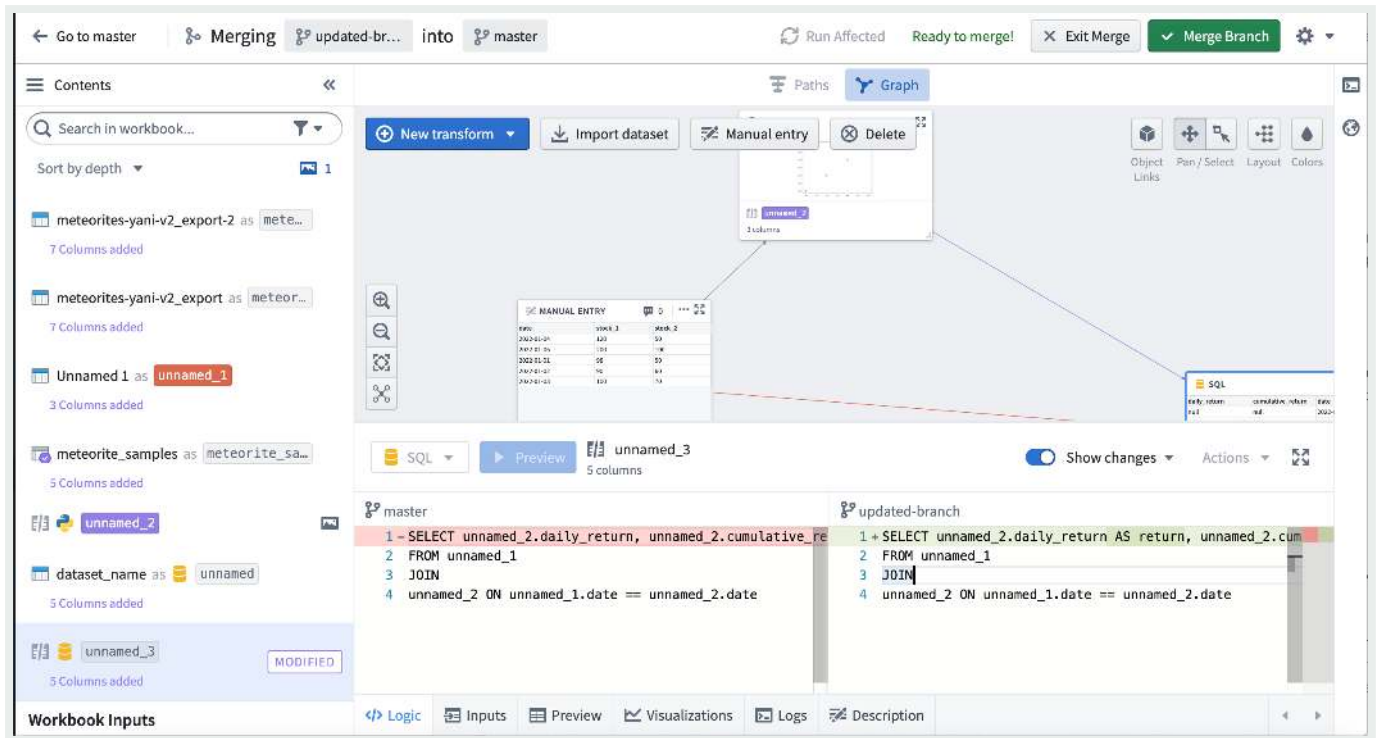
Throughout the rest of this documentation, we refer to the branch that you are attempting to merge as the *source branch*, and the branch you are attempting to merge into as the *target branch*.

Previewing merges



At any point while editing your branch, you can click the **Preview merge** button in the toolbar to enter a merge state and view changes between the source branch and the target branch. Note that Code Workbook only allows branches to be merged into their immediate parent. After entering the merge, differences between the source and target branch will be highlighted on each transform. Look for **Modified** and **Conflict** tags in the Contents panel to see changes. In the below screenshot, code changes between the source branch (on the right) and the target branch (on the left) are highlighted in green.

[API Reference ↗](#)[Send feedback](#)



When you run transforms while merging, Code Workbook automatically creates a *merge branch* on output datasets. This allows the merge to be isolated from both the target branch and the source branch. These merge branches will appear on your dataset in the form `vector-merge-{source}-{target}-{uuid}`.

Resolving merge conflicts

Merge conflicts may arise if the same transform is modified in both the source and target branch. Let's walk through what happens in this scenario.

After clicking **Preview Merge**, a prompt will indicate you need to resolve conflicts before proceeding with the merge.

[API Reference ↗](#)

[Send feedback](#)



Clicking on the conflicting transform will show a merge conflict view. For code, inline conflict markers allow you to pick which logic to choose.

```
1  def titanic_filtered(titanic_dataset):  
    Accept edits from master | Accept edits from feature/filter-logic | Accept from Both  
2  <<<<<<< master  
3      input_df2 = titanic_dataset  
4      output_df2 = input_df.filter( (input_df.Survived == 1) & (inp  
5      return output_df2  
6  =====  
7      input_df = titanic_dataset  
8      output_df = input_df.filter( (input_df.Survived == 0) & (inp  
9      return output_df  
10 >>>>>> feature/filter-logic  
11
```

After resolving the merge conflicts, the **Merge branch** button will become enabled.

Managing merges

While a branch is being merged, you can use the back button in the top left of the toolbar to go back to the source branch before you started the merge, changes you made in the merge will be removed when exiting.

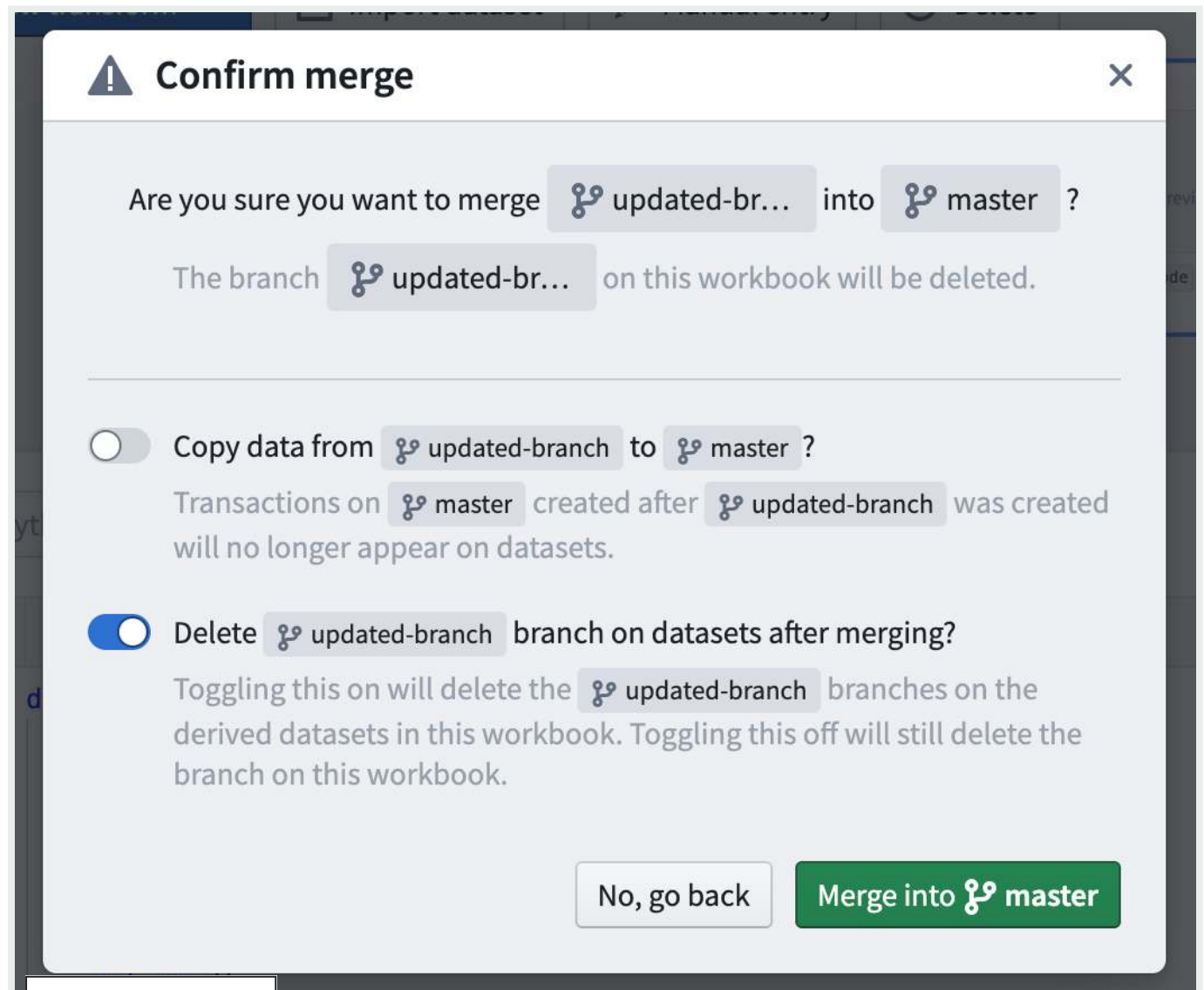
API Reference ↗

Send feedback

Note that preparing and exiting merges is completely safe and will not affect changes you have made on your branch – you can freely preview and exit merges to see the changes that will be introduced to the target branch. If changes are introduced on the target branch after you begin your merge, an **Update merge** button will appear in the toolbar. You can use this button to update your merge with the latest logic from the target branch.

Completing a merge

Once you have resolved conflicts and any other branch-specific checks have succeeded, click **Merge branch** to introduce changes into your target branch. You will be presented with a dialog with two toggles:



The image shows a 'Confirm merge' dialog box. At the top, it asks 'Are you sure you want to merge `updated-br...` into `master` ?'. Below this, it states 'The branch `updated-br...` on this workbook will be deleted.' There are two toggle options: 'Copy data from `updated-branch` to `master` ?' (which is currently off) and 'Delete `updated-branch` branch on datasets after merging?' (which is currently on). The first toggle has a sub-note: 'Transactions on `master` created after `updated-branch` was created will no longer appear on datasets.' The second toggle has a sub-note: 'Toggling this on will delete the `updated-branch` branches on the derived datasets in this workbook. Toggling this off will still delete the branch on this workbook.' At the bottom right, there are two buttons: 'No, go back' and 'Merge into `master`'.

Confirm merge ✕

Are you sure you want to merge `updated-br...` into `master` ?

The branch `updated-br...` on this workbook will be deleted.

☐ Copy data from `updated-branch` to `master` ?
Transactions on `master` created after `updated-branch` was created will no longer appear on datasets.

☒ Delete `updated-branch` branch on datasets after merging?
Toggling this on will delete the `updated-branch` branches on the derived datasets in this workbook. Toggling this off will still delete the branch on this workbook.

No, go back Merge into `master`

[API Reference ↗](#)

The first toggle allows you to choose whether to copy the transaction branch into the target branch. Let's imagine that after branching, we

[Send feedback](#)

work on master and committed new transactions on the derived datasets' `master` branch. If this toggle is set to True, the transactions created on master since the `updated-branch` branch was created will no longer appear on the dataset after merging.

The second toggle allows you to choose whether to delete the source branch from the datasets. Note this is different than deleting the workbook branch itself, which is always done with a merge and not configurable. If the second toggle is True, the source branch will be deleted from the derived datasets created in the Workbook.

Completing a merge may take some time. After the merge is completed, the branch you just merged will be deleted automatically, unless it still has child branches.

Analytics / Code Workbook [Legacy] / Branching / Choose imported dataset branch

Choose imported dataset branch

Code Workbook implements branch *fallbacks* for imported datasets, which means that the branch hierarchy in your Workbook will be used to determine where imported datasets should be pulled from.

For example, suppose your Workbook contains two branches, `master` and `develop`, and your input dataset `titanic` also has both of these branches. When you are on the `develop` branch, the imported dataset will default to pull from `develop`. If there is another dataset that only exists on `master` and not `develop`, input data will be pulled from `master`.

This fallback structure enables useful workflows. For example, if you have imported datasets from another Workbook or a repository and you want to test your transforms with a branched version of that Workbook or repository, you can simply create a new branch with the same name in your Workbook, and imported datasets will fall back to the appropriate branch automatically.

You can also manually choose a branch for imported datasets by clicking on any imported dataset, and choosing a specific branch from the dropdown menu:

API Reference ↗



Send feedback



IMPORTED DATASET

Dataset version based on branch:

Select input branch... ▾

No branches found for this dataset.

Q Filter...

B

C

D

master

Preview Branch Logs Description

PREVIOUS

← Merging branches

NEXT

Datasets / Optional data persistence →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

Send feedback

Optional data persistence

For each transform in a workbook, users can choose whether to save the result as a dataset.

Choose whether to save as a dataset

By default, new transforms are not saved as datasets. To save them, use the toggle in the logic pane.

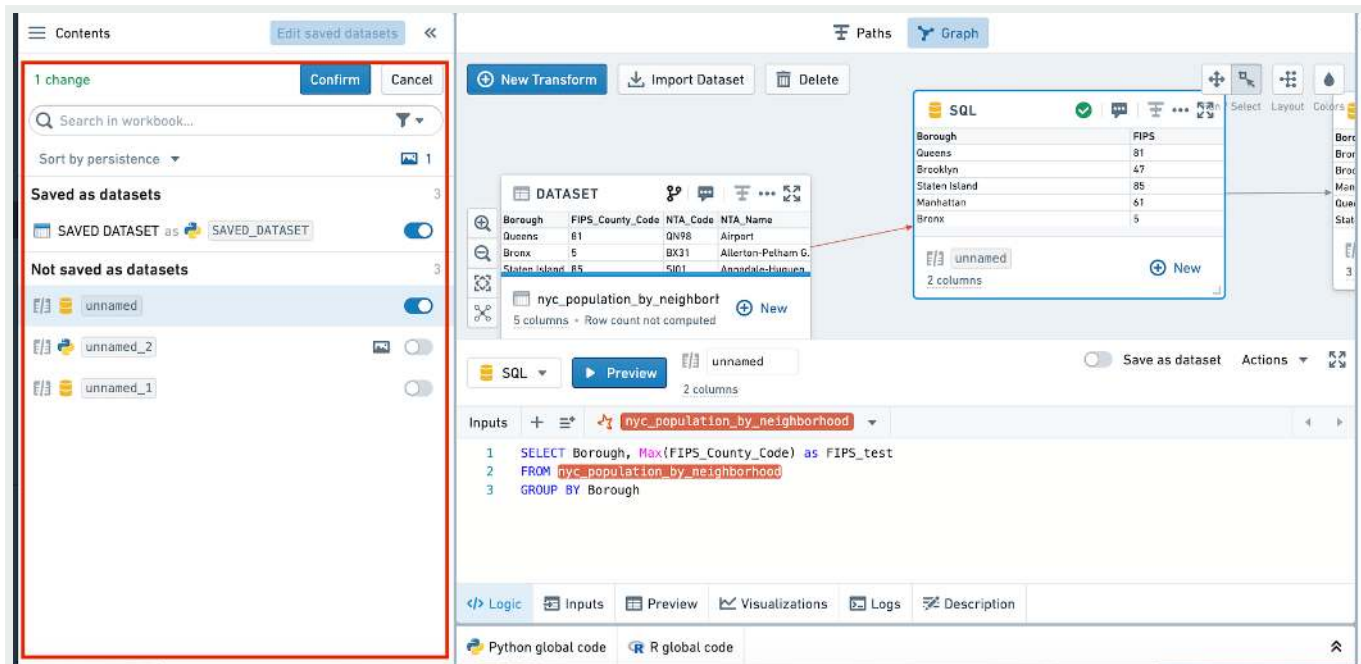
The screenshot displays the Palantir Code Workbook interface. On the left, a 'DATASET' pane shows a table with columns: medallion, hack_license, vendor_id, and rate_code. Below this, a 'taxi_trips' dataset is listed with 25 columns. A red arrow points from this dataset to a 'PYTHON' transform block. The 'PYTHON' block contains a table with the same columns and a 'SAVED DATASET' section below it, labeled 'SAVED_DATASET' with 25 columns. A red box highlights the 'Save as dataset' toggle switch in the bottom right corner of the interface, which is currently turned on. A tooltip message states: 'When disabled, results of executions will not be saved to a dataset.'

These screenshots use open source data from the [NYC Taxi & Limousine Commission](#).

To change the persistence of multiple transforms at once, use the bulk editor on the left-hand side.

[API Reference](#)

Send feedback

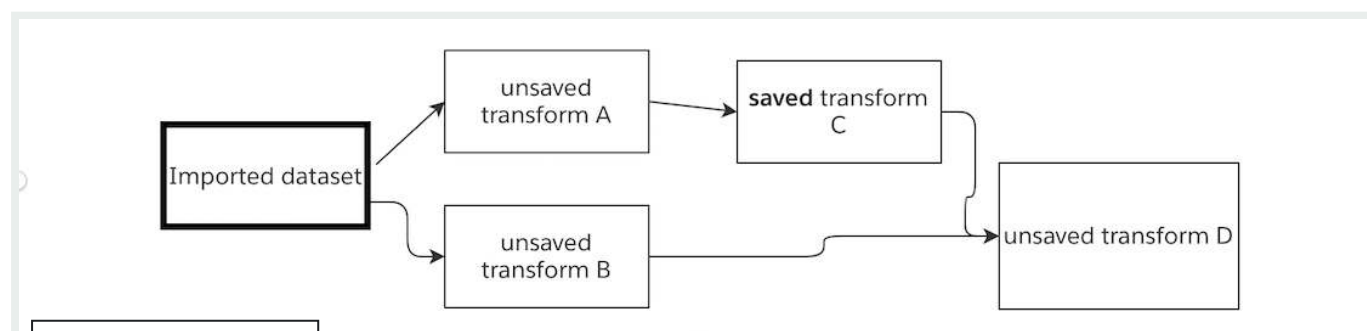


- i** If you choose to change a transform from not saved to saved, it will re-link to its previous saved dataset. If a previous saved dataset does not exist, a new dataset will be created.

Transforms that are saved are denoted with a horizontal blue bar.

Execution Model

When running a node, the logic from all unpersisted nodes upstream of that node will also be run. In the diagram below, when running Saved Transform C, the logic from Unsaved Transform A will also be executed.

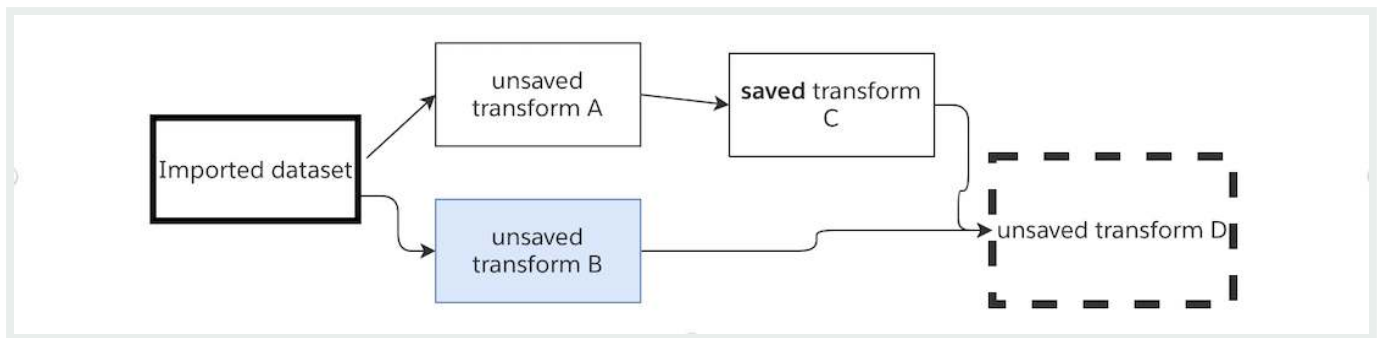


[API Reference ↗](#)

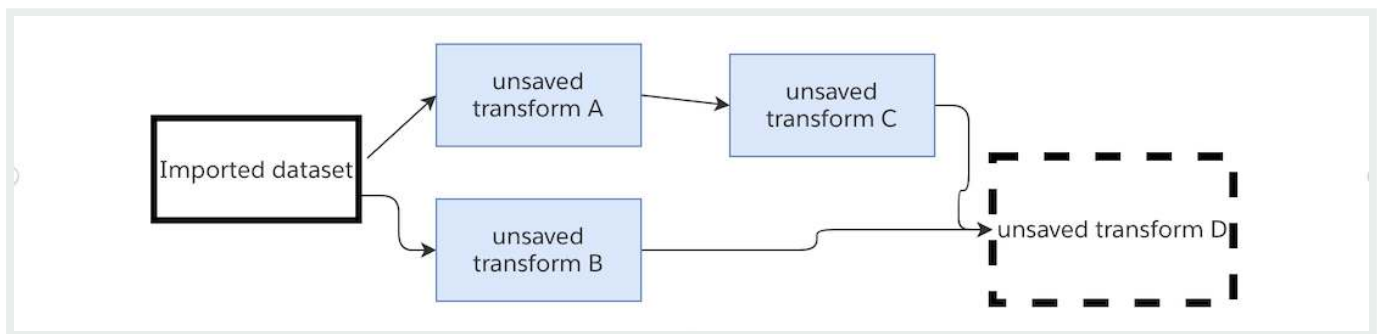
If you change the code in Unsaved Transform A but do not run it, and then run Saved Transform C, the result of Saved Transform C will reflect the change in

[Send feedback](#)

On running Unsaved Transform D, Saved Transform C will be read in from the Foundry dataset, and the logic from Unsaved Transform B will be executed.



Let's imagine we toggle Saved Transform C so it's no longer saved as a dataset. When running Unsaved Transform D, the logic of all three upstream transforms will be executed.



In this case, when running this series of transformations - if my end goal is to view the result in Unsaved Transform D, **there is no need to preview the 3 upstream unsaved transforms**. By running Unsaved Transform D, the latest logic in all four transforms and the latest transaction for the imported dataset will be used.

FAQ

When should I save a transform as a dataset?

If a transform is very computationally intensive, and is used upstream of many other transforms, you may want to save it as a dataset, to prevent poor performance.

If [API Reference](#) results of a transform outside of workbook (for example, in another Code workbook or in a Contour analysis), you should save the results of the transform as a dataset.

Send feedback

If a transform computes a function nondeterministically (for example, using a `row_number` function or a function that calls the current time), you should persist the dataset to Foundry. This will guarantee that downstream transforms will use the exact results that were written to the dataset.

Checkpoint your work

If your workbook contains long chains of nodes that are unpersisted, it is recommended to periodically checkpoint your workbook by persisting an intermediate node.

When should I preview a node?

In general, the preview functionality should be used when creating a series of transforms, to validate their correctness and preview their results. Once a series of transformations is codified, there are fewer reasons to use the preview functionality.

Why don't my unsaved transforms appear in Data Lineage?

Unsaved transforms in Code Workbook are logical blocks, not resources in a Project. When you Explore Data Lineage, datasets will show all of the code they execute. That includes the code in any unsaved transforms upstream of the saved transform.

For example, this workbook contains one unsaved transform and one saved transform.

[API Reference ↗](#)

[Send feedback](#)

The screenshot shows the Data Lineage Explorer interface. At the top, there are tabs for 'Paths' and 'Graph'. Below the tabs, there are buttons for 'New Transform', 'Import Dataset', 'Manual Entry', and 'Delete'. The main area displays a workflow starting with a 'DATASET' node named 'Cocktails' (9 columns, 687 rows). This node is connected to a 'PYTHON' node named 'selection' (3 columns, 687 rows). The 'selection' node is then connected to another 'PYTHON' node named 'limiting' (3 columns, 5 rows). The 'selection' node's code is shown in the bottom panel:

```
1 def selection(Cocktails):
2     return Cocktails.select('Bartender', 'BarCompany', 'Location')
```

The 'limiting' node's code is also shown in the bottom panel:

```
1 def limiting(selection):
2     unnamed = selection
3     return unnamed.limit(7)
```

The bottom panel also includes a 'Preview' button and a 'Save as dataset' toggle.

If you click Explore Data Lineage, it will appear as below. Note that the code for the `selection` transform is prepended to the code in the `limiting` transform.

The screenshot shows the Data Lineage Explorer interface with a workflow from 'Cocktails' to 'limiting'. The 'Cocktails' node is a purple box, and the 'limiting' node is a green box. The workflow is shown in the center of the interface. The bottom panel shows the code for the 'limiting' transform, which includes the code from the 'selection' transform:

```
1 def selection(Cocktails):
2     return Cocktails.select('Bartender', 'BarCompany', 'Location')
3
4 def limiting(selection):
5     unnamed = selection
6     return unnamed.limit(7)
```

The bottom panel also includes a 'Preview' button, a 'History' button, and a 'Code' button. The 'Code' button is selected, and the code is displayed in the bottom panel.

What are the performance and resource usage effects of optional data persistence?

API Reference ↗

Send feedback

Code Workbook originally used an execution model where each transform was saved as a Foundry dataset. The current execution model allows users to choose whether to save transforms as datasets, and there are implications for performance and resource usage.

Scheduled builds

Let's imagine you had a scheduled build running in the previous execution model. In the new execution model, if all transforms remain persisted, the performance will remain identical to before.

If you determine that some intermediate transforms do not need to be saved as Foundry datasets, and choose to unpersist them, there will be strict performance improvements in the speed of the pipeline. This is because the intermediate results of these nodes no longer need to be written to Foundry, and in some cases can help the Spark query planner further optimize downstream computations.

Interactive use

In the interactive case, in the old execution model both the preview result and the write result were computed. With optional persistence, unpersisted nodes compute only a preview, and persisted nodes only compute a write. Thus, no duplicative work is done and running an identical workbook will use less resources in the new execution model than in the previous execution model.

The effects of optional persistence on performance in the interactive case are more nuanced. For workflows with a very computationally intensive transform (for example, a large join), you may want to persist the join to avoid recomputing the large join each time you run a downstream transform.

In all cases, when you decide to unpersist a transform, storage space is saved by not writing to Foundry.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Project references

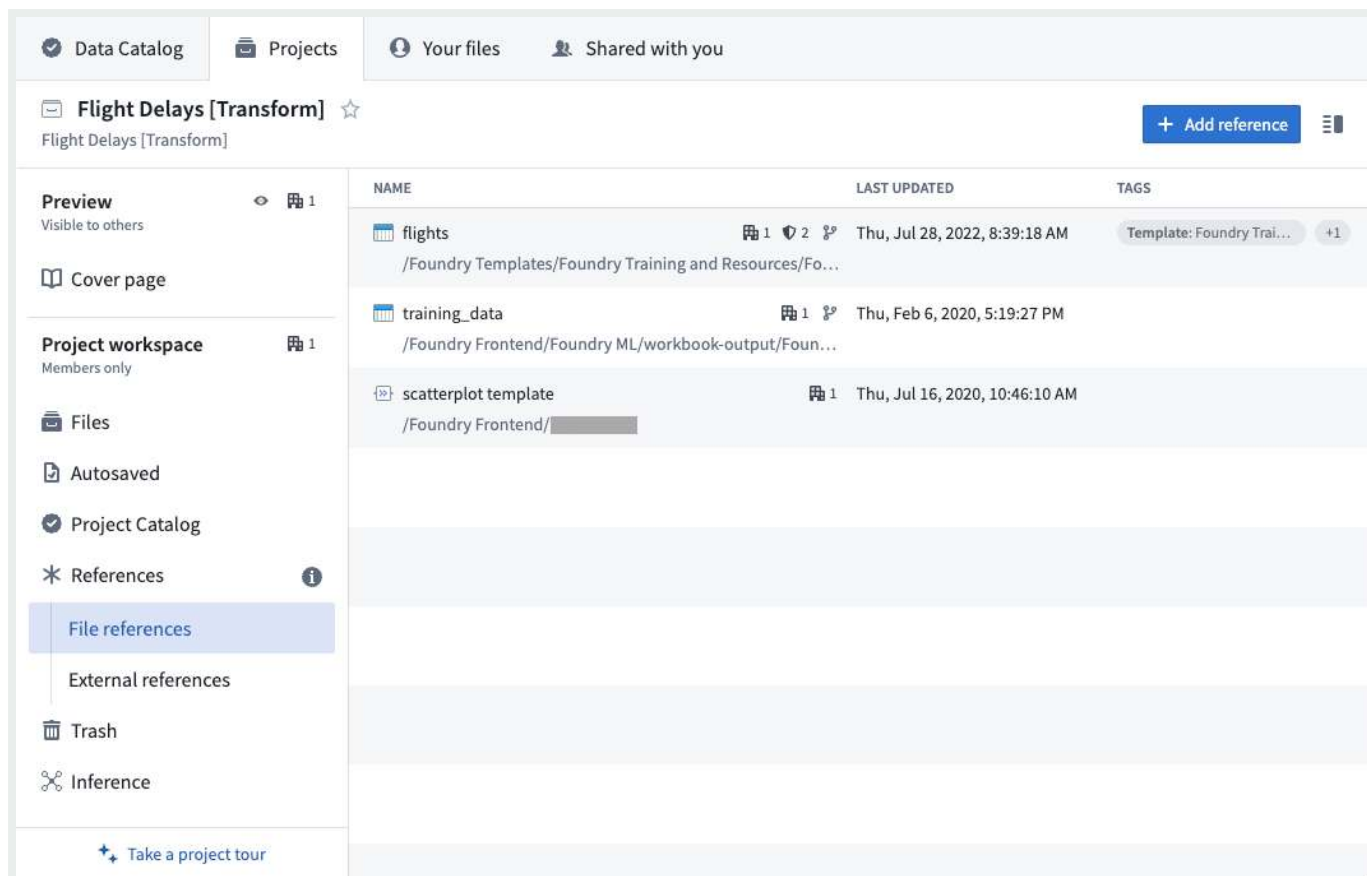
In Foundry, [Projects](#) define both a conceptual boundary around related work, and a security boundary for applying and managing access. Using data across project boundaries must be treated with extra care.

Referencing a resource

Project references provide a mechanism for users with higher permissions - typically the owners of a dataset or pipeline - to allow the discovery and use of data in other projects. They add an increased layer of scrutiny for moving data between projects by explicitly acknowledging when a dataset is imported into a project.

To reference a resource, go to the [Project navigation panel](#) at the root level of the project. Click the **+Add** button to add a reference to a dataset or template. In the below image, the datasets `flights` and `training_data` and the Code Workbook template `scatterplot template` are added as references in the project. This means that Code Repositories and Code Workbooks in the project can use `flights` and `training_data` as an input, and the `scatterplot template` template can be used by Code Workbooks in the project.

[API Reference ↗](#)[Send feedback](#)



- To reference a resource, you must have `compass:import-resource-from` on the resource (usually expanded from the Viewer role) and `compass:import-resource-to` on the destination project (usually expanded from the Editor role).
- These roles can be customized using Custom Roles.

Project-scoped Workbooks

All new Code Workbooks are project scoped, meaning they respect project boundaries and use a token scoped to the project.

In a project-scoped workbook:

- All input datasets and templates must be in project scope: they must be in the same project as the workbook, or added as a reference in the project.
- All output datasets must be in the same project as the workbook.

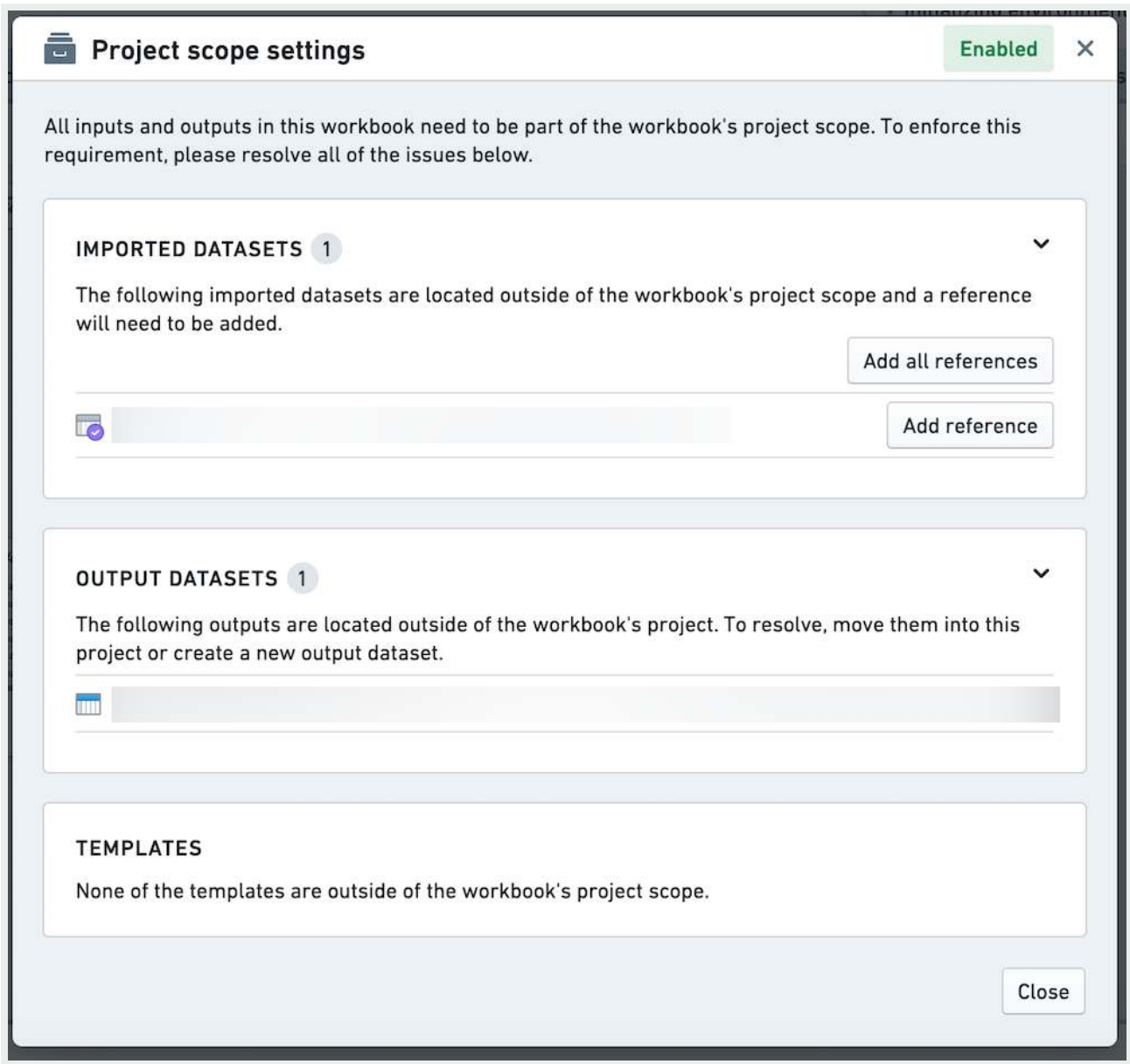
API calls to Foundry services using the job token will fail.

API Reference ↗

If there are any noncompliant inputs or outputs in a project-scoped workbook, transforms will fail to build and their job specs will not update.

Send feedback

In a project-scoped workbook, noncompliant inputs and outputs will be listed in the project scope dialog. If you have the necessary permissions, you can add references to out-of-scope input datasets and templates from the dialog. Alternatively, you can use the Project Summary Panel at the root of the project.



In a project-scoped workbook, noncompliant inputs and outputs are also denoted with a red icon.

Enabling project scoping on your workbook

[API Reference ↗](#)

For workbooks created prior to project scoping being enabled by default, you will be prompted to enable project scoping. The project scope dialog will list [Send feedback](#) inputs and outputs on the selected branch. You must resolve these issues prior to enabling

project scoping on the master branch. **If you do not enable project scoping, the workbook will continue to run with the user token as before. You will be able to continue to work normally, edit, and run builds in the workbook. You will still be able to reach Foundry APIs, use imported datasets that are out of project scope, etc.**

Project scope settings

Disabled ✕

All inputs and outputs in this workbook need to be part of the workbook's project scope. To enforce this requirement, please resolve all of the issues below.



Once project scoping is enabled, builds will fail if any inputs or outputs are outside of the project scope, and the job token will no longer have permissions to make external service calls.

IMPORTED DATASETS 1 ▼

The following imported datasets are located outside of the workbook's project scope and a reference will need to be added.



Add all references

Add reference

OUTPUT DATASETS

None of the outputs are outside of the workbook's project.

TEMPLATES

None of the templates are outside of the workbook's project scope.

Enable

Close

Project scoping can only be enabled on the master branch. Once you have enabled project scoping on the master branch, other branches may still have non-compliant inputs or outputs. These will be listed in the project scope dialog on those branches. Once you have enabled project scoping, there is no way to disable it through the UI.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Code repositories / Comparison: Code Repositories vs. Code Workbook vs. Code Workspaces

Comparison: Code Repositories vs. Code Workspaces vs. Code Workbook

Foundry has three products available for writing code-based data transformations: Code Workbook, Code Workspaces, and Code Repositories. While there is some feature overlap between these products, each is geared toward distinct workflows and user types. The guide below is intended to help you determine which tool is best suited to your needs.

[Code Repositories](#) is recommended for creating robust production pipelines and supporting workflows that require an additional layer of governance and scrutiny. With Code Repositories, data engineers can create efficient pipelines in bulk. Example workflows that are a good fit for Code Repositories include:

- A daily pipeline at high data scale which requires incremental compute.
- A high-visibility pipeline with strict governance requirements to be able to revert to previous versions of historical code, or gate code changes on unit tests passing.

[Code Workspaces](#) is recommended for quick and efficient exploratory analyses using JupyterLab® and RStudio® Workbench to combine familiar IDEs with the benefits of the Foundry platform, such as data security, branching, build scheduling, and resource management. Example workflows that are a good fit for Code Workspaces include:

API Reference ↗ cell data analysis and exporting its contents to a shareable report
a transformation pipeline or a machine learning model



Send feedback

[Code Workbook](#) is recommended for performing code-based analyses on high-scale data that would not otherwise be suitable for Code Workspaces. These analyses can be for one-time use or could produce an artifact that is updated on a recurring basis. Code Workbook can also be used to prototype pipelines, which can then be [promoted to repositories](#). Example workflows that are a good fit for Code Workbook include:

- Investigating the results of a clinical trial by testing out different p-values.
- Creating interactive visualizations to share with others.

In addition to the code-based products above, [Pipeline Builder](#) is the Palantir platform's primary no-code application for building and maintaining production data pipelines. Pipeline Builder uses a graph and form-based interface, enabling users to integrate data and create business logic transformations without writing code. If you are evaluating whether to use Pipeline Builder or Code Repositories for your pipeline, see [Considerations: Pipeline Builder and Code Repositories](#).

Comparison summary

	Code Repositories	Code Workspaces	Code Workbook
Features	Advanced pipelines	Exploratory analysis	Advanced analysis
	Enables complex workflows in long-lasting data pipelines with flexibility in performance optimization and code generation.	Enables interactive exploratory workflows using familiar IDEs tied with Foundry primitives.	Enables data analysis workflows with support for common analytical languages and visualization libraries.
Languages supported	Python, SQL, Java, Mesa	Python, R	Python, R, SQL
Environments	All environments	Kubernetes environments only	All environments
API Reference ↗			
Batch pipeline support	Yes	Yes	Send feedback

	Code Repositories	Code Workspaces	Code Workbook
Incremental computation	Yes	No	No
Transform generation	Yes	No	No
Multi-output transforms	Yes	Yes	No
Filesystem access	Yes	Yes	Yes
Visualization support	No	Yes	Yes
Iteration cycle	Iterate on code logic	Iterate on data discovery and analysis	Iterate on insight generation
	Designed to help iterate on code logic. Runtime debugger and previews can assist in validating transform logic. Data can be analyzed in Foundry after building.	Designed to help rapidly iterate on data discovery and analysis using widely known tools that seamlessly integrate with the rest of Foundry.	Designed to help generate insights from data; all transforms run on the full input data, interactive console enables ad-hoc queries, and Spark execution model is optimized for quick iteration.
Full data preview	Preview data sample, with the ability to pre-filter the input sample	Full data preview	Full data preview
Debugger	Yes	No	No
Console	In debug mode	Yes	Yes
API Reference ↗	Spark modules initiated at the job level	Spark-less environment for fast feedback loop	Spark modules kept
management			Send feedback interactivity, and

	Code Repositories	Code Workspaces	Code Workbook
			initiated at the workbook level
Operations	Data pipeline management	Data exploration management	Data analysis management
	Supports Foundry data management libraries and publishing custom Python libraries	Fully adjustable environment that can consume pip, CRAN, and conda libraries, including those published from Code Repositories	Can consume custom libraries published from Code Repositories; users can save pieces of logic as code templates, enabling point-and-click analysis by other users.
Data Expectations	Yes	No	No
Publish custom libraries	Yes	No	No
Consume custom libraries	Yes	Yes	Yes, for some environments
Point-and-click code templates	No	No	Yes
Change management	Governance	Flexibility	Rapid changes

	Code Repositories	Code Workspaces	Code Workbook
	Prioritizes change traceability and governance to ensure that critical pipelines remain secure and robust; advanced review and approval workflows and complete changelogs.	Prioritizes rapid and flexible iteration with full branching support and automatic Git versioning.	Prioritizes rapid iteration and collaboration with a lightweight branching workflow; does not require CI checks or unit testing.
Full Git workflow	Yes	Yes	No
Copy data after merge	No	No	Yes
Administer and remove security markings	Yes	No	No
Impact analysis views	Yes	No	No
Advanced code review workflows	Yes	No	No
Unit testing	Yes	No	No

► Table summary

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Analytics / Code Workbook [Legacy] / Code repositories / Exporting to a Code Repository

Exporting to a Code Repository

Python and SQL code can be exported from a Code Workbook to a Code Repository. Moving code across to a Code Repository can be useful for production pipelines, as it provides full git version control and several advanced pipelining tools. [Learn more about Code Repositories.](#)

When exporting, a new branch will be created on the repository with Code Workbook code converted to its Code Repository equivalent. Because this is a best-effort conversion, you may need to adjust this code to move files around, change dataset names and paths, adjust imported packages, etc.

i The export is a one-time, one-way export. Further edits to the Code Workbook are not automatically pushed to the Code Repository, and subsequent exports of the same datasets into the same repository are not supported. Code cannot be exported from a Code Repository to a Code Workbook. Additionally, the user who is performing the export must have at least Editor permissions on the destination Code Repository.

Supported Types

Currently, only SQL nodes or Python code nodes with Pandas or Spark dataframe inputs and outputs are supported. The set of exported nodes must be connected to each other, with imported datasets at the root.

[API Reference ↗](#)

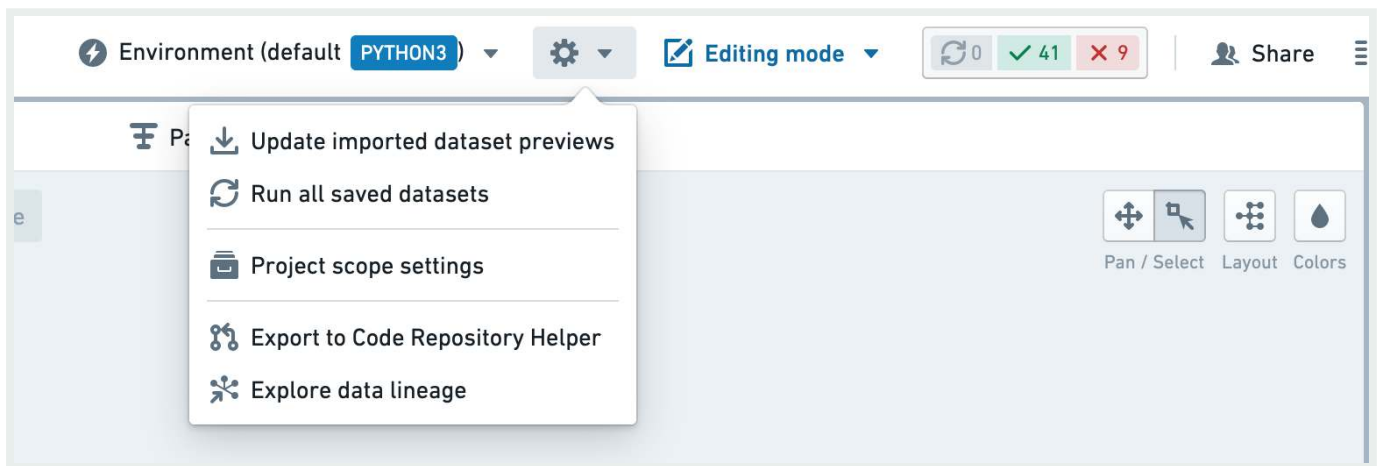
Not supported: input types of Python transform input or object are not supported

[Send feedback](#)

- i** Before exporting your code to a Code Repository, ensure that the needed languages are supported in the Code Repository. For example, if you export both SQL and Python nodes, you may need to add a new subproject to your SQL-only or Python-only repository.

How to Export

To export, from a Code Workbook, go to the Settings cog > Export to Code Repository Helper:



Click Select Repository and choose a Code Repository:

[API Reference ↗](#)

[Send feedback](#)



Workbook 

File ▾ Help ▾ |  Output ▾  Branch: master ▾

 Export to Code Repository 

Select datasets from this workbook to export to a code repository through a pull request.

An export must include a connected set of Python datasets. Selection of a node will select all valid upstream nodes, and deselection will deselect all valid downstream nodes.

CODE REPOSITORY

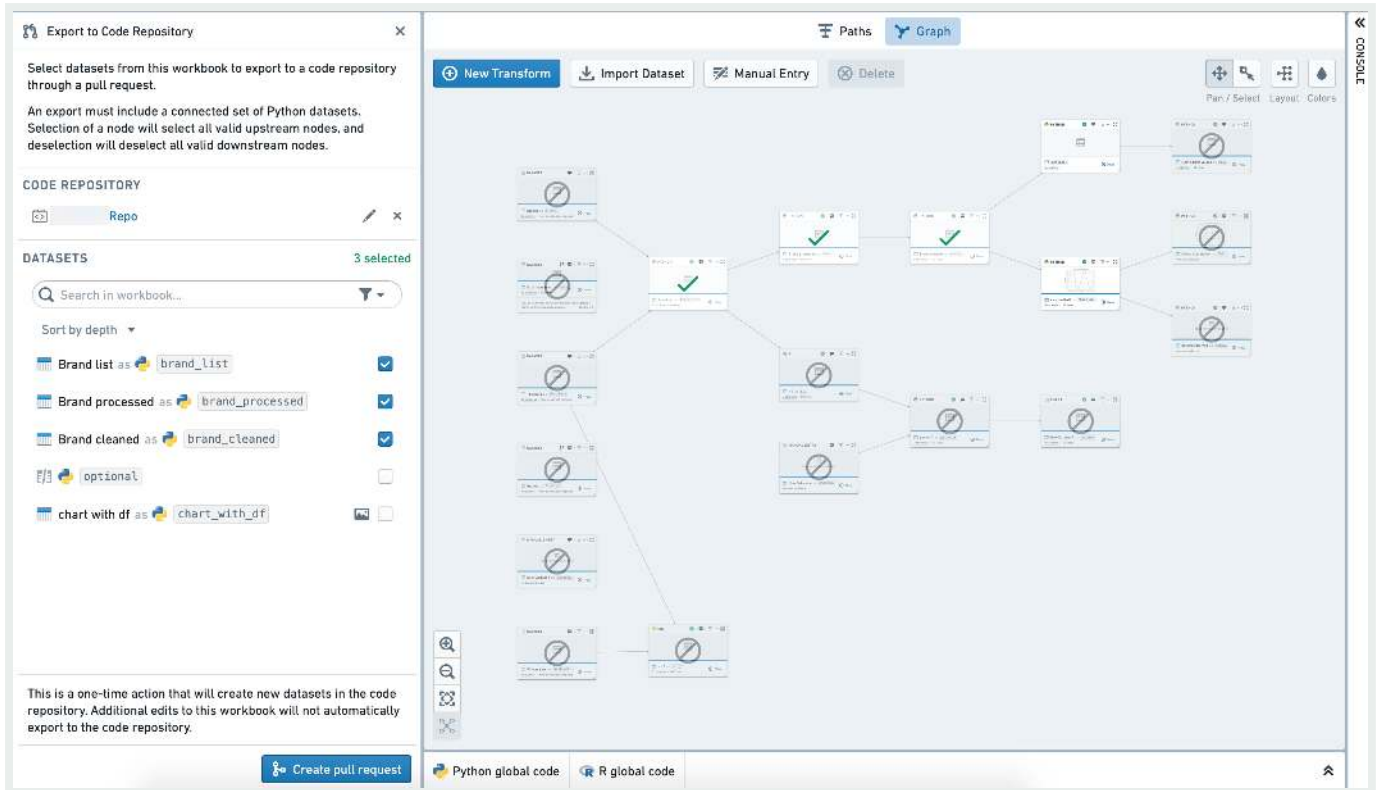
DATASETS

Select datasets to export with the checkboxes in the left hand side panel.

The graph shows which datasets are available for export (white), and which are not available (gray). The export must be a connected graph, and so adding nodes to the selection may make more nodes available for export. Hovering over disabled nodes will explain why they can't be selected.

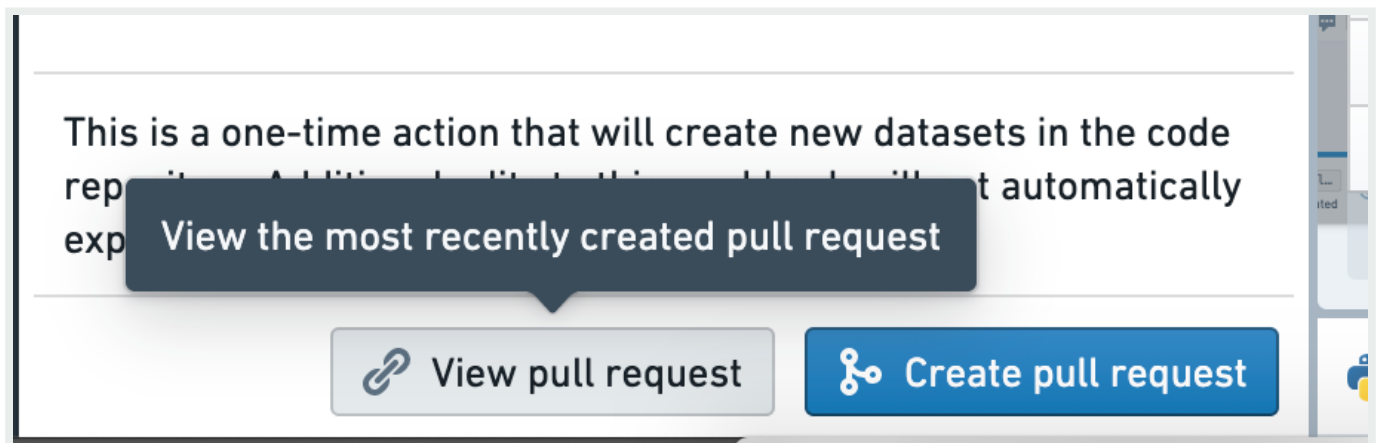
[API Reference ↗](#)

[Send feedback](#)



When ready to export, click Create pull request

A new branch will be created in the selected Code Repository with code exported from the Code Workbook. Click "View pull request" to open the pull request in the Code Repository.



From here you can inspect the exported code and make any required edits such as dataset paths and names.

[API Reference ↗](#)

[Send feedback](#)

```
NEW transforms-python/src/codeWorkbookExport/global_code.py
1 # Functions defined here will be available to call in
2 # the code for any table.
3 from pyspark.sql import functions as F
4
5 import pandas

NEW transforms-python/src/codeWorkbookExport/pandas_1.py
1 # Global imports
2 from pyspark.sql import functions as F
3 import pandas
4 from codeWorkbookExport.global_code import *
5
6 # Local imports
7 from transforms.api import transform, Input, Output
8
9 @transform(
10     pandas_1Output=Output("/Public/kamran/DIW/datasets/pandas_1"),
11     python_1=Input("/Public/kamran/DIW/datasets/python_1")
12 )
13 def pandas_1_transform(python_1, pandas_1Output, ctx):
14     pandas_1Output.write_dataframe(ctx.spark_session.createDataFrame(pandas_1
15     (python_1.dataframe().toPandas())))
16
17 def pandas_1(python_1):
18     df = python_1
19     # df = pandas.DataFrame(python_1)
20     df["concat"] = df["Name"] + df["Colour"]
21
22     return df
23

NEW transforms-python/src/codeWorkbookExport/python_1.py
1 # Global imports
2 from pyspark.sql import functions as F
3 import pandas
4 from codeWorkbookExport.global_code import *
5
6 # Local imports
7 from transforms.api import transform, Input, Output
8
9 @transform(
```

Verify the exported datasets in transforms-python/src/codeWorkbookExport have been imported and added to your pipeline in transforms-python/src/myproject/pipeline.py as desired.

```
MOD transforms-python/src/myproject/pipeline.py
***
5
6 my_pipeline = Pipeline()
7 my_pipeline.discover_transforms(datasets)
8
9 # Imports from vector export
10 import codeWorkbookExport
11 my_pipeline.discover_transforms(codeWorkbookExport)
```

Any custom package versions in the Code Workbook will be added to transforms-python/conda_recipe/meta.yaml.

i Code Workbook and Code Repositories do not support the exact same set of packages. While the majority of packages should work correctly, some may fail to pass repository checks.

[API Reference ↗](#)

[Send feedback](#)

```
MOD transforms-python/conda_recipe/meta.yaml

***
15
16 # Any extra packages required to run your package.
17 run:

18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33 # Existing imports
34
35
36
37 build:
38 script: python setup.py install --single-version-externally-managed --record
=record.txt

18 - python
19 - transforms {{environ.get('PYTHON_TRANSFORMS_VERSION', '')}}
20
21 build:
22 script: python setup.py install --single-version-externally-managed --record
=record.txt

15
16 # Any extra packages required to run your package.
17 run:
18 # Imported to match Vector environment
19 - scipy
20 - gensim
21 - foundry-python-objects <1.0,>=0.17.1
22 - scikit-learn
23 - nltk
24 - numpy
25 - nomkl
26 - foundry-ml-toolkit
27 - pandas
28 - pyarrow <0.11.0|>=0.12
29 - seaborn
30 - matplotlib
31 - foundry_ml
32 - pandas <1.0.0
33 # Existing imports
34 - python
35 - transforms {{environ.get('PYTHON_TRANSFORMS_VERSION', '')}}
36
37 build:
38 script: python setup.py install --single-version-externally-managed --record
=record.txt
```

Once ready, the PR can be created and the usual process followed to merge the exported code into the pipeline.

Writing to the same datasets

By default, exporting to Code Repository will write to new datasets. If you would prefer to write to the same datasets as the Code Workbook, follow these steps below:

1. Navigate to the branch created in repository by the Export to Code Repository action. Change the output dataset paths to those of the desired dataset.
2. For each of the datasets, navigate to the `Details` tab of the dataset page, and delete the dataset's job spec. This is necessary to allow the repository to take ownership of the dataset.
3. Create the PR and follow the usual process to merge the code into the pipeline. When the CI checks run in the repository, new job specs will be created on the target dataset.

Note that by transferring these datasets to the repository, **you will no longer be able to use the original Code Workbook to write to the datasets**. You may wish to delete the exported nodes to avoid confusion.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

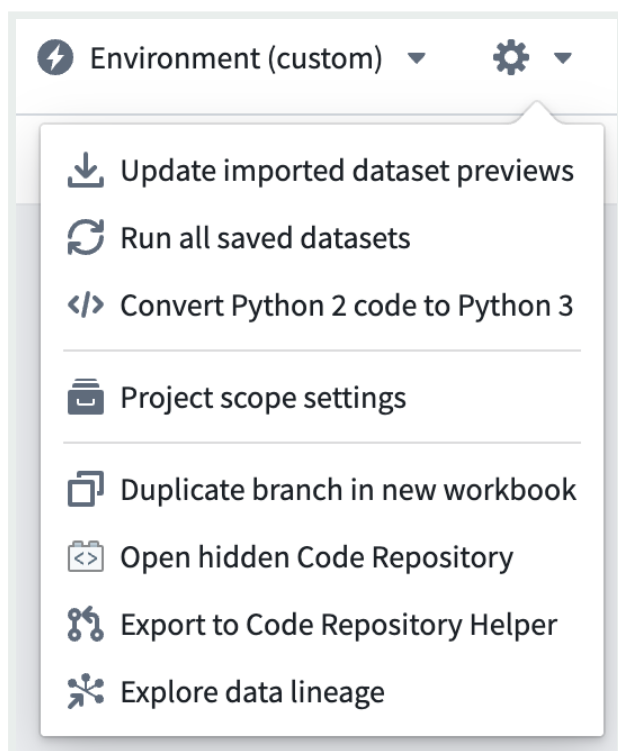
[Send feedback](#)

Analytics / Code Workbook [Legacy] / Code repositories / Hidden code repositories

Hidden code repositories

Code Workbook offers lightweight version control through the use of [branches](#). Additionally, every workbook is backed by a special hidden code repository. This repository serves as a secure backup of the code written in a code workbook while also exposing the history of all code changes made on the workbook.

You can access the hidden code repository of a workbook by opening the gear icon menu at the top right of the Code Workbook interface and selecting **Open hidden Code Repository**.



[API Reference ↗](#)

⌵ 🔍 | properties

[Send feedback](#)

A workbook's hidden repository will always have the following properties:

- **The repository is read-only:** Repository contents can only be viewed and never updated directly in the repository. The only way to update the repository is to make code changes in the workbook.
- **The repository is hidden by default:** It will only be discoverable through the Code Workbook interface.
- **The repository stores three separate files:** The `pipeline.R`, `pipeline.py`, and `pipeline.sql` files each contain all of the code of the converted workbook for their respective language.
- **The repository saves a full change history:** A full history of the workbook's code changes for each branch is available under the **Branches** tab of the hidden repository.
- **The repository contains a hidden `workbook.yml` file:** The file stores basic metadata about the workbook.

Every code change made on a workbook branch automatically creates a new commit to the corresponding branch in the hidden code repository.

Code conversion

Code Workbook code, when committed to the hidden repository, is automatically converted to Code Repository syntax. For example, consider the following code cell in Code Workbook:

```
1 def rename_column(dataset):
2     return dataset.withColumnRenamed("old_name", "new_name")
```

The code will be converted to the following in the `pipeline.py` file of the hidden repository:

```
1 @transform_pandas(
2     Output(rid="ri.foundry.main.dataset.id-1"),
3     dataset=Input(rid="ri.vector.main.dataset.id-2")
4 )
5 def rename_column(dataset):
6     return dataset.withColumnRenamed("old_name", "new_name")
```

API Reference ↗

Send feedback

If more than one code cell of a given language is present in a code workbook, each of the code cells will be appended in the same, unique file: `pipeline.R` for R, `pipeline.py` for python, and `pipeline.sql` for SQL. This allows you to view all of the code of a given language in a single file. Code written in the **Global code** section of the workbook will also be stored in the appropriate files.

Recover lost code

By regularly storing backups of a workbook, hidden code repositories are the recommended way to restore code that was lost or accidentally removed. To consult the history of a given workbook's branch, open the **Branches** tab at the top of the hidden repository, select the desired branch (such as `master`), and select the commit for which you would like to inspect code changes. You can then copy the code and paste it back into the workbook.

PREVIOUS

← [Exporting to a Code Repository](#)

NEXT

[PySpark reference](#) →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement](#) ↗

[Privacy Statement](#) ↗

[Terms of Use](#) ↗

—[Cookies Settings](#)

[API Reference](#) ↗

[Send feedback](#)

PySpark reference

You can browse the documentation on PySpark functionality in the Transforms documentation using the links below:

- [Overview](#)
- [Coming from Python](#)
- [Syntax cheat sheet](#)
- [Style guide](#)
- [Columns](#)
- [Queries](#)
- [User Defined Functions](#)
- [Filtering](#)
- [Dates & Timestamps](#)
- [Strings](#)
- [Math](#)
- [Joins](#)
- [Aggregation & Pivot Tables](#)
- [Window](#)
- [Logging](#)
- [Other](#)

PREVIOUS

← [Code repositories / Hidden code repositories](#)

NEXT

[SparkR reference](#) →

[API Reference ↗](#)



Palantir Technologies Inc. All rights reserved.

[Send feedback](#)

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

SparkR reference

Introduction to SparkR

Code Workbook allows users to use both Spark R and native R. Spark R provides a distributed data frame implementation that supports operations like selection, filtering, and aggregation on large datasets. While users may be more familiar with native R, it is recommended that users first use SparkR to filter large datasets before using native R.

Common SparkR operations

Read the full [API documentation ↗](#) for SparkR to see all possible operations. Below, we outline syntax for common operations.

Filtering

Filter expressions can be a SQL-like WHERE clause passed as a string.

```
1 df_filtered <- SparkR::filter(df, "numeric_col > 10")
```

You can also use column expressions similar to standard R syntax.

API Reference ↗

```
df_filtered <- SparkR::filter(df, df$numeric_col > 10)
```

Send feedback

You can also use `SparkR::where` with similar syntax.

```
1 df_filtered <- SparkR::where(df, "numeric_col > 10")
```

Column operations

Subset columns using `SparkR::select()`.

```
1 df_subset <- SparkR::select(df, "column1", "column2", "column3")
```

Rename a column with `SparkR::withColumnRenamed()`.

```
1 df <- SparkR::withColumnRenamed(df, "old_column_name", "new_column_name")
```

Add new columns using `SparkR::withColumn()`.

```
1 # Add two columns
2 df <- SparkR::withColumn(df, 'col1_plus_col2', df$col1 + df$col2)
3 # Multiply a column by a constant
4 df <- SparkR::withColumn(df, 'col1_times_60', df$col1 * 60)
```

Aggregations

Use `SparkR::groupBy` and `SparkR::agg` to compute aggregates. Calling `SparkR::groupBy` will create a group by object. Pass the group by object into `SparkR::agg` to get an aggregated dataframe.

API Reference ↗

```
1 df_grouped <- SparkR::groupBy(df, "group_col1", "group_col2")
2 df_agg <- SparkR::agg(df_grouped, average_col1=avg(df$col1))
```

Send feedback

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

R Filesystem API

R TransformInput object

The interface for low level operations on a Foundry dataset.

`spark.df()`

- Returns a [pyspark.sql.DataFrame](#) of the input dataset.

`data.frame()`

- Returns an [R data.frame](#) of the input dataset.

`fileSystem()`

- Returns a *FileSystem* object for direct *FoundryFS* access.

R TransformOutput object

The interface for low level write operations on a Foundry dataset.

`write.spark.df(df, partition_cols=NULL, bucket_cols=NULL, bucket_count=NULL, sort_by=NULL)`

API Reference ▸ [DataFrame](#) to the output dataset.

Parameters

- `df` ([pyspark.sql.DataFrame](#)) – The PySpark `DataFrame` to write.

Send feedback

- **partition_cols** (*List[str], optional*) - Column partitioning to use when writing data.
- **bucket_cols** (*List[str], optional*) - The columns by which to bucket the data. Must be specified if bucket_count is given.
- **bucket_count** (*int, optional*) - The number of buckets. Must be specified if bucket_cols is given.
- **sort_by** (*List[str], optional*) - The columns by which to sort the bucketed data.

`write.data.frame(rdf)`

- Writes the given [R data.frame ↗](#) to the output dataset.

`fileSystem()`

- Returns a *FileSystem* object for direct *FoundryFS* access.

R `FileSystem` object

`ls(glob=NULL, regex='.*', show_hidden=FALSE)`

- Lists all files matching the given pattern (either `glob` or `regex`), with respect to the root directory of the dataset.
 - **glob** (*str, optional*) - A unix file matching pattern. Also supports globstar.
 - **regex** (*str, optional*) - A regex pattern against which to match filenames.
 - **show_hidden** (*bool, optional*) - Include hidden files, those prefixed with `'.'` or `'_'`.

Parameters

Returns

R array of the `FileStatus` named tuple (path, size, modified) - The logical path, file size (bytes), modified timestamp (ms since January 1, 1970 UTC)

[API Reference ↗](#), `disk_optimal=FALSE, encoding=default`)

- Open a FoundryFS file in the given mode.

Send feedback

Parameters

- **path** (*str*) – The logical path of the file in the dataset. (**Remote path**)
- **open** (*str*) – A description of the [mode](#) in which to open the connection.
- **disk_optimal** (*bool, optional*) – Controls how FoundryFileSystem handles file i/o.
- **encoding** (*str, optional*) – Defaults to the R language default (UTF-8).

Returns

An R connection object

get_path (*path, open='r', disk_optimal=FALSE, encoding=default*)

- For a given FoundryFS (remote) path, returns the local temporary path.

Parameters

- **path** (*str*) – The logical path of the file in the dataset. (**Remote path**)
- **open** (*str*) – A description of the [mode](#) in which to open the connection.
- **disk_optimal** (*bool, optional*) – Controls how FoundryFileSystem handles file i/o.
- **encoding** (*str, optional*) – Defaults to the R language default (UTF-8).

Returns

str

upload (*local_path, remote_path*)

- Upload the file from the local to the remote path. Write only.

Parameters

- **local_path** (*str*) – The local path of the file to upload.
- **remote_path** (*str*) – The logical path of the file in the dataset.

Returns

None

Advanced topic: **disk_optimal** setting

[API Reference ↗](#)

In the `FoundryFileSystem` methods `open()` and `get_path()`, the `disk_optimal` argument controls how file input and output (i/o) is handled.

[Send feedback](#)

By default, `disk_optimal` is set to `FALSE` in both `open()` and `get_path()`. In this mode, files are guaranteed to be downloaded before they are accessed.

If you choose to set `disk_optimal` to `TRUE`, files are downloaded simultaneously while the code executes. The temporary local path must be opened via `fifo()` in order to read correctly. Note that not all libraries support reading this type of file.

You may choose to set `disk_optimal` to `TRUE` when the file you are reading is very large.

For example, let's imagine we have a very large txt file and we only want to read the first 10 lines. Use the below code to print only the first 10 lines, without reading the entire file.

```
1 disk_optimal_example<- function(large_txt_file) {  
2   fs <- large_txt_file$fileSystem()  
3  
4   ## Open a connection with fifo()  
5   ## The text file is titled large_txt_file.txt  
6   conn <- fs$open("large_txt_file.txt", "r", disk_optimal = TRUE)  
7  
8   A <- readLines(conn, n = 10)  
9   print(A)  
10  return(NULL)  
11 }
```

If you want to use R TransformOutput to write a file and then read it, `disk_optimal` must be set to false.

← PREVIOUS
SparkR reference

NEXT →
Available fonts

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[API Reference ↗](#)

[Terms of Use ↗](#)

[Send feedback](#)

Available Fonts

Some plotting libraries require the developer to explicitly designate the font pack to be used when rendering text.

The full list of available fonts is below:

```

/usr/share/fonts/truetype/dejavu/DejaVuSerifCondensed-Bold.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansCondensed-BoldOblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansCondensed-Bold.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerif.ttf
/usr/share/fonts/truetype/dejavu/DejaVuMathTeXGyre.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerif-BoldItalic.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansCondensed-Oblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerifCondensed-Italic.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerif-Bold.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Oblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerifCondensed.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerif-Italic.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansMono-BoldOblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSans-ExtraLight.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSans-BoldOblique.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansCondensed.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf
/usr/share/fonts/truetype/dejavu/DejaVuSerifCondensed-BoldItalic.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerifCondensed-BoldOblique.ttf

```

API Reference ↗



Send feedback

```
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSans-Bold.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSans.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSans-Oblique.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansCondensed-Bold.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerif.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerif-BoldItalic.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansCondensed-Oblique.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerifCondensed-Italic.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansMono.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerif-Bold.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansMono-Oblique.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerifCondensed.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerif-Italic.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansMono-BoldOblique.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSans-ExtraLight.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSans-BoldOblique.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansCondensed.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSansMono-Bold.ttf
/usr/share/fonts/truetype/ttf-dejavu/DejaVuSerifCondensed-BoldItalic.ttf
/usr/share/fonts/opentype/noto/NotoSerifCJK-Bold.ttc
/usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc
/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc
/usr/share/fonts/opentype/noto/NotoSerifCJK-Regular.ttc
```

← PREVIOUS
R Filesystem API

NEXT →
Notepad / Overview

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

[API Reference ↗](#)

Send feedback

